BOSS PHONE

# 领导电话 · 设计文档集

远程指挥 AI Agent 的"领导手机" —— 设计文档合集

硬件平台 FoloToy AI Passport (ESP32-C3)

配套 PC 总机房 · CLI 插件桥 · 安卓富终端

2026-08-30 · 共 10 篇

# 目录


**1. 产品需求文档 (PRD)**

BOSS PHONE（领导电话）产品需求文档

1. 产品概述
2. 用户场景与链路形态
3. 系统架构
4. 多端联动链路系统（核心需求）
5. 功能需求
6. 交互规范
7. 通讯协议
8. 非功能需求
9. 里程碑
10. 风险与开放问题
11. 附录：硬件能力对照（BSP 现状）

**2. 总体技术设计**

BOSS PHONE 总体技术设计

1. 设计文档体系
2. 设计原则（工程视角）
3. Monorepo 仓库结构
4. 技术栈选型
5. CLI 插件兼容矩阵（核心工程风险）
6. 协议 SSOT 与代码生成
7. 关键代码骨架
8. M0 语音垂直切片（板前预研，2026-08-29 插入）
9. M1 任务分解（垂直切片，USB 先行）
10. 工程风险与待办

**3. 通讯协议规范**

BOSS PHONE 通讯协议规范

1. 设计原则
2. 消息信封（Envelope）
3. 消息字典
4. 帧封装（链路无关）
5. 能力协商（caps 词汇表）
6. 核心交互时序
7. 兼容性保证
8. 待定项

**4. 板端固件设计**

BOSS PHONE 板端固件设计

1. 组件拓扑
2. Flash 分区表（8MB）
3. 任务模型与内存预算（400KB SRAM）
4. 统一导航状态机（boss_app）

5. 链路监管器状态机（boss_transport）
6. transport 接口（骨架）
7. boss_protocol 骨架
8. 音频管线
9. 屏幕与显示
10. M1 固件交付清单
11. 风险

## 5. PC 总机房设计

BOSS PHONE PC 总机房设计（Hub）

1. 模块架构
2. 进程模型（asyncio）
3. 路由决策（core/router）
4. 离线队列与 seq
5. 接线员（operator）
6. 遥测聚合（telemetry）
7. 插件桥接口（plugins）
8. 配置与运行
9. M1 Hub 交付清单
10. 风险

## 6. CLI 插件桥设计

BOSS PHONE CLI 插件桥设计

1. 总体形态
2. 统一适配接口（plugins/core）
3. 能力矩阵与降级
4. Claude Code 适配器（M1 打样）
5. Codex 适配器
6. OpenCode 适配器
7. Trae 适配器（待调研，wrapper 兜底）
8. 安装与生命周期
9. M1 交付清单
10. 风险

## 7. 安卓 App 设计

BOSS PHONE 安卓 App 设计

1. 双重角色
2. 模块划分
3. BLE 网关（数据面）
4. 推送接入
5. 对等富终端 UI（Compose）
6. 通知中心（FR-54）
7. 前台服务与保活
8. M3 App 交付清单
9. 风险

## 8. 部署与运维指南

BOSS PHONE 部署与运维指南

1. 部署拓扑与所需软件
2. Hub 部署（PC 端）
3. 插件安装（PC 端）
4. 设备配对与配网
5. App 部署（手机）
6. 升级流程
7. 运维
8. 故障排查速查表
9. 待定项

## 9. 测试与验收方案

BOSS PHONE 测试与验收方案

1. 测试分层
2. CI 多轨
3. M1 验收清单（骨架版·USB）
4. M2 验收清单（无线+语音）
5. M3 验收清单（户外+富终端）
6. M4 验收清单（LTE 板）
7. 真机硬件测试矩阵
8. 回归与兼容测试
9. 无板开发环境

## 10. 安全设计

BOSS PHONE 安全设计

1. 威胁模型
2. 身份与鉴权
3. 链路安全
4. 隐私与数据最小化
5. 权限最小化与供应链
6. 安全矩阵汇总
7. 待定项

# BOSS PHONE（领导电话）产品需求文档

| 项目 | 内容 |
| --- | --- |
| 状态 | Draft v0.4（新增：安卓 App 对等富终端 —— web coding 工具 / 文本对话 / 复杂表单 / 完整上下文） |
| 硬件平台 | FoloToy AI Passport（ESP32-C3，400KB SRAM / 8MB Flash / 无 PSRAM） |
| 配套系统 | PC 端总机房、云中继、安卓伴侣 App、各 CLI Agent 插件 |
| 日期 | 2026-08-24 |

## 1. 产品概述

### 1.1 一句话定位

把 FoloToy AI Passport 变成一部"领导的手机"：一个微型手机系统（桌面 + 电话簿 + 备忘录 + 设置），用户以领导视角，通过打电话的隐喻（电话簿、拨号、来电、语音指令）远程指挥电脑上的多个 AI 编程 Agent（Claude Code / Codex / OpenCode / Trae 等）在各自仓库中工作。

### 1.2 核心理念

1. **板子是哑终端**：只负责显示、三键、音频输入输出。所有智能（STT、意图理解、Agent 调度）都在 PC 端。这是 ESP32-C3 硬件约束（400KB SRAM、无 PSRAM）下的必然选择，也是产品可维护性的来源——Agent 逻辑更新不需要烧固件。
2. **一套协议，四种传输**：USB 有线、局域网 WiFi、BLE 伴侣、（未来）板载 LTE 共用同一套消息协议，差异全部下沉到传输层。兼容性是架构的自然结果，不是补丁。
3. **三端联动，而非单端切换**：设备、手机、电脑各自感知环境并协同决策链路路由。设备在局域网内时，手机零负担（不收任何推送）；插着 USB 时一切流量走有线，最稳且零电池消耗。
4. **手机系统体验**：不做单一功能的"工具屏"，而是完整的小型手机系统——状态栏、桌面、应用宫格、事件弹窗层。界面不单调，且天然可扩展新应用。
5. **多终端对等分层**：板子 = 随身快捷终端（语音 + 三键，低摩擦）；手机 App = 富终端（文本输入、完整上下文、直接对话，即一个 web coding 客户端）；PC = 全功能开发环境。三者跑**同一套协议**，功能按终端能力自然分层——板子做不了的复杂操作（打字、看长文、多任务并行）下沉到手机，手机不被浪费。
6. **领导体验优先**：一切交互隐喻为"打电话"。用户不需要理解"Agent / 会话 / 权限请求"这些术语，只需要认识"联系人"和"红黄绿灯"。

### 1.3 目标用户

- 同时用多个 AI CLI Agent 管理多个仓库的开发者（个人使用为主）。
- 希望离开电脑（挂脖在身、在屋里走动、出门在外）仍能：掌握 Agent 状态、批准权限请求、下达语音指令、收到异常报警的用户。

### 1.4 核心体验叙事（用户故事）

> 早上到工位，把挂脖的"手机"插上 Type-C——屏幕常亮，进入"工位模式"：顶部状态栏显示充电中·USB 链路，桌面红黄绿总览一眼看清三个仓库的状态。按两下进电话簿，拨给接线员："喂，让小张把测试跑一遍。"顺手在备忘录里录一条"下午提醒我看内存泄漏"。 中午拔线去食堂，板子自动切 WiFi；走出公司大门 WiFi 断开，蓝牙自动回连口袋里的手机，全程无感。 下午人在楼下便利店，口袋里的手机震了一下——repo-B 等你批权限，屏幕亮起来电界面。长按 OK

> 说一句"允许"，继续挑咖啡。 回到家插上线，备忘录自动同步到 PC，转写文字回填标题，其中"内存泄漏"那条可一键派发给 repo-A 的小张。

## 2. 用户场景与链路形态

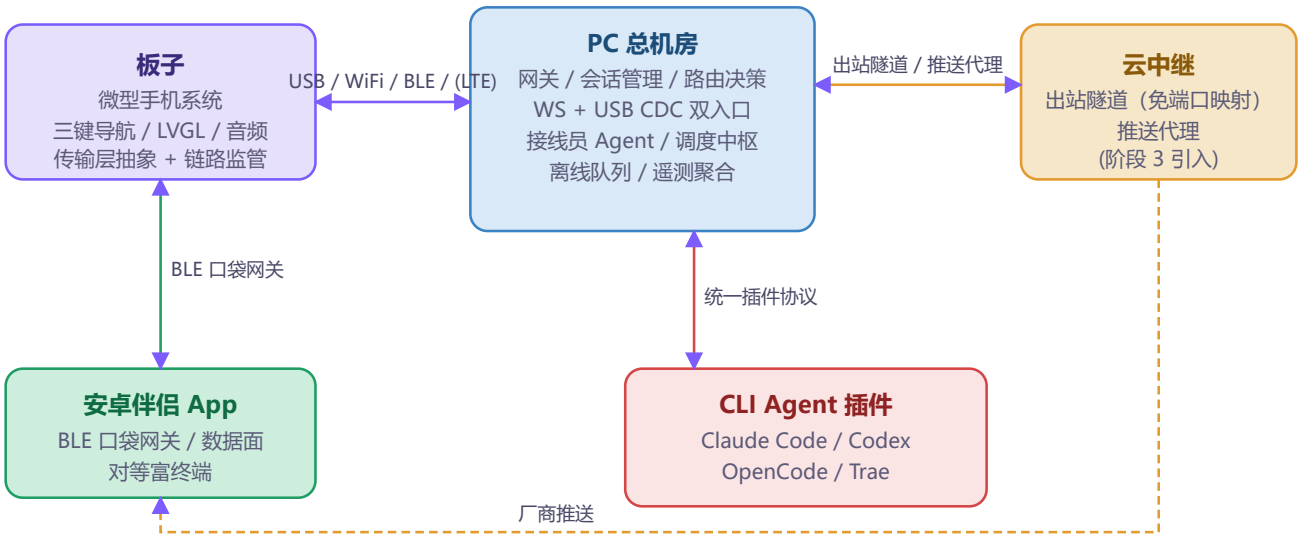
| 场景 | 链路形态 | 说明 |
|---|---|---|
| S0 工位/家里（插线） | 板子 —USB→ PC 总机房 | 供电 + 传输二合一，零电池消耗，屏幕常亮，最稳 |
| S1 室内自由活动 | 板子 —WiFi→ PC 总机房 | 直连，零公网暴露，延迟最低 |
| S2 户外（手机在身边） | 板子 —BLE→ 手机 App —4G/5G→ 云中继 ⇄ PC 隧道 | 手表同款架构，手机与板子均低功耗 |
| S3 户外（手机不在身边） | 板子深睡；消息积压云端队列 | 回连后 sync 补收，来电不丢 |
| S4 大流量任务 | 板子 —手机热点→ 云中继 | 仅 OTA/日志导出时 App 临时开热点，用完即关 |
| S5 未来 LTE 板 | 板子 —4G→ 云中继 | 可脱离开手机独立成机，支持真人语音通话 |

**链路优先级（板端链路监管器自动决策）**：

```
USB 已插入且 PC 端网关在位        → USB 有线模式（供电+传输，屏幕常亮）
检测到已知 SSID（到家/办公室）    → WiFi 直连（同时断 BLE 省电）
已配对手机在 BLE 范围内           → BLE 伴侣模式
BLE 断开且无 WiFi                 → 深睡，等推送唤醒或定时回连
大流量任务（OTA 等）              → USB 可用走 USB；否则请求 App 临时开热点
```

## 3. 系统架构

### 3.1 组件总览



### 3.2 各组件职责

| 组件 | 职责 | 关键约束 |
| --- | --- | --- |
| 板子固件 | 微型手机系统（状态栏/桌面/应用框架/事件弹窗层）、三键统一导航、音频采集/播放、传输层抽象（usb/ws/ble）、链路监管、消息去重、备忘录本地存储、亮度与息屏管理 | 400KB SRAM；TinyUSB + LVGL + 音频共存需严控内存 |
| PC 总机房 | WebSocket + USB CDC 双入口网关、设备会话管理、链路路由决策（直连/中继）、事件总线、离线队列、来电调度、备忘录转写（STT）与回填、遥测聚合（PC 资源采集、token 用量统计、commit/CI 状态查询） | 单机运行，无需公网 IP（出站隧道） |
| 接线员 Agent | 多模态大模型（音频直听或 STT），理解语音指令 → 路由到具体 Agent / 转达任务；接收备忘录派发 | PC 端调用云端 API，模型可替换 |
| 调度中枢 | Agent 注册表、状态聚合、权限请求队列、多设备消息扇出 | — |
| 插件桥 | 各 CLI 装统一插件：上报状态、转发权限请求、收回回答 | 每个 CLI 一份适配层，协议统一 |
| 云中继 | 公网撮合点：PC 与手机/板子均主动外连（出站隧道，免端口映射，可穿 CGNAT）；推送代理 | 阶段 3 引入；UPnP 端口映射仅作直连加速 |
| 安卓伴侣 App | 厂商推送保活、BLE 网关（数据面）、**对等富终端**（与板子同协议同功能 + 文本对话 / 复杂表单打字 / 完整上下文查看，即 web coding 客户端）、临时热点控制 | 各厂商后台限制需引导白名单 |

### 3.3 协议兼容三原则（不可妥协）

1. **只增不改**：消息类型与字段只增不删不改语义；未知字段一律忽略。
2. **hello 版本协商**：每次连接握手交换 `proto` 版本，取双方共同版本。
3. **caps 能力降级**：设备/总机各自声明能力集，网关按对方能力裁剪功能。

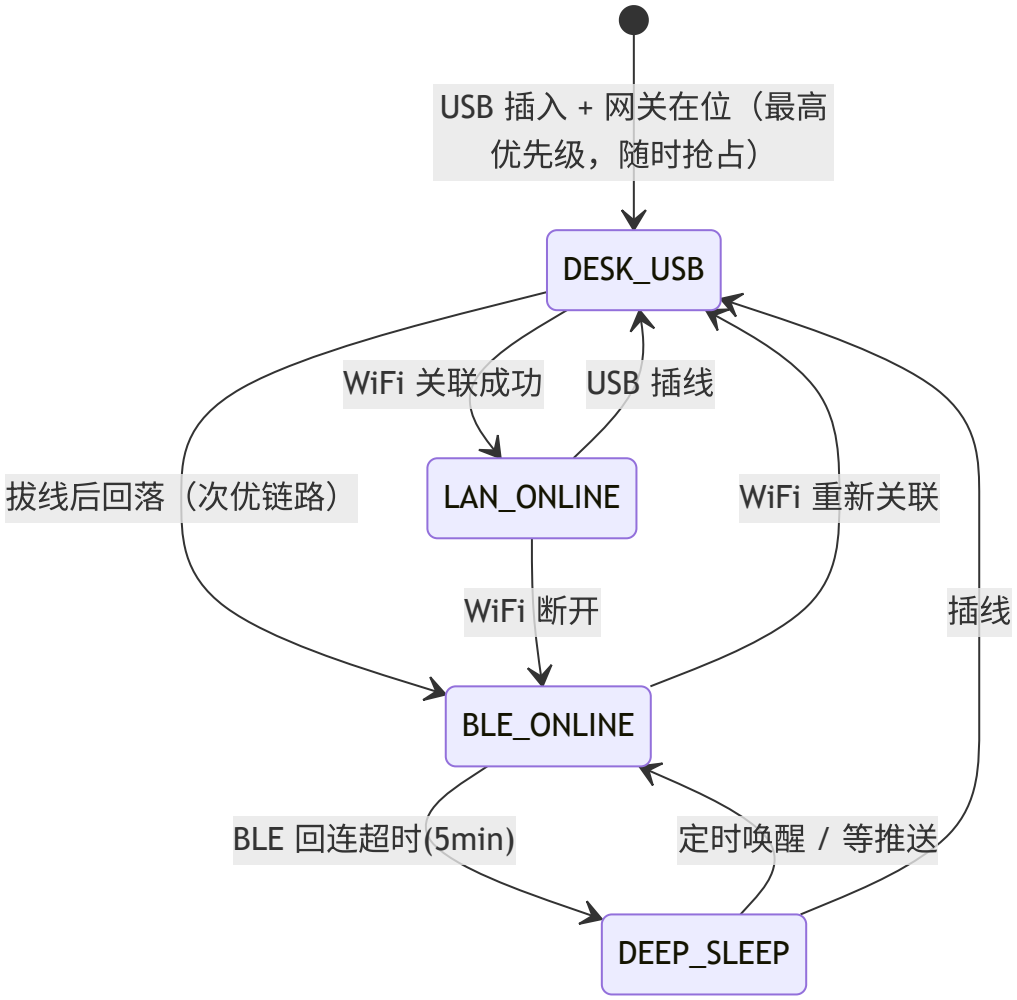
老固件永远可用；未来 LTE 板在 hello 里多声明 `"lte"` 即插即用。

---

## 4. 多端联动链路系统（核心需求）

这是本产品的差异化设计：**链路路由不是板子单方面决定，而是多端协同感知、各司其职。**

### 4.1 板端状态机




| 状态 | USB | WiFi | BLE | 屏幕 | 说明 |
|---|---|---|---|---|---|
| DESK_USB | 已连接，CDC 会话活跃 | 可选关闭 | **关闭** | 常亮（充电中） | 工位稳态：最稳、零消耗 |
| LAN_ONLINE | 断开 | 已连接，WS 会话活跃 | **关闭**（省电，手机零负担的前提） | 常规策略 | 室内活动稳态 |
| BLE_ONLINE | 断开 | 断开/扫描中 | 开启，回连已配对手机 | 常规策略 | 户外稳态 |
| DEEP_SLEEP | 断开 | 断开 | 低频广播或关闭 | 熄灭 | 手机不在身边；定时唤醒试回连 |

### 4.2 PC 端路由决策（总机房）

PC 通过各入口（USB CDC / WebSocket）心跳判断设备可达性：

```
USB CDC 会话活跃 → USB 路由（最高优先级，一切流量走有线）
心跳正常（WS 会话活跃）→ DIRECT 路由：消息直达板子，不向云中继/推送通道发任何东西（手机零负担）
心跳超时（设备离开局域网）→ RELAY 路由：消息上云中继 → 厂商推送 → 唤醒手机 App → BLE 送达板子
                          → 事件写入离线队列（seq 递增），防丢
设备重新可达（任一入口）  → 切回直连；设备端按 seq 去重（双链路瞬间并存时消息可能重复到达）
```

### 4.3 手机端行为

| 情形 | App 行为 | 手机负担 |
|---|---|---|
| 设备在 USB/WiFi | 收不到任何推送，BLE 不活跃 | **零** |
| 收到推送（设备离网） | 激活 → 连 BLE → 中转消息 → 显示通知 | 低（一次唤醒 + BLE 常连 mA 级） |
| 用户没带板子 | App 自己就是软终端，直接批权限/看状态 | 无 BLE，纯 App |

### 4.4 边界情况

| 情形 | 处理 |
|---|---|
| 插线瞬间 WiFi 会话还在 | 双链路并存窗口期，板端按 seq 去重；USB 稳定后关 WiFi 射频省电 |
| 刚到家：WiFi 已连但 BLE 未断 | 双链路并存窗口期，板端按 seq 去重；WiFi 稳定后主动关 BLE |
| 插着线但 PC 网关没开 | USB 仍供电充电；传输回落到 WiFi（链路监管器按"会话在位"而非"仅插线"决策） |
| 手机杀后台且厂商推送未达 | 云端队列兜底；板子下次任何链路上线即 sync 补收 |
| 局域网内 PC 重启 | 设备自动重连（指数退避）；重连期间事件积压队列，不丢 |
| WiFi 有信号但断网（外网故障） | 局域网内直连不受影响（总机房在 PC 本机）——直连路由的天然优势 |

---

## 5. 功能需求

优先级定义：P0 = MVP 必须；P1 = 完整体；P2 = 后续阶段。

### 5.1 板端手机系统（UI 框架）

| ID | 功能 | 优先级 | 描述 |
|---|---|---|---|
| FR-26 | 桌面与应用系统 | P0 | 顶部状态栏（时钟、电池/充电态、链路图标+信号）；桌面 = 红黄绿总览卡片 + 应用宫格（▲▼ 选应用，OK 进入）；应用框架可扩展新应用 |
| FR-27 | 内置应用集 | P0 | 电话簿、备忘录、设置、总览（驾驶舱：面板化设计，详见 5.9） |
| FR-28 | 事件覆盖层 | P0 | 来电 / 红灯报警无视当前所在应用，直接全屏弹窗置顶 + 响铃（见 FR-06/07） |
| FR-29 | 统一导航语义 | P0 | 三键撑起整个系统：短按 OK=进入/确认/接听；**长按 ▲=全局后退**（上一级/上一题/挂断）；长按 ▼=下一题；长按 OK=语音输入/场景化功能。详见 6.1 |
| FR-30 | 桌面快捷拨号 | P1 | 桌面长按 OK 直接拨号接线员（领导最常用动作，一步到位） |
| FR-08 | 链路状态指示 | P0 | 状态栏常驻：USB / WiFi / BLE / 云中继 / 离线 + 信号强度；长按进入链路详情页 |
| FR-31 | 亮度与息屏管理 | P0 | `bsp_display_backlight(0..100)` 已就绪。策略：自动降亮 → 超时息屏（来电/报警/任何按键即亮）→ USB 工位模式常亮。息屏超时与亮度档位可在设置中调 |

## 5.2 电话簿与通话（核心体验）

| ID | 功能 | 优先级 | 描述 |
|---|---|---|---|
| FR-01 | 电话簿 | P0 | 联系人列表：左 = Agent 头像（默认首字母色块，用户可自定义 logo），中 = 联系人名（仓库名/项目名），右 = 状态灯。首项固定为"总机·接线员"。上下键切换，OK 进入 |
| FR-02 | 状态灯 | P0 | 绿 = 运行中；黄 = 等待权限/提问；红 = 意外停止/阻塞。状态由插件推送变更，不轮询 |
| FR-03 | 联系人详情 | P0 | 简要工作状态：当前任务摘要、最近动作、等待事项 |
| FR-04 | 权限请求问答 | P0 | 单选 OK 后自动跳下一题；多选不自动跳，长按 ▲▼ 切题 |
| FR-05 | 语音输入 | P1 | 长按 OK 说话，松开发送。给接线员下指令、补全表单字段 |
| FR-06 | 来电界面 | P0 | Agent 主动联系：响铃 + 全屏显示来电方与阻塞原因；OK 接听，长按 ▲ 拒接/挂断 |
| FR-07 | 红灯报警 | P0 | 状态变红（意外停止/阻塞/**CI 失败**）：扬声器警示音（或 TTS 播报"repo-C 阻塞"），事件层弹窗显示阻塞摘要 |
| FR-09 | 表单补全 | P1 | 权限请求含"需用户输入"字段时，长按 OK 语音输入回填 |
| FR-10 | 对等软终端（App 侧） | P1 | 安卓 App 复刻电话簿/权限/来电/总览 UI，与板子跑同一套协议、功能对等（详见 5.10 富终端能力） |

## 5.3 备忘录（录音笔记 + AI 联动）

| ID | 功能 | 优先级 | 描述 |
|---|---|---|---|
| FR-32 | 录音备忘 | P0 | 备忘录应用内 OK 开始录音、再按 OK 停止保存；录音存本地存储分区（LittleFS），列表显示日期/时长/状态 |
| FR-33 | 本地回放 | P0 | 列表选中 → OK 播放/停止；长按 ▲ 返回 |
| FR-34 | 转写回填 | P1 | USB/WiFi 链路下自动同步录音到 PC → 总机房 STT 转写 → 文字回填为备忘标题与正文；列表区分"未转写/已转写" |
| FR-35 | 备忘派发 | P1 | 已转写备忘可"派发"：发送给接线员解析 → 转成具体 Agent 的任务（如"让小张看内存泄漏"）；派发状态回写备忘 |
| FR-36 | 备忘删除 | P0 | 删除本地录音与转写文本 |

## 5.4 语音与接线员

| ID | 功能 | 优先级 | 描述 |
|---|---|---|---|
| FR-11 | 对讲式通话 | P1 | 半双工：长按 OK 说话，松开放音回复。不做全双工 |
| FR-12 | 接线员转接 | P1 | "喂，接小张"/"让 repo-B 跑测试" → 解析意图并转接/转达 |
| FR-13 | 多模态/STT | P1 | 接线员优先音频直听的多模态模型；不支持时 PC 端 STT。模型可配置替换 |

| ID | 功能 | 优先级 | 描述 |
| --- | --- | --- | --- |
| FR-14 | 语音回执 | P2 | 关键操作后 TTS 复述确认 |

## 5.5 Agent 接入

| ID | 功能 | 优先级 | 描述 |
| --- | --- | --- | --- |
| FR-15 | 插件桥协议 | P0 | 各 CLI（Claude Code hooks / Codex / OpenCode plugin / Trae）装统一插件，上报 presence、state、ask，接收 answer |
| FR-16 | Agent 注册 | P0 | 插件启动即注册：agent_id（仓库名）、CLI 类型、工作目录；下线即电话簿移除/置灰 |
| FR-17 | 状态摘要生成 | P1 | 插件在 PC 端做上下文摘要（可调 LLM），只传摘要不传全文 |
| FR-18 | 主动来电 | P1 | Agent 阻塞/需确认时，插件 → 总机房 → ring → 设备响铃 |

## 5.6 链路与可靠性

| ID | 功能 | 优先级 | 描述 |
| --- | --- | --- | --- |
| FR-37 | USB 有线传输 | P0 | TinyUSB 双 CDC：CDC0 = 调试控制台（替代 USB-Serial/JTAG console），CDC1 = 协议传输。插线且网关在位时为最高优先级链路；烧录走 ROM 下载模式不受影响 |
| FR-38 | 传输抽象层 | P0 | `transport_ops_t` 统一接口，usb / ws / ble / (未来 lte) 各一份实现，链路监管器统一调度与切换 |
| FR-20 | 三端联动 | P1 | 见第 4 节：板端状态机 + PC 路由决策 + 手机零负担原则 |
| FR-21 | 离线队列与 sync | P1 | 每设备事件 seq 递增；重连后 `sync(since_seq)` 补收；板端按 seq 去重 |
| FR-22 | 云中继 | P2 | 出站隧道撮合（免端口映射、穿 CGNAT）；UPnP 仅作直连加速 |
| FR-23 | 厂商推送 | P2 | FCM/小米/华为/OPPO/vivo（可极光/个推聚合）。推送只唤醒 App，不承载业务数据 |
| FR-24 | OTA | P2 | 优先 USB/WiFi；BLE 链路禁止 OTA |
| FR-39 | 文件传输 | P1 | 统一 `file` 消息（分块、断点续传语义）：备忘录上传、头像下发、OTA 镜像共用 |

## 5.7 联系人模型（为 LTE 预留）

| ID | 功能 | 优先级 | 描述 |
| --- | --- | --- | --- |
| FR-25 | 联系人类型 | P2 | `type: agent`（虚拟通话）与 `type: human`（带电话号）。未来 LTE 板遇 human 直走模块语音通道，UI 层与 Agent 通话完全统一 |

### 5.8 设置应用

| ID | 功能 | 优先级 | 描述 |
|---|---|---|---|
| FR-40 | 显示设置 | P0 | 亮度档位（低/中/高/自动）、息屏超时（15s/30s/1min/常亮） |
| FR-41 | 链路设置 | P1 | WiFi 配网（USB/BLE 辅助配网）、配对手机管理、云中继开关 |
| FR-42 | 声音设置 | P0 | 扬声器音量、铃声选择、静音模式 |
| FR-43 | 联系人管理 | P2 | 自定义头像导入（经 `file` 传输）、联系人排序/置顶 |
| FR-44 | 关于 | P0 | 固件版本、proto 版本、电池健康、存储占用、重启/恢复出厂 |

### 5.9 总览驾驶舱（遥测面板）

总览应用为**面板化驾驶舱**：▲▼ 在面板间循环切换（Agent 状态 / Token 消耗 / PC 资源 / 提交与 CI），所有数据由 PC 端推送，板子只显示不计算。遥测消息走既有信封与 seq 机制，断线时计入离线队列，BLE 链路下自动降频。

| ID | 功能 | 优先级 | 描述 |
|---|---|---|---|
| FR-45 | 面板 1 · 状态与系统 | P0 | 各 Agent 红黄绿灯一行一条（名称 + 简要状态），底部：电量、当前链路、存储占用 |
| FR-46 | 面板 2 · Token 消耗 | P1 | 今日总量（tokens + 估算费用）、各 Agent 分项、本月累计与预算进度条。数据源：插件上报会话 usage（Claude Code hooks usage / 各 CLI JSON 输出），总机房按日/月聚合，变更时增量推送 |
| FR-47 | 面板 3 · PC 资源 | P1 | PC 端 CPU %、内存占用、运行中 Agent 会话数、总机房 uptime。`telemetry` 消息周期推送（USB/WiFi 30s，BLE 2min）；PC 过载（CPU 持续 >90% 或内存告急）时主动告警一次 |
| FR-48 | 面板 4 · 提交与 CI | P1 | 各仓库最近一次提交（短 sha + 消息摘要 + 是否已推送）与 CI 状态（通过/失败/运行中）。commit 由插件上报；CI 状态由 PC 端查询（GitHub API / gh）。**CI 失败 = 仓库红灯**，走既有报警链路（响铃 + 事件层弹窗） |
| FR-49 | 卡死检测 | P2 | 总机房综合遥测判断 Agent 卡死（CPU 无变化 + 长时间无输出心跳）→ 标红并来电提醒"疑似卡死，是否重启会话" |

### 5.10 安卓富终端（对等 + 增强）

App 与板子是**协议层面的对等终端**（同一个 Hub 会话的两个客户端，消息扇出、状态一致），在此之上 App 利用手机能力提供板子做不到的增强功能——手机端不被浪费，复杂内容交由手机处理：

| ID | 功能 | 优先级 | 描述 |
|---|---|---|---|
| FR-50 | 对等终端 UI | P1 | 电话簿 / 权限问答 / 来电 / 总览驾驶舱 / 备忘录，与板子同构（触摸交互替代三键）；多终端状态同步（板子上批过的，App 里自动消失） |
| FR-51 | 文本对话（web coding） | P1 | 聊天式界面直接对任意 Agent / 接线员发消息，即一个移动端 coding 客户端：查看回复流、追加指令、粘贴代码/错误信息 |
| FR-52 | 复杂表单文本输入 | P1 | 板子上需长按 OK 语音补全的输入字段，App 内直接键盘输入（长文本、命令、变量名——语音不适合的场景）；两端同一 ask 双端均可答 |
| FR-53 | 完整上下文查看 | P1 | 联系人详情在 App 端可展开完整上下文（非摘要）、历史会话记录、diff/文件树浏览（由 Hub 下发，App 渲染）——板子 240×320 只显示摘要的原因在手机上不复存在 |
| FR-54 | 通知中心 | P2 | App 内聚合：来电记录、报警历史、权限审批历史，支持回查 |

# 6. 交互规范

## 6.1 统一导航语义（全局唯一事实表）

三键撑起整个手机系统，核心是"**长按 ▲ = 全局后退**"的后退语义：

| 按键 | 短按 | 长按 |
|---|---|---|
| ▲ 上键 | 上移一项 | **后退**：上一级页面；问答=上一题（在 Q1 再长按=退出问答）；通话中=挂断 |
| ▼ 下键 | 下移一项 | 问答=下一题（多选切题）；列表=快速到底 |
| OK 键 | 进入/确认/接听/播放暂停 | 语音输入（可输入场景）；桌面=快捷拨号接线员；来电=拒接；备忘录=开始/停止录音 |

## 6.2 屏幕清单

| 屏幕 | 内容 |
|---|---|
| 桌面 | 状态栏（时钟·电池·链路）+ 红黄绿总览卡片 + 应用宫格（电话簿/备忘录/设置/总览） |
| 电话簿 | 联系人列表（头像 \| 名称 \| 状态灯），见 FR-01 |
| 联系人详情 | Agent 名、仓库、CLI 类型、状态摘要、当前任务、最近提交（sha+消息）、CI 状态、今日 token 用量 |
| 通话中 | 对方头像 + "通话中"，录音时显示波形/计时 |
| 权限请求 | 题号（Q n/m）、题目、选项列表（可勾选态）、单选/多选标识 |
| 来电（事件层） | 全屏：来电方头像+名称、阻塞原因摘要、接听/拒接提示；压在任意应用之上 |
| 备忘录列表 | 各条备忘：日期、时长、已转写标题（或"未转写"）、派发状态 |
| 备忘录音中 | 波形 + 计时 + 停止提示 |
| 备忘详情 | 转写全文（可滚动）、播放、派发、删除 |
| 总览驾驶舱 | 面板化（▲▼ 切换）：① Agent 红黄绿 + 电量/链路/存储 ② Token 消耗（今日/分项/月度预算）③ PC 资源（CPU/内存/会话数）④ 提交与 CI（sha + 消息 + CI 状态） |
| 设置各页 | 显示/链路/声音/联系人/关于，见 5.8 |
| 链路详情 | 当前链路、信号、上次同步 seq |

## 6.3 多选题的切题规则（用户明确要求）

- 单选：OK 选中某项 → **自动跳下一题。**
- 多选：OK 勾选/取消勾选，**不自动跳题**；长按 ▲▼ 显式切上一题/下一题。
- 存在"需用户输入"字段：该题显示"长按 OK 语音输入"提示。

---

# 7. 通讯协议

## 7.1 消息信封（所有链路一致）

```
{
  "type": "state | ask | answer | voice | ring | sync | file | ...",
  "from": "agent:repo-b | hub | operator | device",
  "to":   "device | operator | agent:repo-c",
```

```
  "seq":  12345,
  "payload": {}
}
```

## 7.2 消息字典

| type | 方向 | payload 关键字段 | 说明 |
|---|---|---|---|
| hello | 设备→总机 | proto, device, fw, caps[] | 握手 + 版本/能力协商（USB/WS/BLE 各链路同构） |
| hello_ack | 总机→设备 | proto, caps[] | 确认协商结果 |
| presence | 插件→总机 | agent_id, online | Agent 上下线 → 电话簿增删 |
| state | 插件→总机→设备 | agent_id, state, summary | state ∈ running / waiting_input / blocked |
| ask | 插件→总机→设备 | ask_id, agent_id, questions[{kind, options[], multi, needs_input}] | 权限请求/提问，支持多题 |
| answer | 设备→总机→插件 | ask_id, answers[{selections[] \| text}] | 回答回传 |
| voice | 设备→接线员 | opus 帧 | 语音片段（压缩） |
| voice_ack | 接线员→设备 | transcript, intent, route | 转写与解析结果 |
| ring | 总机→设备 | agent_id, reason, summary | 来电（含红灯报警触发） |
| ring_ack | 设备→总机 | accept / reject | 接听/拒接 |
| audio | 双向 | opus 帧 | 通话中音频 |
| hangup | 双向 | — | 挂断 |
| file | 双向 | file_id, kind(memo/avatar/ota), chunk, offset, total | 分块文件传输：备忘上传、头像下发、OTA 共用 |
| file_ack | 双向 | file_id, offset | 分块确认/续传 |
| telemetry | 总机→设备 | cpu, mem, agents_active, uptime | PC 资源周期遥测（USB/WiFi 30s，BLE 降频 2min；离线时段不补发，只发最新值） |
| usage | 总机→设备 | today{tokens, cost}, per_agent[], month{tokens, cost, budget} | Token 消耗聚合，变更时增量推送 |
| commit | 插件→总机→设备 | agent_id, sha, branch, message, pushed | 最近提交（联系人详情与总览面板 4 显示） |
| ci | 总机→设备 | agent_id, status(pass/fail/running), summary | CI 状态查询结果；fail 转入红灯报警链路 |
| memo_result | 总机→设备 | memo_id, title, text | 备忘录转写回填 |

| type | 方向 | payload 关键字段 | 说明 |
|---|---|---|---|
| memo_dispatch | 设备→总机 | memo_id, target | 备忘派发给接线员/某 Agent |
| sync | 设备→总机 | since_seq | 断线补收请求 |
| sync_data | 总机→设备 | events[] | 补收的事件批次 |

### 7.3 各链路传输实现

| 链路 | 载体 | 分帧 | 说明 |
|---|---|---|---|
| USB | TinyUSB CDC1（双 CDC：CDC0 调试台） | 直接 TLV/JSON 行帧 | 无 MTU 压力，全速 12Mbps；插线即最高优先级 |
| WiFi | WebSocket（局域网直连 PC） | WS 帧 | 带宽充裕 |
| BLE | 自定义 GATT（NUS 风格 RX/TX 特征值） | MTU 协商 247B + 分帧重组 + 轻量重传 | 连接参数：交互态 15-30ms，空闲态 100-500ms |
| LTE（未来） | 模块 TCP/UDP | 同 USB 分帧 | 新板 transport_lte 实现 |

### 7.4 音频规格

| 链路 | 编码 | 码率 | 说明 |
|---|---|---|---|
| USB / WiFi | Opus 16kbps（或 PCM） | ≤ 256kbps | 带宽充裕 |
| BLE | Opus 16kbps（备选 codec2 2.4kbps） | ≤ 32kbps | 对讲模式，延迟 1-2s 可接受 |
| 采样 | 16kHz 单声道 | — | 板端 ES8311 全双工采集（备忘录同规格） |

---

## 8. 非功能需求

### 8.1 电源与显示（官方基准：全亮 3-4h，低亮/息屏 8h）

| 模式 | 屏幕策略 | 电池消耗 |
|---|---|---|
| DESK_USB | 常亮（充电中） | **零消耗**（边充边用） |
| LAN_ONLINE | 自动降亮 → 超时息屏 | 中等（目标支撑工作日内碎片使用） |
| BLE_ONLINE | 激进降亮 → 快速息屏（事件即亮） | 低（目标日均 < 10mA） |
| DEEP_SLEEP | 熄灭 | μA 级 |

设计目标：插线工位 + 户外 BLE + 偶发充电的组合，满足 8 小时工作制整天使用。

### 8.2 其他

| 维度 | 指标 |
|---|---|
| RAM 预算 | TinyUSB + WiFi + BLE + LVGL + 音频缓冲共存需实测基线；互斥状态机保证不同时全开；LVGL 缓冲维持现状不加大（无 PSRAM，总量 400KB） |

| 维度 | 指标 |
| --- | --- |
| 状态延迟 | 状态变更 → 屏幕更新：USB/局域网 < 1s；BLE 链路 < 3s |
| 语音链路延迟 | 长按 OK 说完 → 接线员确认收到：< 3s（有线/局域网）/ < 5s（BLE+蜂窝） |
| 可靠性 | 消息不丢（seq 队列 + sync 补收）；不重（板端 seq 去重） |
| 兼容性 | 只增不改 + hello 协商 + caps 降级；老固件永不淘汰 |
| 隐私 | 局域网/USB 模式零公网暴露；云中继仅阶段 3+，且只转发信封（摘要 PC 端生成，可配置脱敏） |
| 安全 | BLE 配对绑定；WS/TLS；USB CDC 需简单配对码（防工位邻居插线冒充）；插件桥本机回环或局域网鉴权 |

## 9. 里程碑

按"每阶段加一种传输 + 一组功能"推进，USB 先行让全链路最早打通（无需无线协议栈，风险最小）：

| 阶段 | 范围 | 出口标准 |
| --- | --- | --- |
| M1 骨架版·USB | 板端：手机系统框架（桌面/状态栏/事件层/统一导航）+ 电话簿/问答/来电/报警 UI + USB 传输（TinyUSB 双 CDC）+ 备忘录录音/回放 + 亮度管理 + 总览面板 1（状态/电量/链路）；PC 端：总机房（CDC 入口）+ 插件桥（先接 Claude Code） | 真机演示：插线后电话簿显示真实 Agent 状态，远程批准一次权限，红灯触发响铃，录一条备忘录 |
| M2 无线+语音 | WiFi 传输、接线员 Agent、对讲通话、STT/多模态、备忘录转写回填与派发、遥测面板（Token/PC 资源/提交与 CI，含 CI 失败报警） | 真机演示："喂，让小张跑测试"全链路闭环；备忘录转写并派发成功；总览四面板数据实时刷新 |
| M3 户外+富终端 | 云中继、安卓伴侣 App（推送 + BLE 网关）、对等富终端（FR-50~54，局域网直连形态先行，随后 BLE/中继形态）、三端联动、离线队列、file 传输（头像/OTA） | 真机演示：带板出门 BLE 收权限并批准；回家自动切直连、手机零推送；App 直接文本对话指挥 Agent |
| M4 LTE 板 | transport_lte、真人联系人（type:human）、语音直拨 | 新板插 SIM 即用，老板子不受影响 |

## 10. 风险与开放问题

| # | 风险/问题 | 等级 | 应对 |
| --- | --- | --- | --- |
| R1 | ESP32-C3 RAM：TinyUSB + WiFi + BLE + LVGL + 音频共存可能超预算 | 高 | M1 即建内存实测基线；状态机保证互斥（USB 时关 WiFi/BLE 射频）；必要时缩 LVGL 对象池 |
| R2 | C3 单射频 WiFi/BLE 共存吞吐打折 | 中 | 两链路设计上不并发；并存窗口期仅控制消息 |
| R3 | 各 CLI 插件 API 差异大、部分无稳定 hook | 高 | 按 CLI 分适配层；Claude Code hooks 最成熟先打样，协议定型后再铺开 |
| R4 | 安卓厂商杀后台 + 推送自启动限制 | 中 | 引导白名单/自启动；云端队列兜底 |
| R5 | CGNAT 下端口映射不可用 | 低 | 出站隧道为主，端口映射仅加速 |
| R6 | BLE 吞吐对语音的约束 | 中 | Opus 16k 低码率；对讲半双工；codec2 兜底 |
| R7 | TinyUSB 与 USB-Serial/JTAG 模式互斥（同一 USB 控制器） | 中 | 双 CDC 方案（CDC0 调试台 + CDC1 协议）保住开发体验；烧录走 ROM 下载模式验证；UART 日志作为后备 |

| # | 风险/问题 | 等级 | 应对 |
|---|---|---|---|
| R8 | 8MB Flash 分区规划：OTA 双分区 + 备忘录 LittleFS 分区 + 字体/头像资源 | 中 | M1 即定分区表（预留 OTA A/B 与 ≥1.5MB 存储），避免后期重分区丢数据 |
| R9 | Token 用量数据可得性：各 CLI 暴露 usage 的方式不一（Claude Code hooks 有 usage；部分 CLI 需解析 JSON 输出甚至不可得） | 中 | 插件桥按 CLI 适配，取不到精确值时降级为"会话数/消息数"统计；面板标注数据来源可信度 |
| Q1 | 接线员模型选型（多模态音频直听 vs STT 两段式）？ | — | M2 前定，接口预留两种 |
| Q2 | 头像资产：各公司 logo 版权与自定义头像上传方式? | — | 默认首字母色块头像规避版权，logo 用户自行放入 |
| Q3 | 多终端同时在线（板 + App）消息扇出与已读策略? | — | 方向已定：双端对等扇出、同一 ask 先答先得（后答收"已被处理"回执）；细节 M3 设计时定 |
| Q4 | 云中继自建（frp/WS 中转）还是 Tailscale 组网? | — | M3 技术选型专题 |
| Q5 | 备忘录转写的 STT 与接线员是否共用同一模型/供应商? | — | M2 前定，倾向共用降成本 |

## 11. 附录：硬件能力对照（BSP 现状）

| 能力 | 现状 | 本项目使用 |
|---|---|---|
| 显示 | ST7789P3 240×320，SPI 40MHz，无触控，LEDC 背光 | 手机系统 UI（状态栏/桌面/应用/事件层），纯三键交互 |
| 背光亮度 | `bsp_display_backlight(0..100)` **已就绪** | 亮度档位 + 自动降亮 + 息屏策略，零 BSP 开发量 |
| 按键 | 3 键（up/down/ok，ADC 分压） | 统一导航语义（含长按与全局后退） |
| 音频 | ES8311 全双工 I2S，播放+麦克风 | 对讲语音、响铃、TTS、备忘录录音 |
| USB | Type-C 接原生 USB（GPIO18/19），现用于烧录/控制台 | TinyUSB 双 CDC：CDC0 调试台 + CDC1 协议传输（最高优先级链路） |
| 无线 | ESP32-C3 WiFi 2.4G + BLE 5.0（BSP 未初始化，需新增组件） | 传输层 ws/ble 两实现 |
| 电源 | 电池 + bsp_battery；官方续航全亮 3-4h / 低亮息屏 8h | 四档电源策略（8.1） |
| 存储 | 8MB Flash，无 PSRAM | 需规划分区表：OTA A/B + LittleFS 备忘录分区（R8） |

# BOSS PHONE 总体技术设计

| 项目 | 内容 |
| --- | --- |
| 状态 | Draft v0.1（对应 PRD v0.4） |
| 上游文档 | BOSS_PHONE_PRD.md |
| 日期 | 2026-08-24 |

本文档定义整个系统的工程结构：仓库布局、技术栈、模块边界、各 CLI 插件的兼容策略与关键代码骨架。细节设计下沉到各专项文档（见第 1 节）。**当前无实机，所有代码均为设计骨架，进入 M1 后再落地。**

> **硬件时间线（2026-08-29 更新）**：预售板子预计 **2026-09-13 发货**，9 月中旬到手。到货前为"无实机窗口期"——先完成协议 SSOT、Hub/STT 全部 PC 侧开发与 WAV 仿真评测（**M0**，见第 8 节），板子到货后把"WAV 文件输入"换成"串口 PCM 输入"即进入真机联调。

> **固件语言定稿（2026-08-29）**：混合策略——C 保留 BSP 与 LVGL UI 薄层，**业务层（协议/传输/状态机/音频管道）用 Rust（esp-idf-hal，std 环境）**，详见 4.3 节。

## 1. 设计文档体系

| 文档 | 范围 | 状态 |
| --- | --- | --- |
| BOSS_PHONE_PRD.md | 产品需求（唯一需求事实来源） | v0.4 |
| 本文 DESIGN_OVERVIEW | 工程结构 / 技术栈 / 模块边界 / 代码骨架 | v0.1 |
| DESIGN_PROTOCOL | 协议规范：消息字典全量字段、帧格式（JSON 控制帧 + 二进制媒体帧）、seq/sync、错误码、时序图 | v0.1 |
| DESIGN_FIRMWARE | 板端固件：Flash 分区表、任务/内存预算、boss_transport / boss_protocol / boss_app（LVGL 应用框架）设计 | v0.1 |
| DESIGN_HUB | PC 总机房：CDC+WS 双入口网关、会话/路由、事件总线、离线队列、遥测聚合 | v0.1 |
| DESIGN_PLUGIN | 插件桥：统一 AgentAdapter 协议 + 各 CLI（Claude Code / Codex / OpenCode / Trae）适配层实现 | v0.1 |
| DESIGN_ANDROID | 安卓 App：BLE 网关、对等富终端 UI、推送接入、web coding 对话界面 | v0.1 |
| DESIGN_DEPLOYMENT | 部署与运维：Hub/插件/配网/配对/升级/故障排查 | v0.1 |
| DESIGN_TEST | 测试与验收：M1-M4 验收清单、真机测试矩阵、CI 双轨、回归 | v0.1 |
| DESIGN_SECURITY | 安全设计：威胁模型、鉴权、链路安全、隐私、最小权限 | v0.1 |

写作顺序建议：PROTOCOL → HUB → PLUGIN → FIRMWARE → ANDROID（协议是全部下游的契约，先冻结），随后 DEPLOYMENT → TEST → SECURITY 收口。文档集可一键导出为汇总 PDF（`tools/pdf-export`）。

## 2. 设计原则（工程视角）

1. **协议单一事实来源（SSOT）**：`protocol/` 目录下定义一次，生成 C / Rust / Kotlin / Python 四端代码。禁止任何端手写消息结构。
2. **模块只依赖协议，不依赖彼此**：板子固件、Hub、插件、App 四个模块间**只通过协议交互**，仓库内可独立构建、独立演进。
3. **能力协商代替版本分支**：hello + caps（PRD 3.3），任何端的新能力都要以"老对端可忽略"的方式加入。
4. **哑终端纪律**：固件不做任何语义理解（不含 STT/意图/摘要），显示的一切文字都由 Hub 下发。违反这条 = 固件开始变笨重。
5. **现有 BSP 零破坏**：`components/bsp` 与 `main/` 的 demo 体系保持不动（AGENTS.md 契约），Boss Phone 作为新增组件与新增入口并存。

---

## 3. Monorepo 仓库结构

当前仓库改造为 monorepo（固件保持 ESP-IDF 根项目，其余子项目为独立构建单元）：



**与 ESP-IDF 构建的兼容**：`protocol/`、`hub/`、`plugins/`、`android/`、`tools/` 不是 ESP-IDF 目录，`idf.py` 会忽略它们；固件业务层（Rust 组件）依赖 `protocol/rust` crate，UI 薄层 include `protocol/c/boss_protocol.h`。ESP-IDF 根 `CMakeLists.txt` 只需照常列 components。

**Cargo workspace（仅固件侧 Rust）**：固件业务层组件（`components/boss_protocol` 等）共用 `protocol/rust` crate；`hub/` 与 `plugins/*` 是 Python 包（`hub/` 下 `pyproject.toml` + `python -m hub` 入口），不经 cargo 构建。`tools/gen-protocol` 是独立 Node 脚本（开发期工具），负责输出 C/Rust/Kotlin/Python 四端绑定。

---

## 4. 技术栈选型

| 端 | 选型 | 理由 |
|---|---|---|
| 板子固件 | C（BSP + LVGL UI 薄层）+ Rust（esp-idf-hal，std 环境）+ TinyUSB（双 CDC） | BSP/demo 零破坏（AGENTS.md 契约）；业务层（协议/传输/状态机/音频管道）Rust 编译期内存安全；WiFi/BLE 走 esp-idf-svc（IDF 成熟 C 栈封装） |
| Hub | Python（asyncio） | 中枢中转 + 接线员：STT/LLM/意图/备忘的 AI 逻辑 Python 生态最强（faster-whisper、openai/anthropic SDK、pydantic）；原定计划即 Python；`pyserial` + `websockets` 够用 |
| 插件桥 | Python（与 Hub 同语言） | CLI 适配器是纯逻辑（hook 事件 → 本地 WS 上报），无性能压力；与 Hub 同语言零成本共享协议绑定 |
| 安卓 App | Kotlin + Jetpack Compose + 原生 BLE + kotlinx-serialization | 现代安卓标准；BLE 网关用平台 API 避免第三方依赖黑盒 |
| 协议 SSOT | JSON Schema + 自研生成器（Node.js 脚本） | 生成器是开发期工具，Node 最顺；只负责输出 C/Rust/Kotlin/Python 四端结构声明，与运行时性能无关 |

> **固件语言不变**：板子固件仍是 C（BSP + LVGL UI 薄层）+ Rust（业务层），内存安全需求在板上，PC 侧无此痛点（Python 本身内存安全）。

### 4.1 运行时内存特性

| 端 | 语言 | 运行时内存模型 |
|---|---|---|
| Hub | Python | 常驻 asyncio 单进程；典型 RSS ~60-80MB；whisper 模型按需加载（base 147MB 进程外，faster-whisper 可量化） |
| 插件桥 | Python | 常驻轻量适配器进程，几 MB 起步 |
| 生成器（开发期） | Node.js | 一次性工具，短暂运行即退出，不计入常驻 |
| （替代对比）Hub 用 Rust | Rust | 更省内存（<30MB），但接线员 AI 逻辑开发成本高——取舍：PC 内存不紧，Python 开发效率优先 |

**原则（2026-08-30 定稿）**：板子固件业务层用 **Rust**（内存安全）；PC 侧 Hub/插件桥/接线员用 **Python**（AI 逻辑优先，原定计划）；协议 SSOT 语言无关，生成器用 Node，产物 Rust/Kotlin/C/Python 四端。

### 4.2 各 CLI 生态机制盘点

- **OpenAI Codex 本体就是 Rust**（`openai/codex`）——其 `exec --json` 事件流、notify 进程外协议，Python 适配器直接解析其 JSON 事件即可。
- Claude Code hooks 是"外部命令 + stdin/stdout JSON"，与语言无关，Python 子进程天然胜任。

- OpenCode（sst/opencode，TypeScript/Node 实现）的 plugin API 是 TS 事件总线；Python 侧用其 JSON 事件协议对接即可，能力挂起机制（permission 事件）由适配器实现。

### 4.3 固件语言策略：混合 C + Rust（2026-08-29 定稿）

| 层 | 语言 | 内容 | 理由 |
|---|---|---|---|
| 硬件驱动层 | C | `components/bsp`（显示/按键/音频/电池），已实机验证 | 零破坏零重写（AGENTS.md 契约） |
| UI 薄层 | C | `boss_app` 的 LVGL 声明式布局 | 声明式布局不是内存 bug 重灾区；lvgl-rs 绑定尚繁琐，不值得 |
| 业务层 | Rust | `boss_protocol` / `boss_transport` / 链路状态机 / 音频管道 / 队列 | 解析外部输入的重灾区：编译期消灭越界/UAF/数据竞争 |

- **接入方式**：`esp-idf-sys` 生成 `bsp_*` C API 的 Rust 绑定；Rust 组件编出 `.a` 由 ESP-IDF 链接进固件；WiFi/BLE/USB 用 `esp-idf-svc`（IDF C 栈的安全封装）。
- **为什么是 std 而不是 no_std esp-hal**：复用 IDF 成熟网络栈；BSP 直接链接；`serde_json/std` 生态可用。彻底 no_std（esp-hal + esp-wifi）留作远期可选。
- **诚实边界**：Rust 消灭的是内存**安全 bug**，不是内存**预算工作**——400KB SRAM 的静态分配与栈规划照做（`heapless` / 静态缓冲优先，谨慎 `alloc`）。

---

## 5. CLI 插件兼容矩阵（核心工程风险）

各 CLI 的扩展机制差异很大，统一策略是：**adapter-core 定义能力接口，各 CLI 适配器声明自己支持的能力子集，Hub 按能力降级。**

### 5.1 各 CLI 机制盘点

| 能力 | Claude Code | Codex CLI | OpenCode | Trae |
|---|---|---|---|---|
| 扩展机制 | hooks（`settings.json`，stdin 收 JSON 事件） | `config.toml` 的 `notify`（外部命令，JSON 事件）+ `exec --json` 事件流 | plugin API（`@opencode-ai/plugin`，TS 事件总线） | 待调研 |
| 状态上报 | Stop / PostToolUse / Notification 事件 | agent-turn-complete / token-count 事件 | chat.message / tool.execute.* 事件 | 待调研 |
| **权限拦截（挂起等待回答）** | **强**：PreToolUse hook 返回 `permissionDecision: allow/deny/ask`，进程挂起等待 | 弱：approval_policy 只能预设（untrusted/on-request/...），notify 只通知不挂起 | **强**：permission 事件可异步等待外部决定 | 待调研 |
| 主动提问 | Notification hook | notify 事件 | event 总线 | 待调研 |
| token usage | hook 事件内含 usage / transcript 可解析 | token-count 事件 | 消息级 usage | 待调研 |
| commit 事件 | PostToolUse(Bash/git) 解析或 Stop 后查 git | 同左 | 同左 | 同左 |

## 5.2 统一适配接口（adapter-core 设计核心）

```
// plugins/adapter-core/src/index.ts —— 设计骨架
export interface AgentAdapter {
  readonly cli: 'claude-code' | 'codex' | 'opencode' | 'trae';
  readonly agentId: string;            // 仓库名，即电话簿联系人名
  /** 能力声明：Hub 据此降级（如 codex 无挂起拦截 → 权限改为"预批+事后通知"） */
  readonly caps: AdapterCaps;
  start(hub: HubClient): Promise<void>;
  stop(): Promise<void>;
}

export interface AdapterCaps {
  statusReport: boolean;               // 状态上报 → 红黄绿灯
  permissionSuspend: boolean;          // 能否挂起等用户回答（黄灯核心）
  proactiveAsk: boolean;               // 主动提问
  usage: 'exact' | 'approx' | 'none';  // token 统计精度（R9 风险应对）
}
```

**降级规则（Hub 侧执行）**：

- `permissionSuspend = false`（如 Codex notify 模式）：权限请求降级为**事后通知**——Agent 按 CLI 自身策略先跑，Hub 推送"repo-B 已自动执行 X（策略：on-failure）"，用户可事后叫停。黄灯在该 Agent 上退化为"通知灯"。
- `usage = 'none'`：Token 面板该 Agent 显示"不支持统计"。
- 适配器对 Hub 只暴露统一事件流（presence/state/ask/usage/commit），Hub 不感知 CLI 差异。

## 5.3 Claude Code 打样要点（M1）

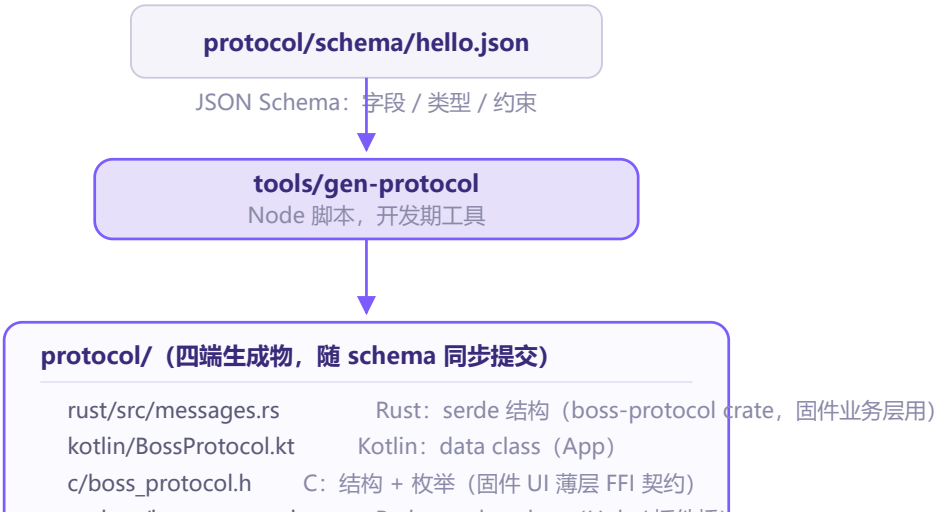
hooks 配置（`.claude/settings.json`，由插件安装器写入）：

```
{
  "hooks": {
    "PreToolUse":  [{ "hooks": [{ "type": "command", "command": "boss-plugin claude-code pre-tool" }] }],
    "PostToolUse": [{ "hooks": [{ "type": "command", "command": "boss-plugin claude-code post-tool" }] }],
    "Notification":[{ "hooks": [{ "type": "command", "command": "boss-plugin claude-code notify" }] }],
    "Stop":        [{ "hooks": [{ "type": "command", "command": "boss-plugin claude-code stop" }] }],
    "SessionStart":[{ "hooks": [{ "type": "command", "command": "boss-plugin claude-code session-start" }] }]
  }
}
```

`boss-plugin` 是插件 CLI：从 stdin 读 hook 事件 JSON → 转成统一协议 → 经本机 WS 发给 Hub；收到 `ask` 时以 hook 期望的 JSON 格式输出 permissionDecision（**这是"挂起等领导批准"的实现点**：hook 进程阻塞等待板子/手机回答）。

---

## 6. 协议 SSOT 与代码生成




- 控制消息 = **JSON 行帧**（\n 结尾，cJSON / serde_json / kotlinx 原生处理，链路无关）。
- 音频/文件 = **二进制帧**（帧头 `magic(2B) | type(1B) | len(2B) | payload`，与 JSON 帧在同一传输流上按 magic 区分）。
- 生成器只做"Schema → 结构声明"，编解码逻辑各端手写一次（消息少，不值得引入 protobuf 级别的重量）。

Schema 示例（hello）：

```
{
  "$id": "hello",
  "type": "object",
  "required": ["type", "proto", "device", "fw", "caps"],
  "properties": {
    "type": { "const": "hello" },
    "proto": { "type": "integer", "minimum": 1 },
    "device": { "type": "string" },
    "fw": { "type": "string" },
    "caps": { "type": "array", "items": { "type": "string" } }
  }
}
```

---

## 7. 关键代码骨架

### 7.1 板端传输抽象（components/boss_transport）

```
// components/boss_transport/include/boss_transport.h —— 设计骨架
typedef enum {
    BOSS_LINK_USB,      // TinyUSB CDC1
    BOSS_LINK_WS,       // WiFi WebSocket
    BOSS_LINK_BLE,      // GATT 伴侣
} boss_link_t;

typedef struct {
    boss_link_t link;
```

```
    esp_err_t (*connect)(void);                /* 阻塞直到会话建立        */
    bool      (*up)(void);                     /* 心跳健康                */
    int       (*send_frame)(const uint8_t *f, size_t n);
    int       (*recv_frame)(uint8_t *buf, size_t n, int timeout_ms);
    void      (*disconnect)(void);
} boss_transport_ops_t;

/* 链路监管器：唯一决策点（PRD 4.1 状态机），按优先级尝试 USB > WS > BLE，
 * 切换时通知协议层发 sync(since_seq)。 */
void boss_transport_supervisor_start(void);

/* 帧回调 → boss_protocol 统一入口（seq 去重在此层） */
typedef void (*boss_frame_cb_t)(const uint8_t *frame, size_t len);
void boss_transport_on_frame(boss_frame_cb_t cb);
```

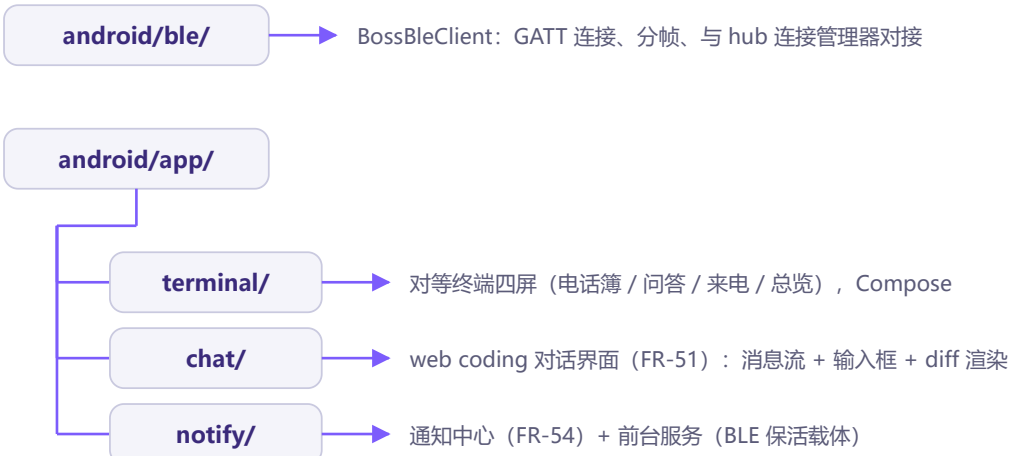
## 7.2 Hub 网关与会话（hub/boss_hub/gateway）

```
# hub/boss_hub/gateway/session.py —— 设计骨架
# CDC 与 WS 两个入口产生同一种 DeviceSession：路由层不关心来自哪个链路
from abc import ABC, abstractmethod

class DeviceSession(ABC):
    @abstractmethod
    async def send(self, msg: dict) -> None: ...   # Envelope(dict) → 帧 → 链路
    def on_message(self, cb) -> None: ...          # 帧回调 → boss_protocol 解码
    @property
    @abstractmethod
    def link_kind(self) -> str: ...                # 'usb' | 'ws' | 'ble-relay'

# hub/boss_hub/core/router.py
# 心跳活跃的 session 集合 → DIRECT 路由；全部失活 → RELAY（云中继 + 推送）
class Router:
    def route(self, envelope: dict) -> None:
        sessions = self.registry.active_sessions()
        if not sessions:
            self.relay_queue.push(envelope)        # seq 入队防丢
        else:
            for s in sessions:
                s.send(envelope)                   # 板 + App 对等扇出
```

### 7.3 安卓 App 模块边界（android/）




App 与板子的关系：**同一 device 身份的两个表现**还是**两个 device**？设计取**两个 device 会话**（App hello 时 `device: "android-app"`），Hub 按 to 语义扇出——避免"一个身份两个链路"的去重复杂度，且 App 天然常在线。

---

## 8. M0 语音垂直切片（板前预研，2026-08-29 插入）

板子 2026-09-13 才发货。为不阻塞开发，M0 在无实机窗口期用 **PC 仿真**完成语音链路的风险验证——所有 STT 逻辑先在 hub-lite 上跑通，板子到货后只需把"WAV 文件输入"换成"串口 PCM 输入"。

### 8.1 架构澄清：STT 的位置

板子是哑终端，**STT 不在板子上。**板子只负责录音与流式传输：

- 采样格式：16kHz / 16bit / mono PCM，约 **32KB/s。**
- USB CDC 裸流带宽富余百倍，**无需压缩、无需降质**；流式发送环形缓冲几 KB 即可，不缓存整段、不落 Flash。
- 16kHz 恰好是 Whisper 原生采样率，直接喂模型。

STT 是 **Hub 接线员模块的可插拔引擎**（主通道）；安卓 App 端兜底可用系统 `SpeechRecognizer`（免费，即用户所说"最差作为输入法"的落点）。

### 8.2 STT 三引擎可插拔（全部实装，配置切换）

| 引擎 | 接入 | 定位 |
| --- | --- | --- |
| `windows-native` | `sapi-lite`（SAPI 封装，支持文件/内存流识别） | 零成本零延迟；中文一般、无热词——**先用它跑通链路** |
| `whisper-local` | `whisper-rs 0.16`（whisper.cpp 绑定，支持 initial_prompt 热词） | 免费离线、术语较好；需模型文件 + CMake 构建 |
| `cloud-api` | HTTP（字节火山/讯飞/OpenAI，配置切换） | 效果最好；按量付费，M2 起可切 |

统一接口（Hub 侧 trait）：

```
/// 16kHz/16bit/mono PCM → 文本；hotwords 为词表纠错提示（引擎支持则注入）
pub trait SttEngine {
    fn name(&self) -> &'static str;
```

```
    fn transcribe(&mut self, pcm: &[i16], hotwords: &[&str]) -> anyhow::Result<String>;
}
```

### 8.3 词表纠错层（生物/专业术语的解法）

专业术语（BWA / fastqc / hg38、仓库名、CLI 命令）识别差的解法**不是换更强的 STT，而是词表约束的 LLM 纠错**：

- 接线员 LLM 拿"已知词表"（联系人=仓库列表 + CLI 命令库 + 用户自定义生物术语词典）对转写文本做音近纠正。
- 词表放 Hub 配置文件，**热更新、不烧固件**——哑终端架构红利。
- 实测中"中等 STT 底座 + LLM 词表纠错"对专有名词的准确率远超单换顶级 STT。

### 8.4 任务与出口标准

| # | 任务 | 依赖 | 状态（2026-08-30） |
|---|---|---|---|
| 1 | Python hub 工程：asyncio 骨架 + STT 引擎（windows/whisper/cloud）+ hub-lite.py CLI（WAV → PCM → 文本） | — | 🔶 进行中（语音链路已用 Rust 验证工具 stt-validate 跑通，§8.5；正移植到 Python hub） |
| 2 | 评测样本：SAPI TTS 生成含生物术语的中文指令 WAV（如"把 BWA 比对跑一遍，参考基因组用 hg38"） | 1 | ✅ 完成，`tools/stt-samples/out/` 五样本 + manifest |
| 3 | 三引擎转写准确率对比 + 词表纠错收益验证 | 2 | 🔶 whisper 首轮完成（§8.5）；windows-native 待用户终端跑 `eval-windows.ps1`（沙箱拦截 SAPI 缓存写入） |
| 4 | 协议 SSOT：九条消息 Schema + 生成器 + 四端生成物（M1 任务 1 前置，纯 PC 侧可完成） | — | ✅ 完成：`protocol/schema/` 九条 + `tools/gen-protocol`（含 `--check` CI 校验）+ Rust crate（6 测试全绿，供固件）+ Kotlin + C 头（gcc 校验通过）+ Python 绑定 |
| 5 | 固件 Rust 骨架：esp-idf-hal 组件编译验证（不烧录） | 4 | ⬜ 未开始 |
| 6 | 语音输入法：词表纠错层（`operator/correct`）+ Windows 光标注入（`input/inject`）+ hub-lite `--type` | 1,3 | 🔶 本日实装（§8.6），冒烟通过 + 注入自检 OK；待用户终端真机注入验证 |

**出口标准**：含生物术语的指令 WAV 经 hub-lite 全链路，终端打出正确文本（三引擎中至少一个 + 纠错层达标）；`--type` 模式将纠错后文本落到光标处。

### 8.5 首轮评测结果（2026-08-29，ggml-base，验证工具 Rust `stt-validate` 于 WSL Arch 构建）

> **说明**：本评测用 M0 验证工具（Rust whisper-rs 引擎）跑通，目的是验证"哑终端 + 16kHz 直喂 whisper"链路成立。**生产 STT 已定用 Python hub 的 faster-whisper**，同一模型精度一致，评测结论复用。

五样本（SAPI TTS 合成，含生物/CLI 术语的中文指令）经 whisper-local 引擎转写：

| 样本 | 期望 | whisper-base 实际 | 关键差异 |
|---|---|---|---|
| 01-daily | 喂，让小张把测试跑一遍。 | 会让小张把特事跑一遍。 | 近音字：测试→特事 |
| 02-bio-bwa | 把BWA比对跑一遍，参考基因组用hg38。 | 8BWA比隊跑一遍,参考基因組用HG38。 | **BWA✓ hg38✓**；繁体混排 |
| 03-bio-fastqc | 用fastqc检查一下测序数据的质量，然后用multiqc汇总。 | 用fastq 检查一下测序数据的质量,然后用multiq会总。 | 术语尾部截断；汇总→会总 |
| 04-cli | 在ai-passport仓库上rebase一下主分支，然后提交PR。 | 在内,Passport倉庫上rebase一下主分支,然後提交PR。 | **rebase✓ PR✓**；仓库名偏 |

| 样本 | 期望 | whisper-base 实际 | 关键差异 |
|---|---|---|---|
| 05-mixed | 看一下github上CI的结果，把clippy的警告都修掉。 | 看一下Graphub，CI的結果，clippy的警告都修掉。 | **CI✓ clippy✓**；github→Graphub |

结论：

1. **拉丁字母术语基本存活**（BWA/HG38/rebase/PR/CI/clippy 全对或仅大小写差异）——哑终端 + 16kHz 直喂 whisper 的链路设计成立。
2. 错误集中在两类，**均在词表纠错层（§8.3）的射程内**：繁简混排（比隊→比对、倉庫→仓库）与近音字（特事→测试、会总→汇总、Graphub→github）——词表 + 音近规则可修。
3. base 模型对中文一般；后续可换 small（~466MB）再测一轮，预期普通话错误率明显下降。
4. windows-native 引擎链路可通但 SAPI 需写 `%LOCALAPPDATA%\Microsoft\Speech` 适配缓存，TRAE 沙箱内被拦截（识别返回空）——用户终端跑 `tools/stt-samples/eval-windows.ps1` 出对比数据。
5. **生产路径（2026-08-30 定稿）**：Hub 用 Python（原定计划，AI 逻辑优先）。M0 的 Rust whisper-rs 引擎属验证工具，迁至 `tools/stt-validate/` 保留；Python hub 用 `faster-whisper`（同 ggml 模型，精度一致、加载更快），windows-native 引擎原样移植为 PowerShell 子进程实现。

### 8.6 语音输入法（PC 侧第一优先级，2026-08-30 定稿）

PC 端定位不是"转写展示台"，而是**系统级语音输入法**：STT + 词表纠错后的文字**直接打到当前前台光标处**——用户在哪个应用编辑，文字就落在哪里。**板子长按 OK 键是触发端**（M1 接 gateway `voice.start`，推-按说话）。agent 对话查询、意图路由都是后话，先把语音输入链路做健壮。

```
板子长按 OK → (USB CDC PCM 流) → Hub STT → 词表纠错 → Windows 光标注入
```

PC 侧四段（本日实装）：

| 模块 | 职责 | 健壮性 |
|---|---|---|
| `operator/stt.py` | 三引擎可插拔（windows / whisper / cloud） | 空转写不注入；引擎异常不静默丢字 |
| `operator/correct.py` | 词表纠错层（无 LLM 依赖，确定性） | 术语大小写归一（词边界，不误伤子串）+ 音近/形近替换（长词优先）+ 繁简归一 + 空白归一；用户词表 JSON 热更新、不烧固件 |
| `input/inject.py` | 文字落到光标处（Windows） | SendInput(UNICODE) 逐字键入主通道 + 剪贴板 Ctrl+V 兜底；UIPI 拦截自动降级；无前台窗口明确报错 |
| `dictation.py` | 编排 STT → 纠错 → 注入 | 空结果/注入失败记录不阻断；注入方式 type / paste / auto |

验证命令：

```
python hub/hub-lite.py tools/stt-samples/out/02-bio-bwa.wav --engine whisper --model <模型> --hotwords BWA,hg38 --type
```

- `--type`：纠错后文字注入当前光标处（语音输入法模式）
- `--no-correct`：关闭纠错层，度量纠错收益（§8.4 任务 3）
- `--selftest`：仅校验注入能力（前台窗口 + SendInput 通路），不发字

---

## 9. M1 任务分解（垂直切片，USB 先行）

| # | 任务 | 产出 | 依赖 |
|---|---|---|---|
| 1 | 协议 SSOT：hello/hello_ack/presence/state/ask/answer/ring/ring_ack/hangup 九条 Schema + 生成器 | `protocol/` + 四端生成物 | — |
| 2 | Hub 骨架（Python）：CDC 入口 + DeviceSession + Router + 事件总线 | `hub/` 可跑，`python -m hub serve` 监听串口 | 1 |
| 3 | 固件传输层（Rust）：boss_transport（usb 实现）+ boss_protocol（serde_json + seq 去重） | 插线即 hello 握手成功 | 1 |
| 4 | 固件手机系统框架（C + LVGL）：状态栏/桌面/导航/事件层（boss_app）+ 电话簿/问答/来电 UI | 全 UI 流程可演示（模拟数据） | 3 |
| 5 | 插件打样（Python）：claude-code hooks → adapter-core → Hub | 真实会话状态上电话簿 | 2 |
| 6 | 端到端联调：真实 Claude Code 权限 → 板子批准 → 回传 | M1 出口标准达成 | 4+5 |

任务 2/3/4 可三人并行；单人则按序。任务 1 是一切的前置，**协议 Schema 评审是 M1 的第一道闸门。**M0（第 8 节）在板子到货前完成，其任务 4 即本表任务 1。

---

## 10. 工程风险与待办

| # | 事项 | 说明 |
|---|---|---|
| E1 | Trae CLI 插件能力待调研 | 若无开放 hook，兜底方案：外层 wrapper 进程（解析 stdout/exit code），能力降级同 Codex |
| E2 | Codex 权限"事后通知"模式的产品体验 | PRD 黄灯语义在该模式下退化的具体 UI 措辞，DESIGN_PLUGIN 中细化 |
| E3 | 生成器首版范围 | M1 只做九条消息；后续增量加 schema 即重新生成，四端同步 |
| E4 | monorepo CI | 固件（ESP-IDF 构建）与 Rust 业务层（cargo build/test）双轨 + Hub/插件（pytest）一轨，`.github/workflows` 分离 |
| E5 | Android BLE 分帧重传协议 | DESIGN_ANDROID 中定义（帧头含 seq + 分片号，选择性重传） |

# BOSS PHONE 通讯协议规范

| 项目 | 内容 |
| --- | --- |
| 状态 | Draft v0.1 |
| 上游 | BOSS_PHONE_PRD.md §7 · BOSS_PHONE_DESIGN_OVERVIEW.md §6 |
| 日期 | 2026-08-24 |

本文档是**协议的唯一事实来源（SSOT）**。所有端（板子固件 C + Rust 业务层 / PC 总机房 Python / 安卓 App Kotlin）必须依据 `protocol/schema/*.json` 生成代码，禁止手写消息结构。本文描述字段语义与交互时序。

---

## 1. 设计原则

1. **只增不改**：消息类型与字段只增不删不改语义；未知字段一律忽略，未知消息类型一律忽略并回复 `error(type=unsupported)`。
2. **hello 版本协商**：每次连接握手交换 `proto` 整数版本，取 min 共同版本。
3. **caps 能力协商**：双方声明能力字符串集，按对方能力裁剪功能（见 §6）。
4. **统一信封**：所有消息套同一信封，与链路无关；链路差异只在帧封装层（§4）。
5. **可靠语义**：控制消息有 `seq` 与去重；媒体（voice/audio/file chunk）允许少量丢弃，靠上层重试。

---

## 2. 消息信封（Envelope）

```
{
  "type":  "state",
  "from":  "agent:repo-b",
  "to":    "device",
  "seq":   12345,
  "ts":    1756000000000,
  "payload": {}
}
```

| 字段 | 类型 | 必填 | 说明 |
| --- | --- | --- | --- |
| type | string | 是 | 消息类型，见 §3 |
| from | string | 是 | 发送方 ID：`agent:<repo>` / `hub` / `operator` / `device` / `android-app` |
| to | string | 是 | 接收方：`device` / `operator` / `agent:<repo>` / `all`（多端扇出见 §8） |
| seq | uint | 控制消息必填 | 发送方本地递增；对端据此去重与补收（§5） |
| ts | uint64 | 否 | 毫秒时间戳（发送时本地时钟，仅展示） |
| payload | object | 是 | 各类型专属字段 |

> 媒体帧（voice/audio/file 数据段）不套 JSON 信封，走二进制帧（§4.2），其控制信息由配套控制消息携带。

---

# 3. 消息字典

> 字段表只列关键字段；完整 JSON Schema 见 `protocol/schema/*.json`。▲表示数组元素对象。

## 3.1 握手与会话

### hello（任意端 → Hub）

| 字段 | 类型 | 说明 |
|---|---|---|
| proto | uint | 客户端支持的协议版本（当前 1） |
| device | string | 设备身份：`passport:<mac>` / `android-app:<uuid>` |
| fw | string | 固件/App 版本（如 `0.1.0`） |
| caps | string[] | 能力集（§6） |
| resume_seq | uint | 上次会话最后收到的 seq，用于断线补收 |

### hello_ack（Hub → 端）

| 字段 | 类型 | 说明 |
|---|---|---|
| proto | uint | 协商后的共同版本 |
| caps | string[] | 服务端能力集 |
| device_id | string | 分配/确认的设备号 |
| server_time | uint64 | 服务端毫秒时间戳（端校准时钟用） |

## 3.2 Agent 生命周期与状态

### presence（插件 → Hub → 端）

| 字段 | 类型 | 说明 |
|---|---|---|
| agent_id | string | 仓库名（电话簿联系人名） |
| online | bool | 上线/下线 |
| cli | string | `claude-code` / `codex` / `opencode` / `trae` |
| cwd | string | 工作目录 |
| caps | ▲{permission_suspend, usage, ...} | 该 Agent 的能力（Hub 据此降级） |

### state（插件 → Hub → 端）

| 字段 | 类型 | 说明 |
|---|---|---|
| agent_id | string | — |
| state | enum | `running` / `waiting_input` / `blocked` |
| summary | string | 状态摘要（PC 端生成，板端只显示） |
| task | string | 当前任务简述（可空） |
| since | uint64 | 状态起始时间 |

> 状态灯映射：`running`→绿，`waiting_input`→黄，`blocked`→红。

## 3.3 权限请求与回答（核心交互）

### ask（插件 → Hub → 端）

| 字段 | 类型 | 说明 |
|---|---|---|
| ask_id | string | 问题组 ID（UUID） |
| agent_id | string | 发起 Agent |
| reason | string | 为什么需要权限/输入（摘要） |
| questions | ▲[] | 问题数组（有序） |

questions▲ 元素：

| 字段 | 类型 | 说明 |
|---|---|---|
| q_no | uint | 题号（从 1 起） |
| kind | enum | `single`（单选）/ `multi`（多选）/ `input`（需用户补全） |
| prompt | string | 题目文本 |
| options | string[] | 选项（input 题可空） |
| optional | bool | 是否可跳过 |
| default | string | 默认值（可选，input 题预填） |

### answer（端 → Hub → 插件）

| 字段 | 类型 | 说明 |
|---|---|---|
| ask_id | string | — |
| answers | ▲{q_no, kind, selection[], text} | 每个被答题目 |
| answered_by | string | `device` / `android-app`（多端场景先答先得，见 §8） |
| ts | uint64 | 回答时间 |

- 单选：`selection` 单元素；多选：`selection` 数组；input：`text` 为补全内容。

## 3.4 通话与来电

### ring（Hub → 端）

| 字段 | 类型 | 说明 |
|---|---|---|
| ring_id | string | 通话 ID |
| agent_id | string | 来电方 |
| kind | enum | `agent_call` / `red_alert` / `operator` |
| reason | string | 来电/报警原因摘要 |
| summary | string | 阻塞摘要（red_alert 必填，如"CI 失败：编译错误"） |
| ringtone | string | 铃声标识（可空，用默认） |

### ring_ack（端 → Hub）

| 字段 | 类型 | 说明 |
|---|---|---|
| ring_id | string | — |
| accept | bool | 接听/拒接 |

### audio（双向，媒体帧）

| 字段（控制伴消息或帧头） | 说明 |
|---|---|
| call_id | 通话 ID（帧头字段） |
| opus 帧 | 二进制载荷（§4.2） |
| seq | 帧序号（丢帧检测） |

### hangup（双向）

| 字段 | 说明 |
|---|---|
| call_id | — |
| reason | `local` / `remote` / `error` |

## 3.5 语音对讲（walkie-talkie）

### voice（端 → operator）

| 字段 | 说明 |
|---|---|
| voice_id | 片段 ID |
| opus | 二进制载荷（媒体帧，§4.2） |
| dur_ms | 片段时长 |
| final | bool：长按 OK 松手前最后一段 |

### voice_ack（operator → 端）

| 字段 | 说明 |
|---|---|
| voice_id | 对应片段 |
| transcript | 转写文本 |
| intent | 解析意图（`call_agent` / `dispatch_task` / `memo` / `unknown`） |
| reply | 接线员文字回复（板端显示用） |
| route | 若 `call_agent`，含目标 agent_id |

## 3.6 遥测与驾驶舱

### telemetry（Hub → 端）

| 字段 | 说明 |
|---|---|
| cpu | PC CPU % |

| 字段 | 说明 |
| --- | --- |
| mem | 内存占用（used/total 字符串或两个 uint） |
| agents_active | 活跃 Agent 会话数 |
| uptime | Hub 运行时长秒 |
| period | 推送周期（端据此决定是否降频） |

### usage（Hub → 端）

| 字段 | 说明 |
| --- | --- |
| today | ▲{tokens, cost, per_agent[{agent_id,tokens}]} |
| month | ▲{tokens, cost, budget} |

### commit（插件 → Hub → 端）

| 字段 | 说明 |
| --- | --- |
| agent_id | — |
| sha | 短 sha |
| branch | 分支 |
| message | 提交消息（截断到 ≤60 字符） |
| pushed | bool 是否已推送远端 |

### ci（Hub → 端）

| 字段 | 说明 |
| --- | --- |
| agent_id | — |
| status | `pass` / `fail` / `running` / `unknown` |
| summary | 失败原因（fail 时） |

> `ci=pass` 不改灯；`ci=fail` 直接驱动仓库红灯（与 `state=blocked` 同效，走 ring(red_alert)）。

## 3.7 备忘录与文件

### file（双向，媒体帧 + 控制）

| 字段 | 说明 |
| --- | --- |
| file_id | 传输 ID |
| kind | `memo` / `avatar` / `ota` |
| meta | ▲{name, size, mime}（首帧） |
| offset | 本次分块偏移 |
| total | 总字节 |
| data | 二进制载荷（媒体帧） |

### file_ack（双向）

| 字段 | 说明 |
| --- | --- |
| file_id | — |
| offset | 已确认接收到的最大偏移（续传点） |
| status | `ok` / `retry` / `abort` |

### memo_result（Hub → 端）

| 字段 | 说明 |
| --- | --- |
| memo_id | 本地备忘 ID |
| title | 转写标题 |
| text | 转写全文 |
| status | `transcribed` / `failed` |

### memo_dispatch（端 → Hub）

| 字段 | 说明 |
| --- | --- |
| memo_id | — |
| target | `operator` / `agent:<repo>` |

## 3.8 同步与错误

### sync（端 → Hub）

| 字段 | 说明 |
| --- | --- |
| since_seq | 从哪个 seq 开始补收 |

### sync_data（Hub → 端）

| 字段 | 说明 |
| --- | --- |
| events | Envelope[]（按 seq 升序） |
| next_seq | 下一可用 seq |

### error（任意）

| 字段 | 说明 |
| --- | --- |
| code | 错误码（§5.3） |
| msg | 人类可读说明 |
| ref | 触发错误的消息 seq/type |

---

# 4. 帧封装（链路无关）

同一传输流上两种帧按 magic 区分：控制帧（JSON）与媒体帧（二进制）。

```
JSON 控制帧：  {envelope_json}\n            # magic 隐含：以 { 开头
媒体二进制帧：  MAGIC(2B)|TYPE(1B)|FLAGS(1B)|LEN(2B)|PAYLOAD(LEN)
               MAGIC = 0xAA 0x55
               TYPE  = 0x01 voice | 0x02 audio | 0x03 file-data | 0x04 file-meta
               FLAGS = bit0 final 段 | bit1 first 段
```

### 4.1 各链路分帧细则

| 链路 | 载体 | 特殊处理 |
|---|---|---|
| USB (CDC1) | 字节流 | 直接写帧；接收按 \n 或 magic+len 解析 |
| WiFi (WS) | WebSocket binary/text | 控制帧用 text；媒体帧用 binary |
| BLE | GATT RX/TX 特征值 | MTU 协商 247B → 媒体帧切片，加 FRAG(2B 分片号) 前缀 + 分片 ACK（§DESIGN_ANDROID） |
| 云中继 | 出站 WS 隧道 | 与 WiFi 同 |

### 4.2 seq 语义

- 每发送方独立维护 seq，从 1 递增（不跨端共享）。
- 控制帧必带 seq；媒体帧用 seq 做丢帧检测（不补，只丢弃后提示）。
- 断线重连：端在 hello 里带 resume_seq，Hub 从 resume_seq+1 开始补发 sync_data；端按 (from, seq) 去重。

### 4.3 错误码

| code | 含义 |
|---|---|
| 4000 | 协议版本不兼容 |
| 4001 | 未知消息类型 |
| 4002 | 必填字段缺失 |
| 4003 | 未握手（先 hello） |
| 4100 | ask 已过期/不存在 |
| 4101 | 目标 Agent 离线 |
| 4200 | 传输分块校验失败 |
| 4300 | 设备忙（媒体通道占用） |

---

## 5. 能力协商（caps 词汇表）

| cap | 含义 | 消费者 |
|---|---|---|
| proto.v1 | 支持协议 v1 | 所有 |
| audio.opus16k | 支持 Opus 16k 单声道 | 板子/App/接线员 |
| audio.pcm | 支持裸 PCM（USB/WiFi 链路） | 板子 |
| input.voice | 支持长按 OK 语音输入 | 板子 |

| cap | 含义 | 消费者 |
|---|---|---|
| `agent.suspend` | Agent 支持挂起等回答 | 插件（permission_suspend） |
| `agent.usage-exact` / `usage-approx` | token 统计精度 | 插件 |
| `file.memo` / `file.avatar` / `file.ota` | 文件传输类型支持 | 端 |
| `relay` | 支持云中继路由 | 板子/App |
| `push.fcm` / `push.vendor` | 推送通道 | App |
| `ui.peer-term` | 对等富终端 UI | App |

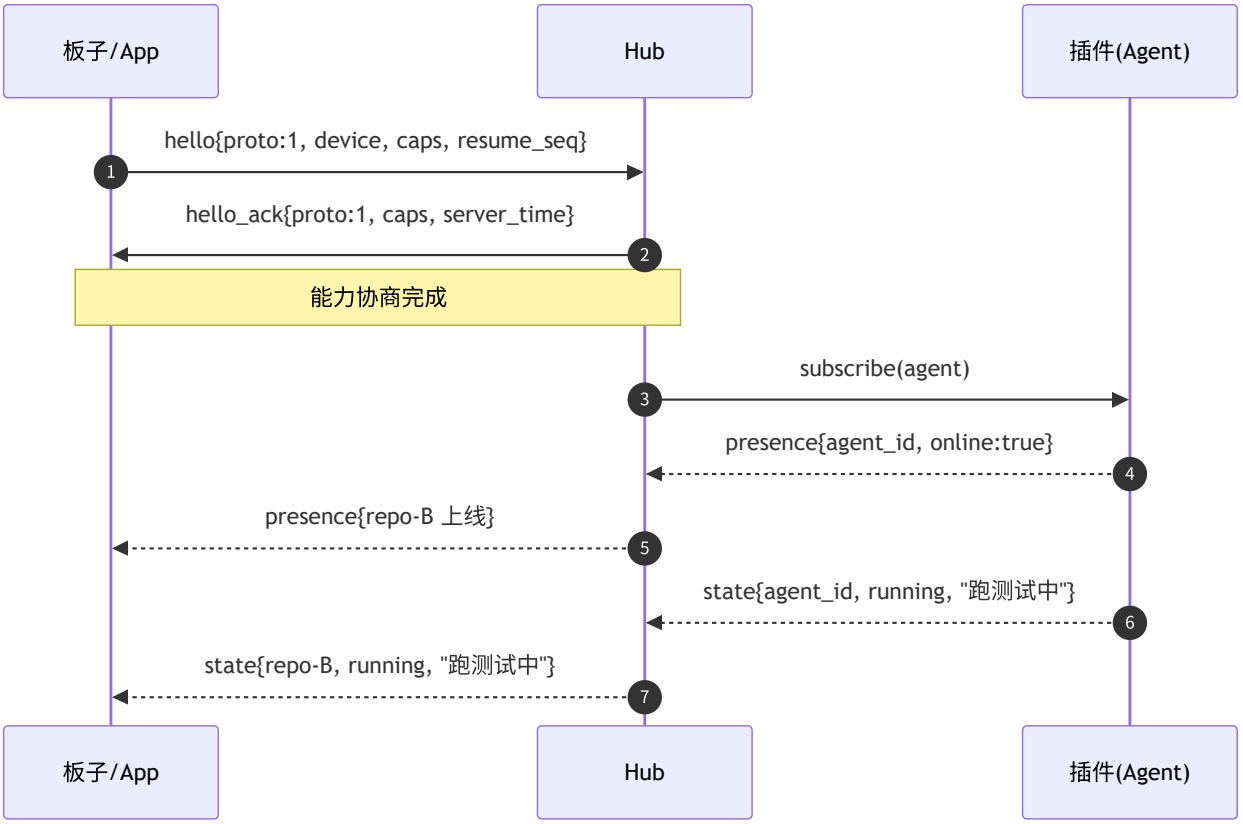
协商规则：`hello` 双向声明 → Hub 合并为每设备 `effective_caps` → 降级逻辑在各端执行（例：板子无 `file.ota` 时 Hub 不推 OTA 帧）。

---

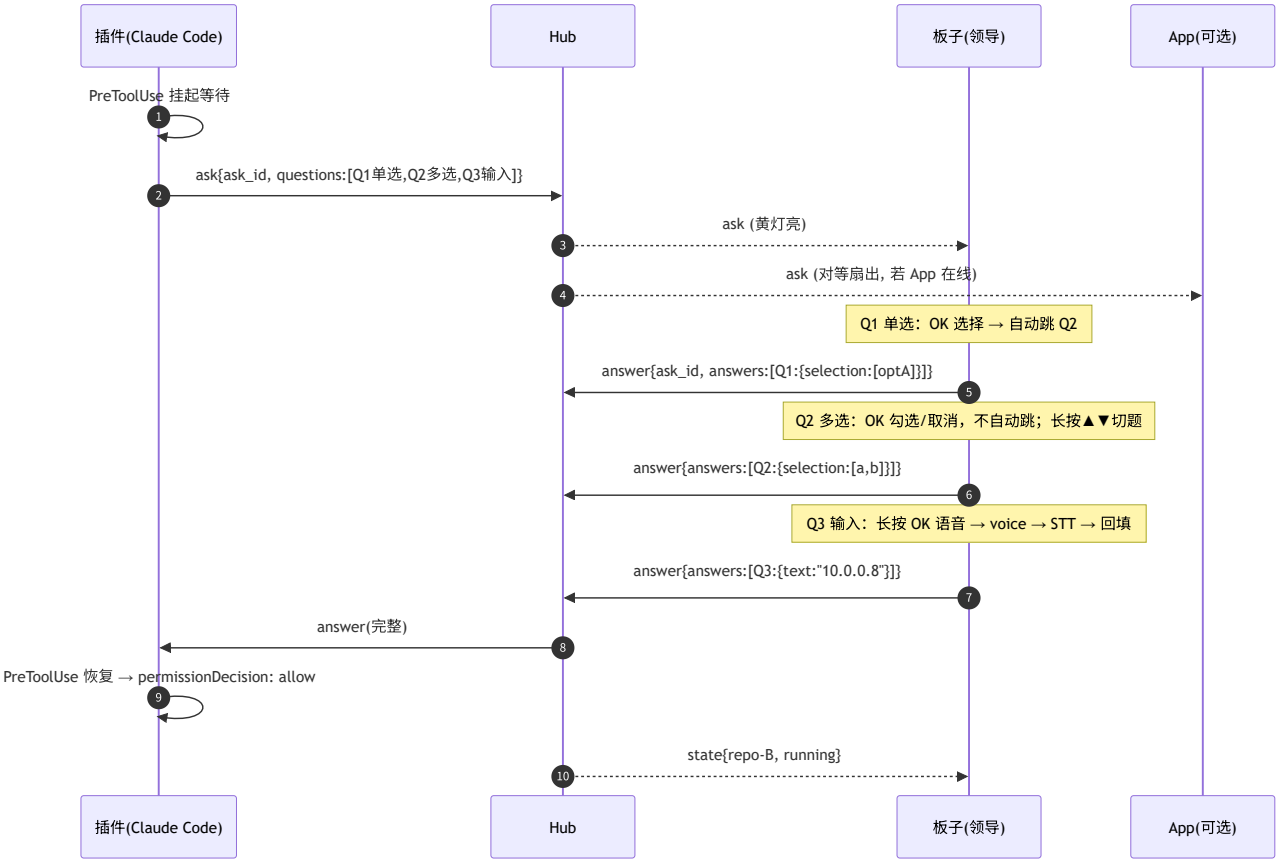
## 6. 核心交互时序

> 以下为 mermaid 时序图；PDF 渲染后为矢量图。

### 6.1 连接握手与状态订阅

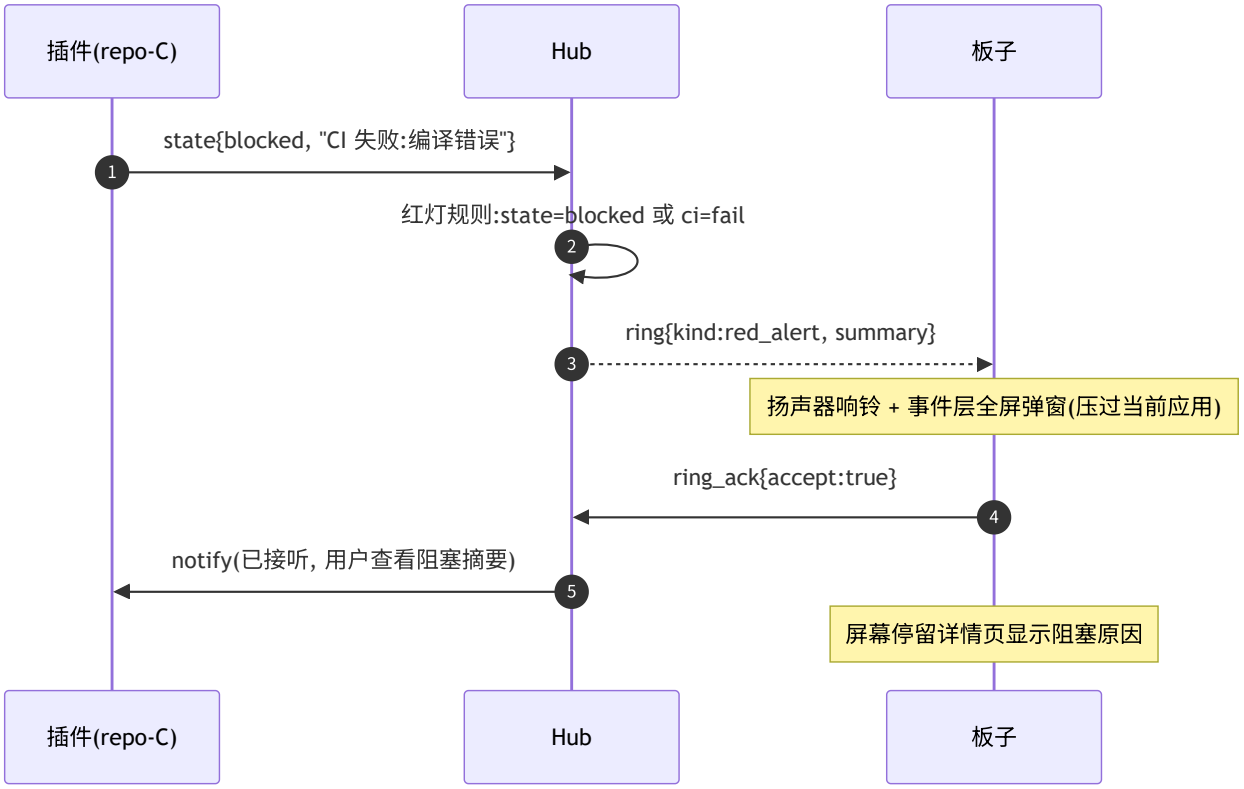


## 6.2 权限请求（含多题、单选/多选/输入混合）

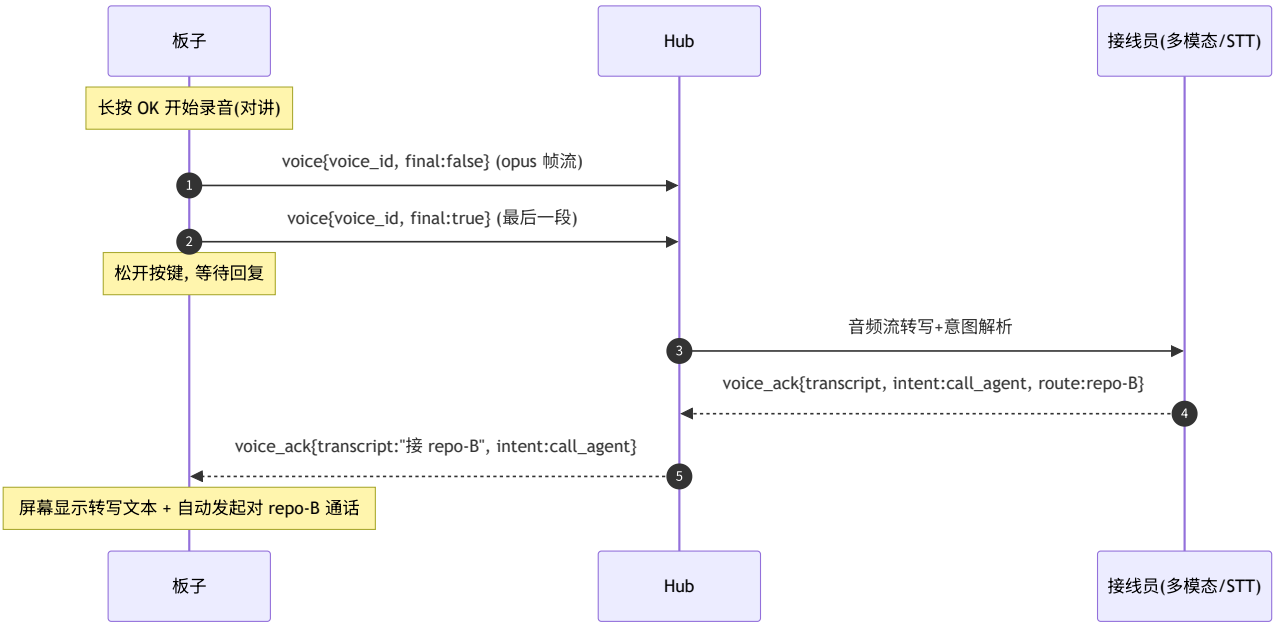



> **先答先得**：若 App 先对某题作答，Hub 向板子下发 `answer_done{ask_id, q_no}`，板子该题置为"已处理"并跳过，避免双端重复回答。

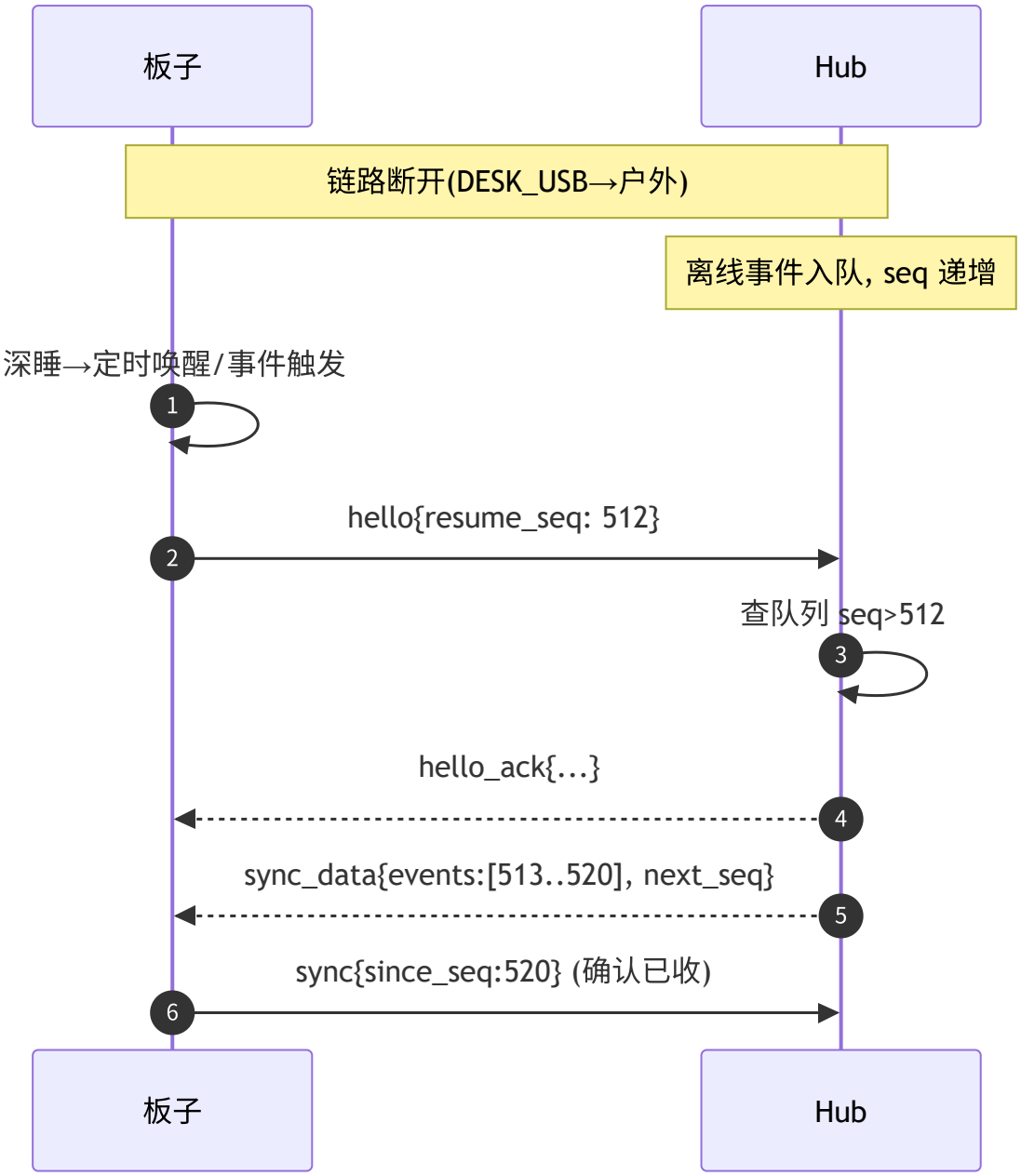
## 6.3 主动来电与红灯报警



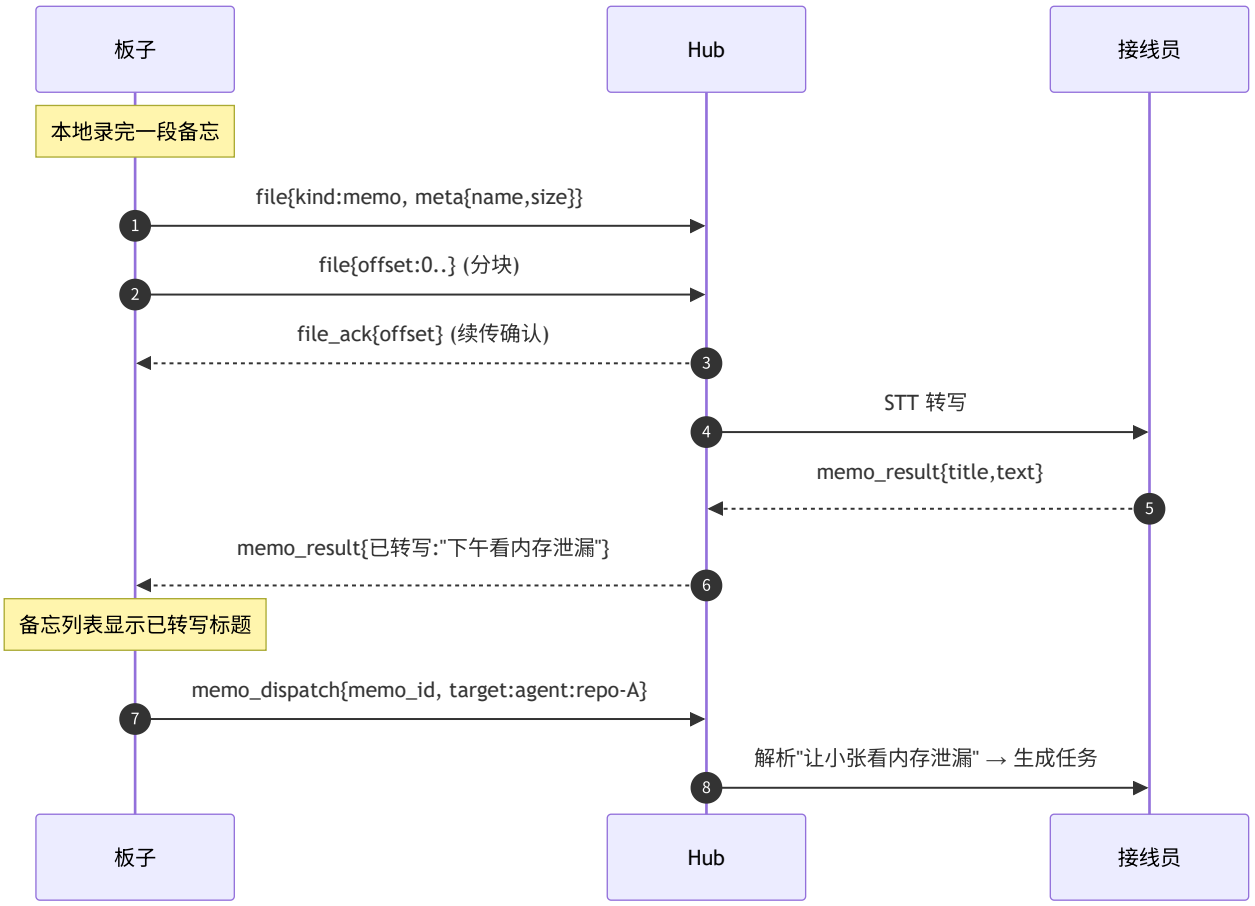
## 6.4 语音对讲（领导→接线员）



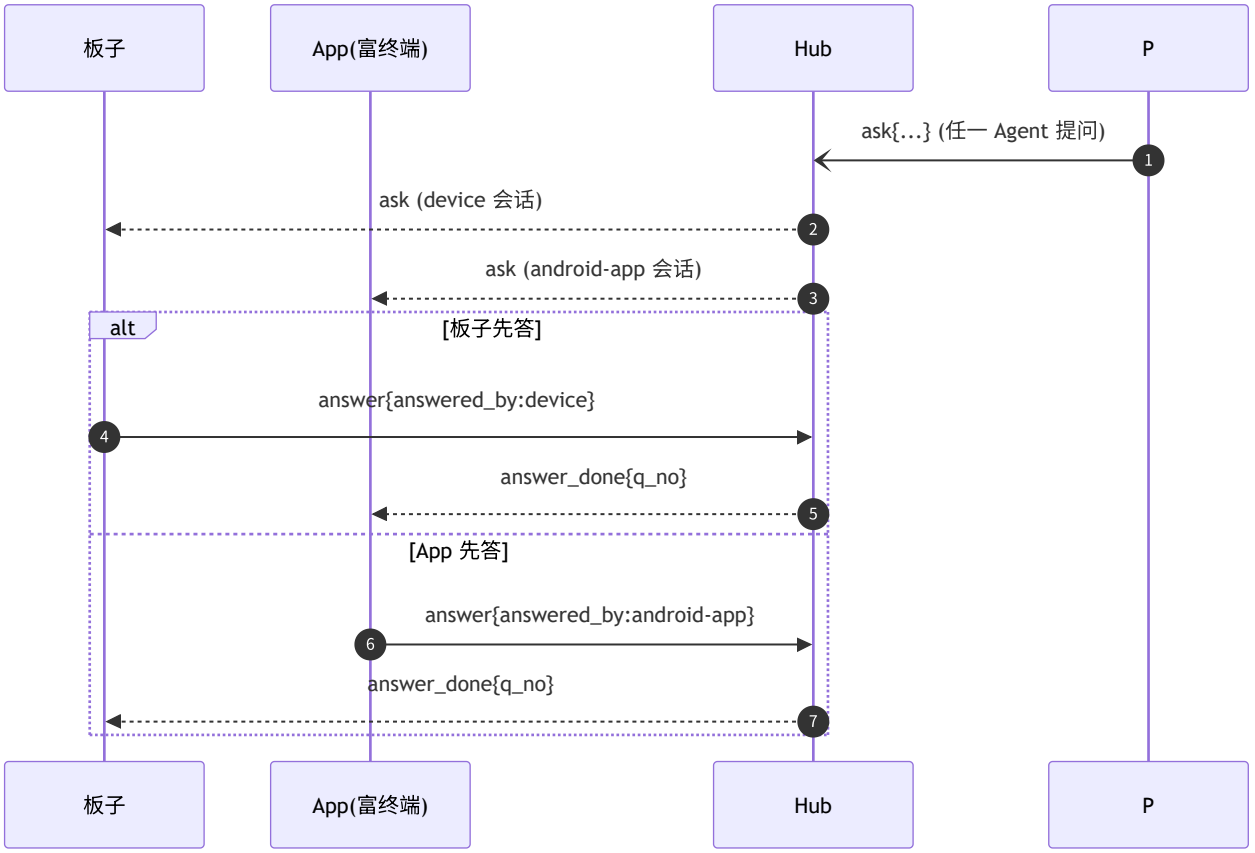
### 6.5 断线补收（sync）


板子
Hub
链路断开(DESK_USB→户外)
离线事件入队, seq 递增
深睡→定时唤醒/事件触发
1
hello{resume_seq: 512}
2
查队列 seq>512
3
hello_ack{...}
4
sync_data{events:[513..520], next_seq}
5
sync{since_seq:520} (确认已收)
6
板子
Hub

## 6.6 备忘录上传与派发




## 6.7 多端对等扇出（板 + App）



## 7. 兼容性保证

1. 新增消息类型：老端收到未知 type → 忽略（原则 1）。
2. 新增字段：老端忽略未知字段。
3. 语义变更：**禁止**——只能新增类型/字段模拟新语义（如需要"同意且不再询问"→ 新增 `ask_ack{remember:true}`，不改 `answer`）。
4. 版本不兼容（`proto` 差距过大）：hello_ack 带 `4000` 错误，端提示升级。

---

## 8. 待定项

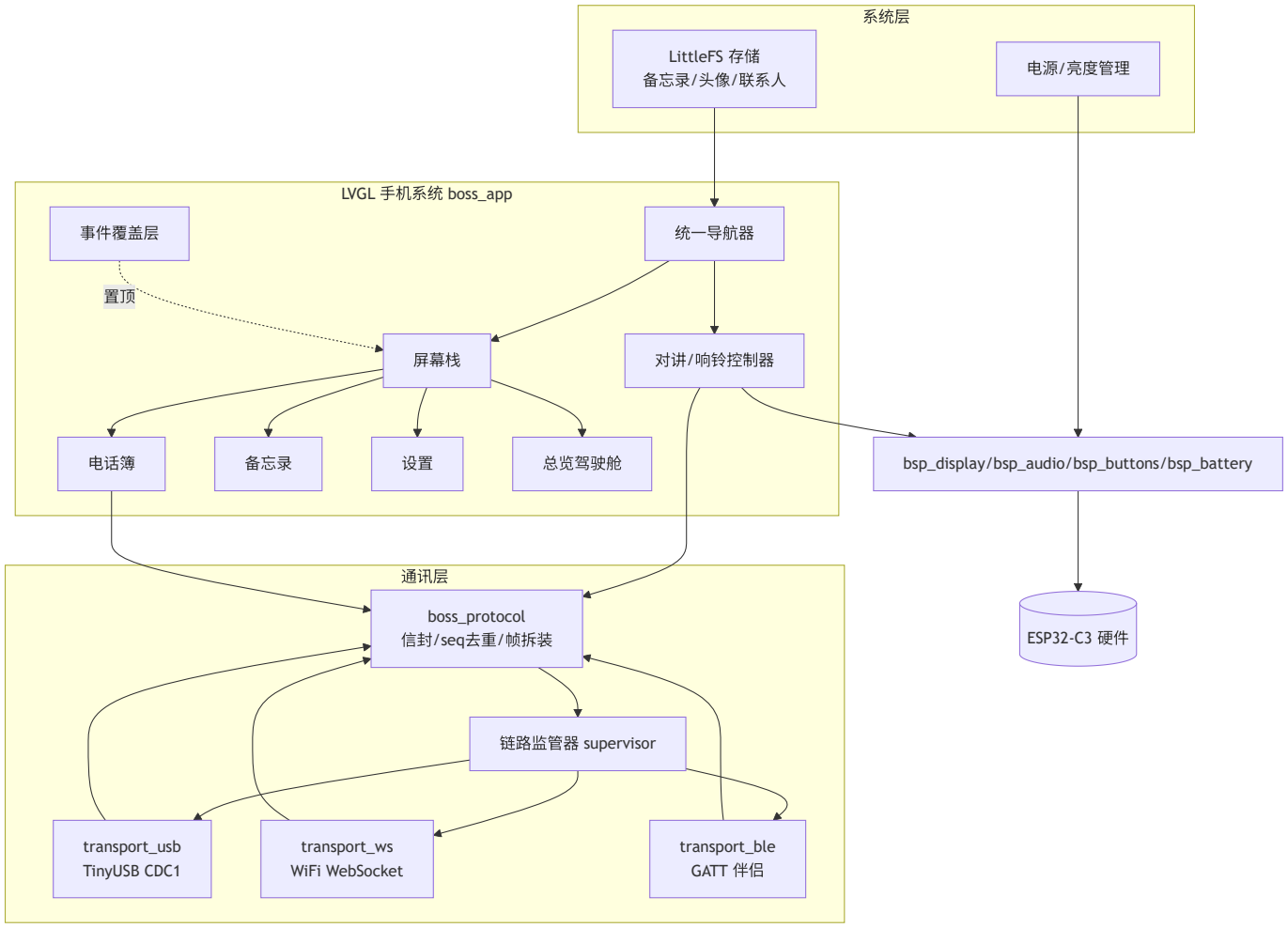
| # | 事项 |
|---|---|
| P1 | media 帧的 Opus 包长/采样率是否固定（建议固定 60ms/16k/单声道） |
| P2 | BLE 分帧 ACK 的窗口大小与重传超时（DESIGN_ANDROID 定） |
| P3 | `answer_done` 与 `ring` 的幂等规则在并发场景下的细化 |
| P4 | 云中继握手是否需要额外鉴权（token 由 PC 端配对生成） |

# BOSS PHONE 板端固件设计

| 项目 | 内容 |
| --- | --- |
| 状态 | Draft v0.1 |
| 上游 | BOSS_PHONE_PRD.md §5.1/§5.2/§5.3/§8 · DESIGN_OVERVIEW §7.1 |
| 硬件 | ESP32-C3：RISC-V 160MHz / 400KB SRAM / 8MB Flash / 无 PSRAM / WiFi+BLE / ES8311 / ST7789P3 |
| 日期 | 2026-08-24 |

本文定义固件侧三个新组件（`boss_transport` / `boss_protocol` / `boss_app`）的模块设计、Flash 分区、任务模型与内存预算。**现有 `components/bsp` 与 `main/` demo 体系零改动。**

## 1. 组件拓扑



## 2. Flash 分区表（8MB）

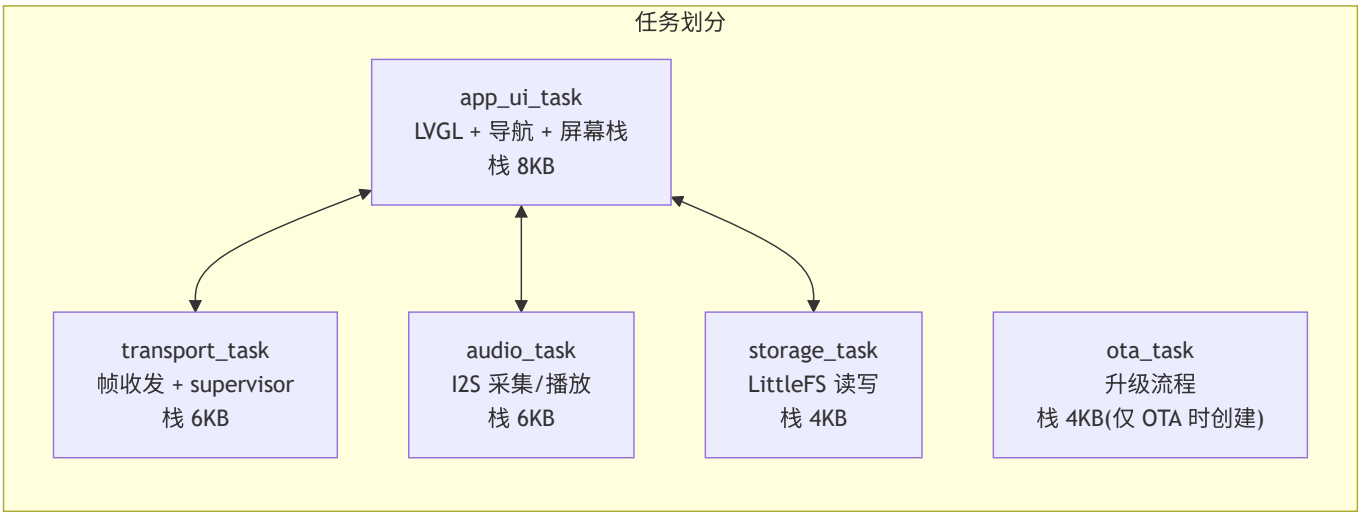



注：app0/app1 各 2MB 满足 OTA 双分区；storage 保留充足空间给录音备忘（1 分钟 Opus≈120KB，3.9MB≈30 分钟）

| 分区 | 类型 | 大小 | 用途 |
| --- | --- | --- | --- |
| nvs | data | 24KB | WiFi/蓝牙配对、设置、seq 持久化 |
| phy_init | data | 4KB | 射频校准 |
| app0 (factory) | app | 2MB | 出厂固件（OTA A） |
| app1 (ota) | app | 2MB | 升级目标（OTA B） |
| ota_data | data | 8KB | OTA 状态 |
| storage | data(LittleFS) | 3.9MB | 备忘录/头像/字体/升级暂存 |

> 与现有仓库 `sdkconfig.defaults`（8MB 无 PSRAM）对齐；新固件以 `factory` 槽烧录。

## 3. 任务模型与内存预算（400KB SRAM）




| 分配项 | 预算 | 说明 |
| --- | --- | --- |
| LVGL 显示缓冲 | ~46KB | 复用现有单 DMA 缓冲方案，不加大 |
| LVGL 对象/动态 | ~60KB | 默认配置 + 精简对象池 |
| 音频 DMA/PCM 缓冲 | ~24KB | `bsp_audio` 已有 |
| Opus 编码缓冲 | ~12KB | 16k 单声道 60ms 帧 |
| transport 帧缓冲 | ~8KB | RX/TX 各 4KB 环形 |

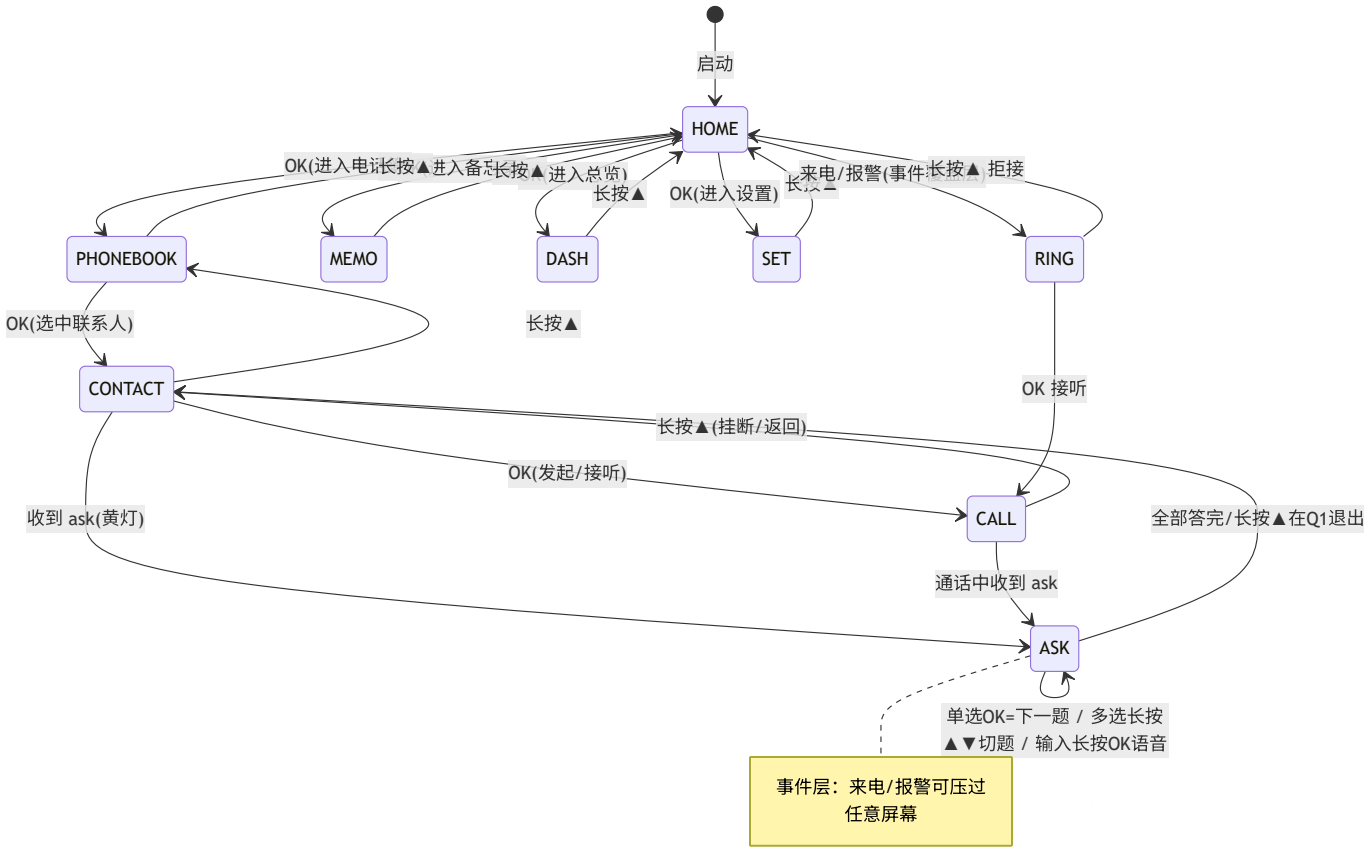
| 分配项 | 预算 | 说明 |
| --- | --- | --- |
| 任务栈合计 | ~28KB | 上述 5 任务 |
| WiFi/BLE 协议栈堆 | ~40KB | 由 esp_wifi/esp_ble 内部 |
| cJSON/协议解析暂存 | ~16KB | 信封解析峰值 |
| 系统/驱动余量 | ~剩余 | 保持 ≥15% 空闲堆余量 |

**内存纪律（PRD 8.2）**：

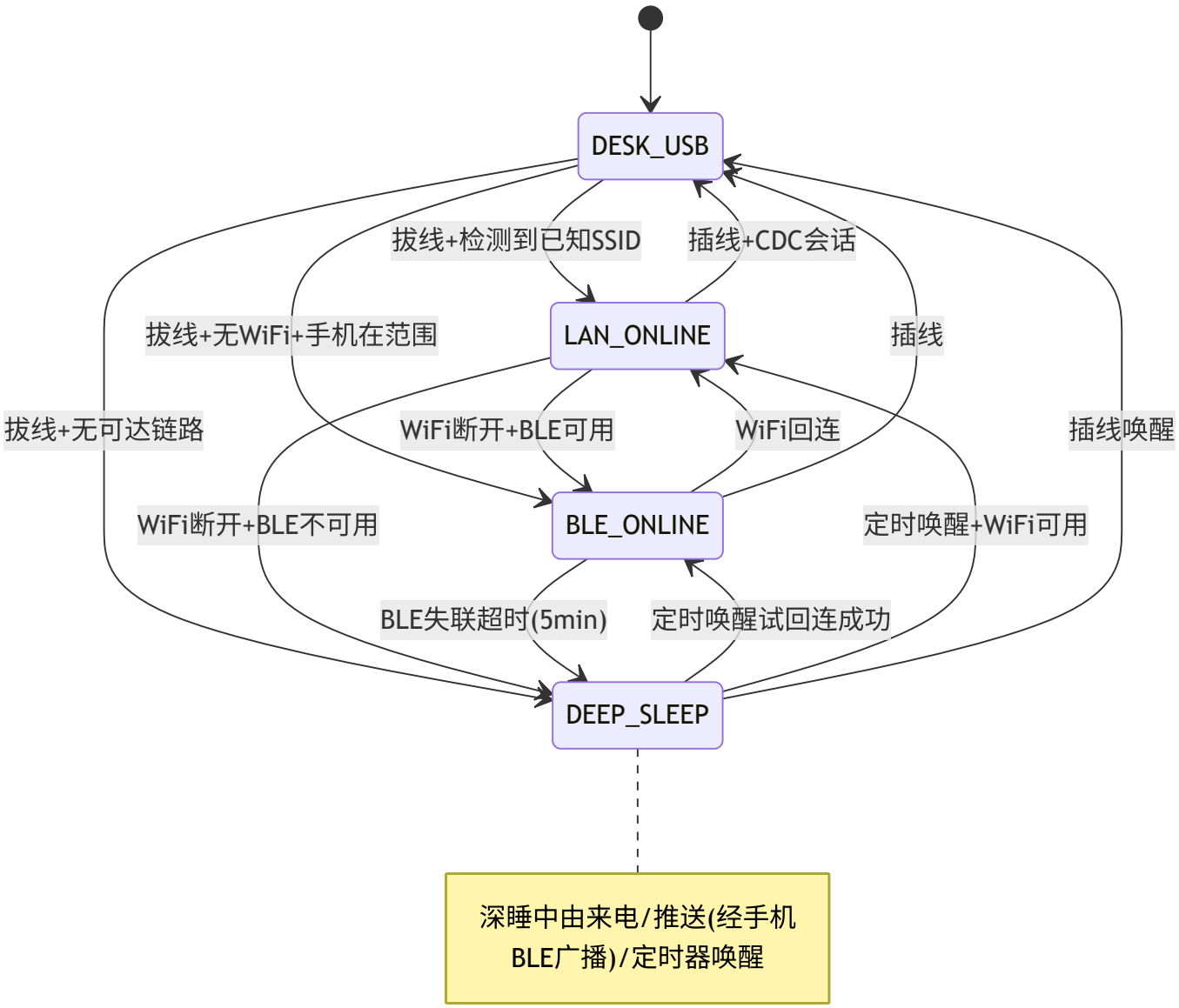
- USB 模式关 WiFi/BLE 射频（省电 + 省堆），仅保留 CDC。
- BLE 模式不建 WiFi，二者不同时开（互斥状态机保证）。
- 备忘录录音仅在 storage_task 空闲时写盘，避免 I2S 中断抖动。
- 新增任何缓冲必须过"400KB 实测基线"检查（AGENTS.md 约束）。

## 4. 统一导航状态机（boss_app）

三键语义见 PRD §6.1。核心是"长按 ▲ = 全局后退"。



## 5. 链路监管器状态机（boss_transport）




- **切换动作**：切换前 protocol 发 sync(since_seq)，保证消息不丢。
- **会话在位判定**：USB 模式以 CDC 会话活跃（非仅插线）为准；WiFi 以 WS 心跳为准。
- 切换瞬间双链路并存 → 板端按 (from,seq) 去重。

## 6. transport 接口（骨架）

```
/* components/boss_transport/include/boss_transport.h */
typedef enum { BOSS_LINK_USB, BOSS_LINK_WS, BOSS_LINK_BLE } boss_link_t;

typedef struct {
    boss_link_t link;
    esp_err_t (*connect)(void);                 /* 阻塞直到会话建立 */
    bool      (*up)(void);                      /* 心跳健康 */
    int       (*send_frame)(const uint8_t *f, size_t n);
    int       (*recv_frame)(uint8_t *buf, size_t n, int timeout_ms);
    void      (*disconnect)(void);
```

```
} boss_transport_ops_t;

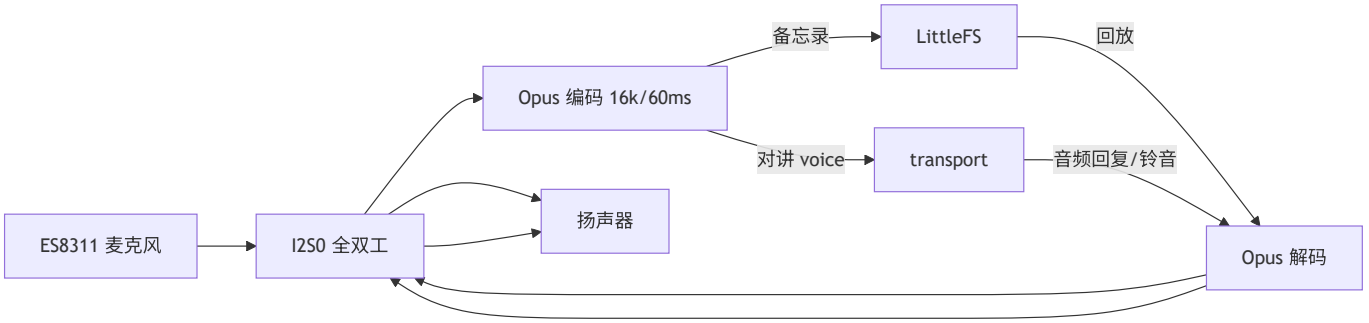
void boss_transport_supervisor_start(void);
void boss_transport_on_frame(boss_frame_cb_t cb); /* → boss_protocol */
```

## 7. boss_protocol 骨架

```
/* components/boss_protocol/include/boss_protocol.h */
typedef void (*boss_msg_cb_t)(const cJSON *envelope);

void boss_protocol_init(boss_msg_cb_t cb);
int  boss_protocol_send(const char *type, const char *to, cJSON *payload);
/* 内部：seq 自增、帧组装、按 (from,seq) 去重表(最近 256 条 LRU) */
void boss_protocol_handle_sync(const cJSON *sync_data); /* 补收入库 */
```

## 8. 音频管线




- 对讲：长按 OK 采集 → Opus 帧发 `voice`；松开 `final:true`。
- 备忘录：与对讲同管线，但落盘 LittleFS（`kind:memo` 文件）。
- 响铃：本地预存铃声（或 Hub 下发 `file:ota/ringtone`）→ Opus 解码播放。
- 音量：`bsp_audio_set_volume` 直控；设置页联动。

## 9. 屏幕与显示

- 屏幕栈：每应用一个 LVGL screen，压栈/弹栈由导航器管理。
- 事件覆盖层：`ring/red_alert` 时在栈顶压全屏弹窗（不销毁下层）。
- 240×320 内容约束：联系人行 = 头像(48) + 名称(省略号) + 状态灯(16)；正文摘要 ≤4 行，长文由 Hub 预截断。
- 亮度/息屏：`bsp_display_backlight(0..100)`；策略由电源管理实现（PRD 8.1）。

## 10. M1 固件交付清单

| # | 交付 | 验收 |
|---|---|---|
| 1 | 分区表落地 + LittleFS 初始化 | `idf.py build` 通过，`storage` 可读写 |
| 2 | boss_protocol（信封/seq/帧） | 与 Hub 跑通 hello/sync |
| 3 | boss_transport + usb 实现（TinyUSB 双 CDC） | 插线握手成功，CDC0 日志 + CDC1 协议并存 |
| 4 | boss_app 框架（导航/屏幕栈/事件层） | 模拟数据全 UI 流程可走通 |
| 5 | 电话簿/问答/来电/备忘录录音回放 | 演示脚本逐条过 |

| # | 交付 | 验收 |
| --- | --- | --- |
| 6 | 内存基线报告 | 空闲堆 ≥15%，写入 PR |

## 11. 风险

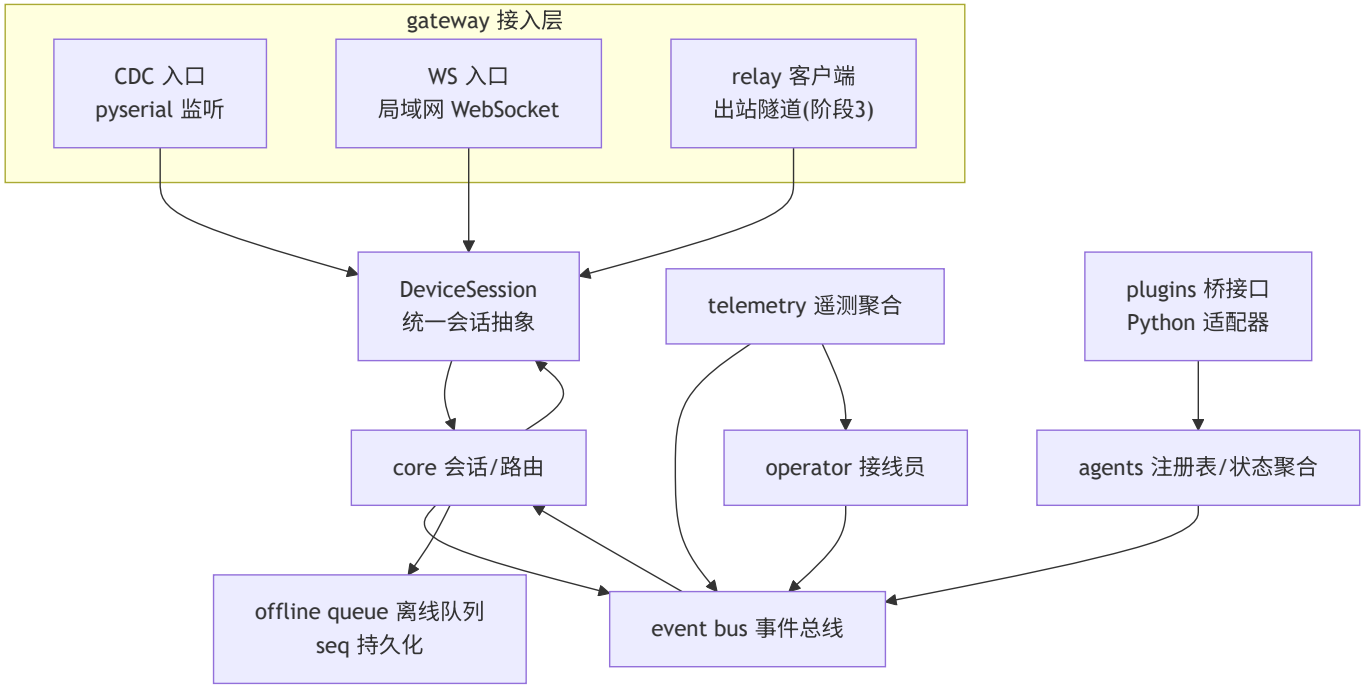
| # | 风险 | 应对 |
| --- | --- | --- |
| F1 | TinyUSB 与 USB-Serial/JTAG 模式互斥（同一 USB 控制器） | 双 CDC（CDC0 调试台+CDC1 协议）；开发期可切回 JTAG；烧录走 ROM 下载模式验证 |
| F2 | Opus 在 C3 的编码耗时/内存 | 实测 60ms 帧实时率；不足则降码率或换 codec2 |
| F3 | LittleFS 与 OTA 并发写 | 录音/写盘走 storage_task 串行；OTA 前校验空闲空间 |
| F4 | 深睡唤醒源限制 | 优先用 BLE 广播做唤醒（手机触发），RTC 定时兜底 |

# BOSS PHONE PC 总机房设计（Hub）

| 项目 | 内容 |
| --- | --- |
| 状态 | Draft v0.2 |
| 上游 | BOSS_PHONE_PRD.md §3/§4/§5.6 · DESIGN_OVERVIEW §7.2 |
| 技术栈 | Python 3.11+ / asyncio；常驻单进程；出站连接为主（可穿 CGNAT，无需公网 IP） |
| 日期 | 2026-08-30（v0.1 曾为 Rust+tokio，按计划书定为 Python） |

Hub 是整系统的中枢：接收多端连接（USB CDC / WebSocket / 云中继隧道）、维护设备会话、做链路路由决策、转发 Agent 事件、聚合遥测、调度接线员。

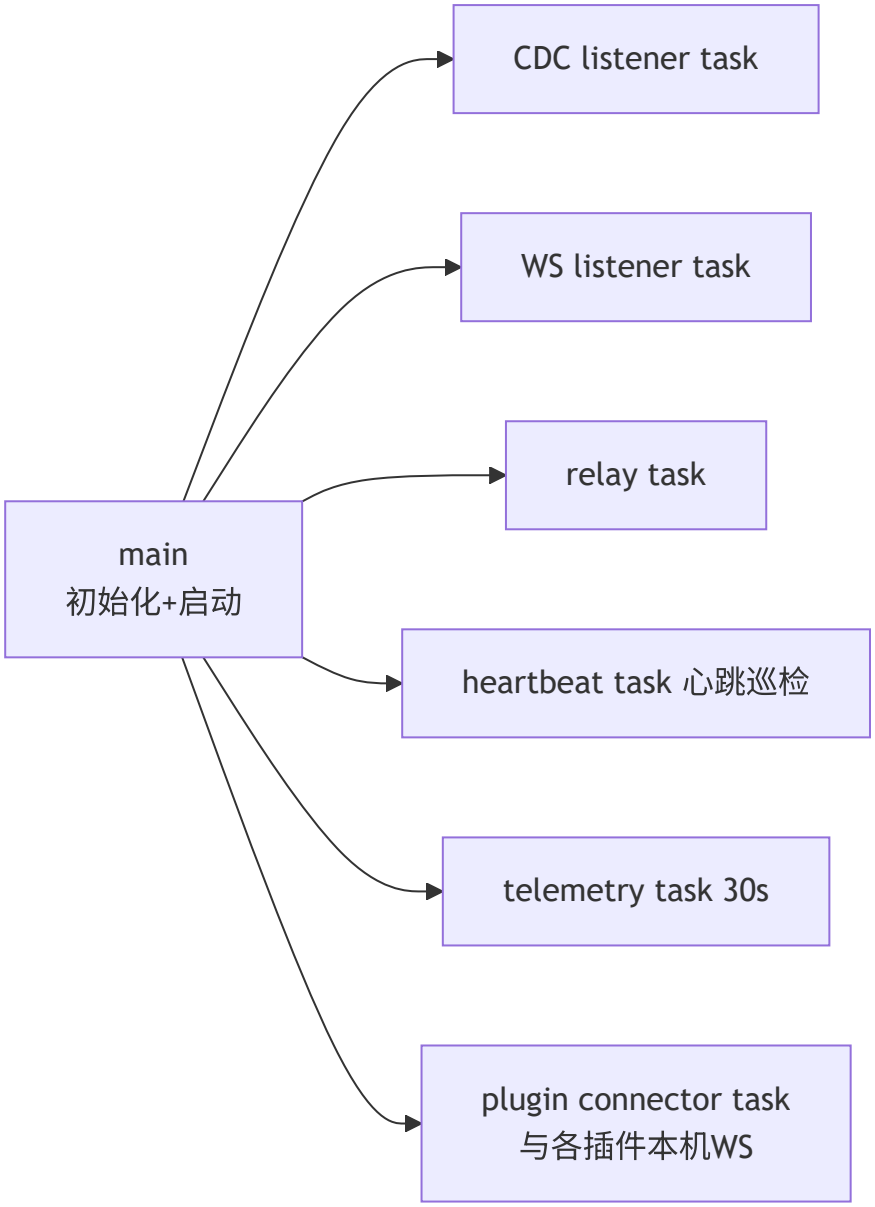
## 1. 模块架构




### 模块职责

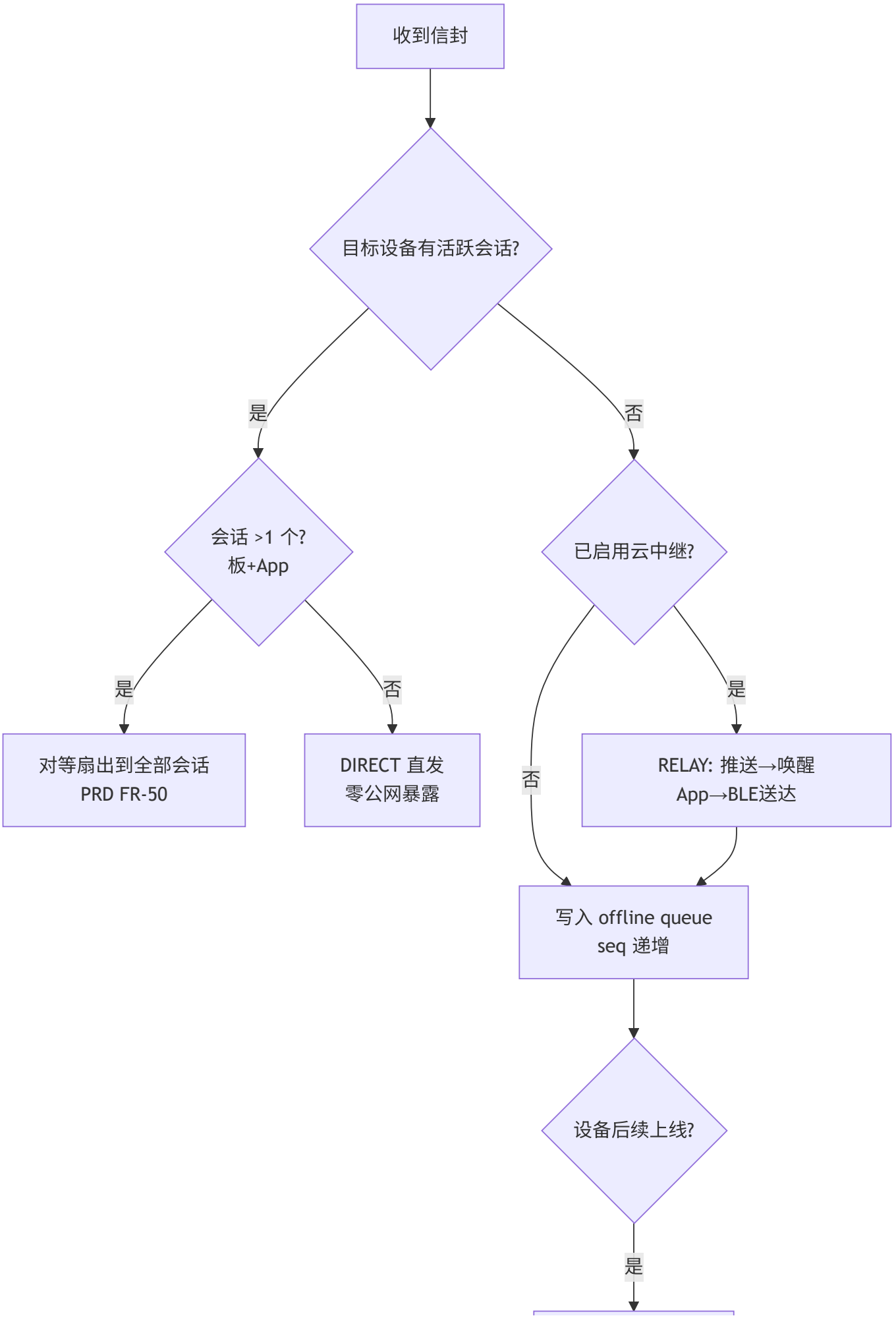
| 模块 | 职责 |
| --- | --- |
| gateway | 三入口监听（`pyserial` / `websockets` / `aiohttp` 隧道），各自解析帧 → 生成同一 `DeviceSession` |
| core/router | 心跳判定 + DIRECT/RELAY 路由决策（PRD §4.2） |
| event bus | 全内部分发（asyncio 事件总线）；消息顺序保证（按 from seq） |
| offline queue | 每设备 seq 递增，断线事件落盘（`queue/<device>.jsonl`），重连补发 |
| agents | 插件注册的 Agent 表：presence/state/ask 聚合，来电调度 |
| operator | 接线员：STT（faster-whisper / windows-SAPI / 云 API）+ 词表纠错 + LLM 意图路由 + 备忘转写/派发 |
| telemetry | PC 资源采集（`psutil` 30s）、token 聚合、commit/CI 查询（GitHub API） |

## 2. 进程模型（asyncio）




- 每接入一个会话 spawn 一个 session task；媒体（voice/audio）流量不进 event bus，走直连通道（避免序列化瓶颈）。
- 状态/遥测等低频消息经 event bus（asyncio 事件分发 + 每订阅者序号）。
- 说明：asyncio 单事件循环承载全部任务，CPU 密集工作（whisper 推理）放 `asyncio.to_thread` / 进程池，避免阻塞事件循环。

## 3. 路由决策（core/router）


收到信封
目标设备有活跃会话?
是
否
会话 >1 个?
板+App
已启用云中继?
是
否
是
否
对等扇出到全部会话
PRD FR-50
DIRECT 直发
零公网暴露
RELAY: 推送→唤醒
App→BLE送达
写入 offline queue
seq 递增
设备后续上线?
是

sync_data 补发

路由决策表：

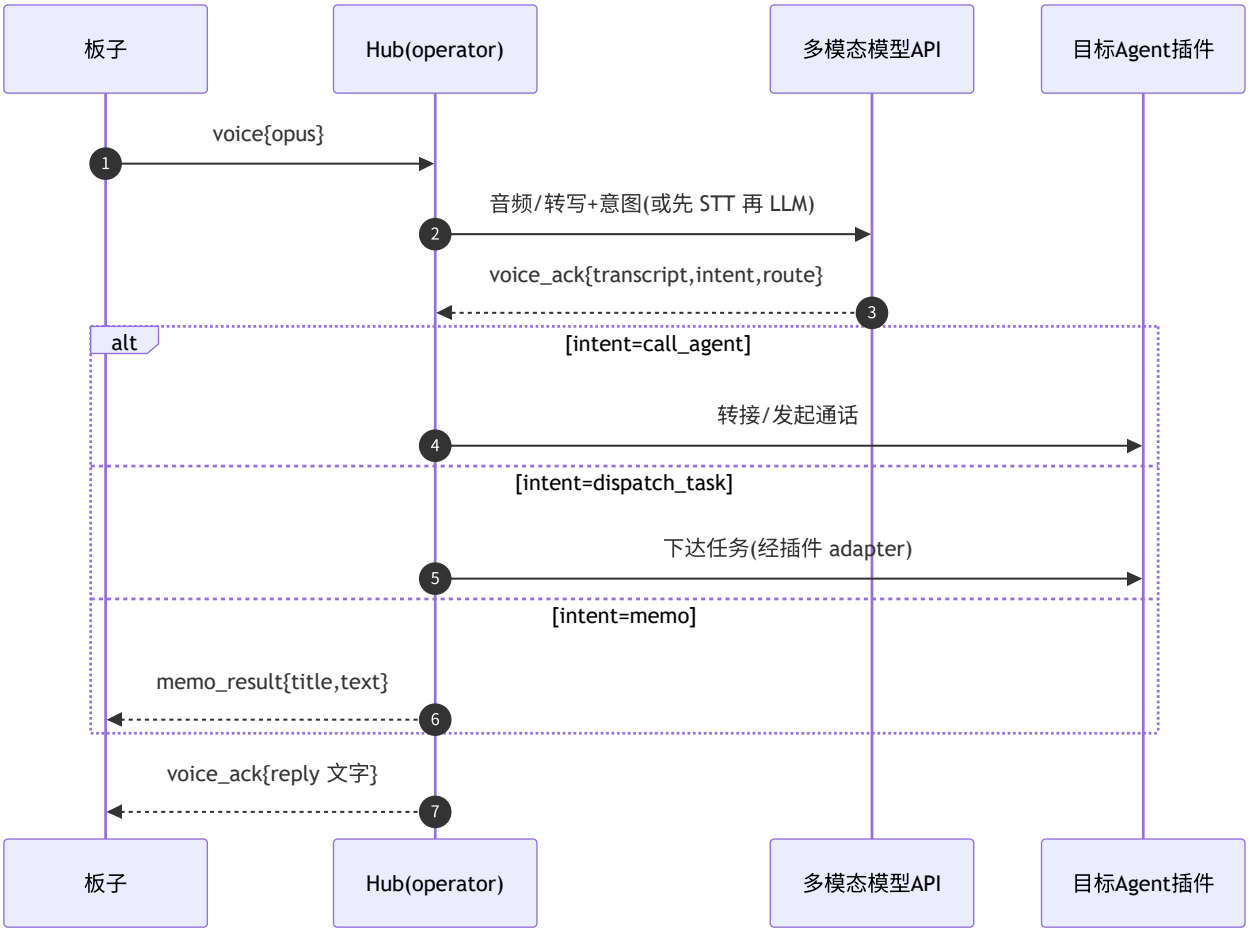
| 设备会话状态 | 行为 |
| --- | --- |
| USB CDC 会话活跃 | 最高优先级：一切走 USB（同时停无线推送） |
| WS 心跳正常（局域网） | DIRECT 直发；**不发公网推送（手机零负担）** |
| 全部失活 + 中继开启 | RELAY：消息→云中继→厂商推送→唤醒 App→BLE |
| 全部失活 + 中继关闭 | 写 offline queue，等设备回连 |

**心跳**：设备 10s 一个 ping（WS/CDC 各层）；Hub 30s 巡检，连续 2 次超时判失联。

## 4. 离线队列与 seq

- 每设备一个单调递增 `seq` 池（Hub 侧分配，持久化）。
- 断线事件追加写 `queue/<device>.jsonl`（含 seq）。
- 设备重连 hello 带 `resume_seq` → Hub 从 `resume_seq+1` 开始 `sync_data` 分页补发（每批 ≤64 条）。
- 板端确认 `sync{since_seq}` 后，Hub 可归档已确认段（保留 7 天轮转）。

## 5. 接线员（operator）




- 接线员本身是 Hub 内的一个任务，通过**可替换 provider 接口**接入模型（Q1 决策点）：
  - 多模态直听（audio→意图）

    - 两段式（STT→LLM）
- 备忘录转写与接线员共用 provider 以降成本（Q5 倾向）。
- **语音输入法（第一优先级，§OVERVIEW 8.6）**：STT 之后不是"展示文本"，而是经**词表纠错层**（`operator/correct`，无 LLM 依赖、JSON 热更新）修正专有名词后，由 `input/inject` **打到当前前台光标处**（Windows SendInput + 剪贴板兜底）。编排在 `dictation.py`；板子长按 OK 是触发端（M1 接 gateway）。

## 6. 遥测聚合（telemetry）

| 数据 | 采集方式 | 推送 |
| --- | --- | --- |
| PC CPU/内存 | `psutil` 30s | `telemetry` 周期；BLE 端自动降频 2min |
| 活跃会话数 | agents 注册表计数 | 同上 |
| Token 用量 | 各插件 usage 上报 → 聚合按日/月 | `usage` 变更即推 |
| commit | 插件上报 + 轮询 git | `commit` 事件 |
| CI 状态 | `gh` CLI / GitHub API 查询（每 commit 一次） | `ci` 事件；fail→red_alert |

- Token 统计精度：依插件 `usage-exact / usage-approx / none` 分级，面板标注可信度（R9 风险）。
- CI 失败联动：`ci=fail` 或 `state=blocked` → 触发 `ring(red_alert)` + 状态灯红。

## 7. 插件桥接口（plugins）

```
# plugins/core 提供的统一接入接口（Hub 侧视角，Python ABC）
from abc import ABC, abstractmethod

class AgentHandle(ABC):
    agent_id: str
    cli: str            # claude-code / codex / opencode / trae
    caps: list[str]     # AdapterCaps

    @abstractmethod
    async def send_ask(self, ask: dict) -> list[dict]: ...   # 权限问题组 → 回答列表
    @abstractmethod
    async def send_task(self, task: str) -> None: ...
    @abstractmethod
    async def notify(self, note: dict) -> None: ...
    @abstractmethod
    async def kill(self) -> None: ...                        # 重启会话(FR-49)
```

Hub 侧不感知 CLI 差异；差异全在插件适配器（见 DESIGN_PLUGIN）。

## 8. 配置与运行

- 配置文件 `hub.toml`：串口名/波特率、WS 端口、中继地址、模型 provider、STT 引擎选择、预算告警阈值、语音输入法（`dictation`：注入方式/阈值/纠错开关/用户词表/热词）。
- 命令（`hub/` 包，`python -m hub`）：
    - `python -m hub serve`（默认）
    - `python -m hub link <device>` 显示设备会话/seq/链路

    - `python -m hub telemetry` 打印遥测
    - `python hub/hub-lite.py <wav> --engine ... --type` 语音输入法入口（STT→纠错→注入光标处；`--selftest` 只验注入能力，见 DESIGN_OVERVIEW §8.6）
- 依赖：`pyserial`、`websockets`、`aiohttp`、`pydantic`、`psutil`；STT 按引擎选装 `faster-whisper` / `openai`。
- 常驻内存目标 ~60-80MB（§DESIGN_OVERVIEW 4.1）。

## 9. M1 Hub 交付清单

| # | 交付 | 验收 |
|---|---|---|
| 1 | CDC 入口（pyserial）+ DeviceSession | 板子插线 hello 握手成功 |
| 2 | core/router + event bus | 板↔Hub 状态流转通 |
| 3 | offline queue + sync | 断开/重连补发验证 |
| 4 | plugin connector（接 claude-code 适配器） | 真实状态上电话簿 |
| 5 | operator 骨架（provider 接口） | mock provider 回显 |
| 6 | 语音输入法：gateway 接 `voice.start`（长按 OK 触发）→ STT → 纠错 → 光标注入 | 板子长按 OK 说话，文字落到 PC 光标处 |

## 10. 风险

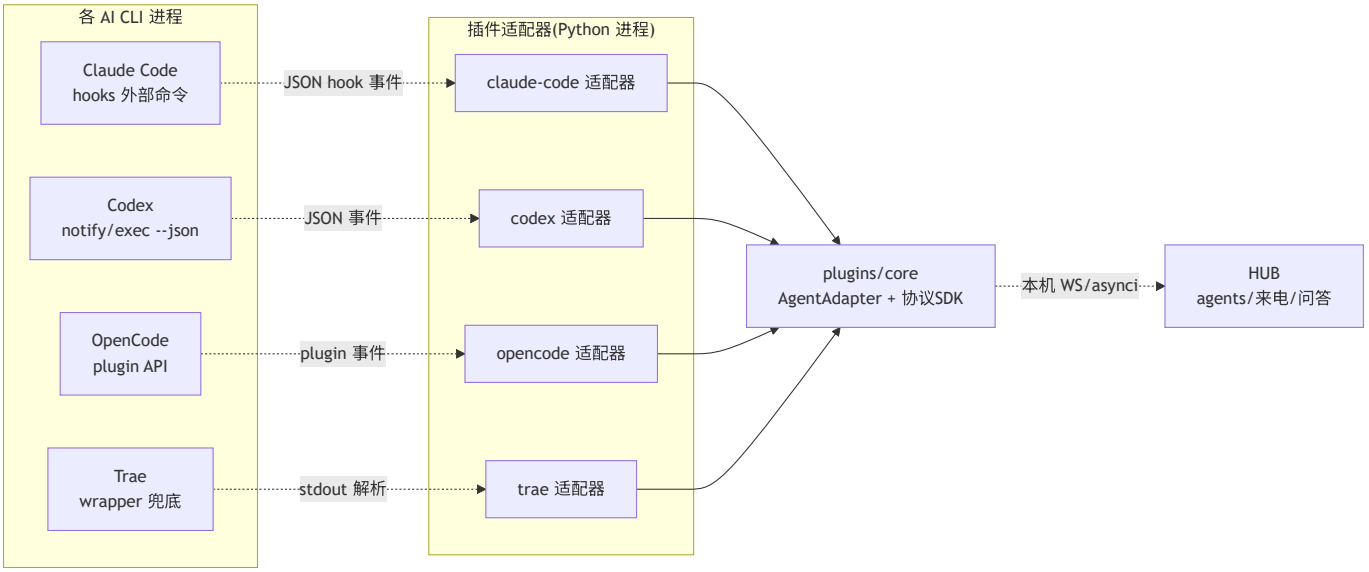
| # | 风险 | 应对 |
|---|---|---|
| H1 | 串口被占用（VS Code 串口监视器） | 检测占用并提示；提供虚拟 CDC 模拟模式供无板调试 |
| H2 | asyncio 任务数随设备增长 | 上限控制；媒体直连通道独立，不占 event bus |
| H3 | GitHub API 限流 | CI 查询缓存（5min）+ 请求合并 |
| H4 | 多模态模型延迟 | 语音段并发转写（to_thread/进程池）；UI 上"接线员处理中"状态 |

# BOSS PHONE CLI 插件桥设计

| 项目 | 内容 |
| --- | --- |
| 状态 | Draft v0.2 |
| 上游 | BOSS_PHONE_PRD.md §5.5 · DESIGN_OVERVIEW §5 |
| 技术栈 | Python 3.11+（plugins/core 共享包；各 CLI 一个适配器进程） |
| 日期 | 2026-08-30（v0.1 曾为 Rust，按计划书定为 Python，与 Hub 同语言） |

插件桥让各 AI CLI（Claude Code / Codex / OpenCode / Trae）接入 Boss Phone：上报状态、挂起等权限、主动提问、上报 usage/commit。**统一协议，能力降级**是核心策略。

## 1. 总体形态




## 2. 统一适配接口（plugins/core）

```
# plugins/core/adapter.py —— 设计骨架（Python ABC，与 Hub 同语言共享协议绑定）
from abc import ABC, abstractmethod
from enum import Enum


class CliKind(str, Enum):
    ClaudeCode = "claude-code"
    Codex = "codex"
    OpenCode = "opencode"
    Trae = "trae"


class UsagePrecision(str, Enum):
    exact = "exact"
    approx = "approx"
    none = "none"
```

```
class AdapterCaps:
    def __init__(self, *, status_report=False, permission_suspend=False,
                 proactive_ask=False, usage=UsagePrecision.none, commit_hook=False):
        self.status_report = status_report      # 状态上报 → 红黄绿灯
        self.permission_suspend = permission_suspend  # 能否挂起等用户回答（黄灯核心）
        self.proactive_ask = proactive_ask      # 主动提问
        self.usage = usage                      # exact / approx / none
        self.commit_hook = commit_hook          # 能否捕获 commit 事件


class AgentAdapter(ABC):
    agent_id: str           # 仓库名
    cli: CliKind
    caps: AdapterCaps       # 能力声明 → Hub 降级依据

    @abstractmethod
    async def start(self, hub_ws) -> None: ...          # 连上 Hub 本机 WS
    @abstractmethod
    async def stop(self) -> None: ...
    @abstractmethod
    async def send_ask(self, ask: dict) -> list[dict]: ...  # 权限问题组 → 回答列表
    @abstractmethod
    async def send_task(self, task: str) -> None: ...
    @abstractmethod
    async def notify(self, note: dict) -> None: ...
    @abstractmethod
    async def kill(self) -> None: ...                   # 重启会话（FR-49）
```

## 3. 能力矩阵与降级

| 能力 | Claude Code | Codex | OpenCode | Trae |
|---|---|---|---|---|
| status_report | ✅ hooks | ✅ notify | ✅ event | ⚠️ wrapper |
| **permission_suspend** | ✅ 强(PreToolUse 挂起) | ⚠️ 弱(notify 只通知) | ✅ 强(permission 事件) | ⚠️ 待调研 |
| proactive_ask | ✅ Notification | ✅ notify | ✅ event | ⚠️ |
| usage | exact | exact(token-count) | exact(消息级) | ⚠️ |
| commit_hook | ✅ | ✅ | ✅ | ⚠️ |

**Hub 降级规则**（依 `AdapterCaps` 执行）：

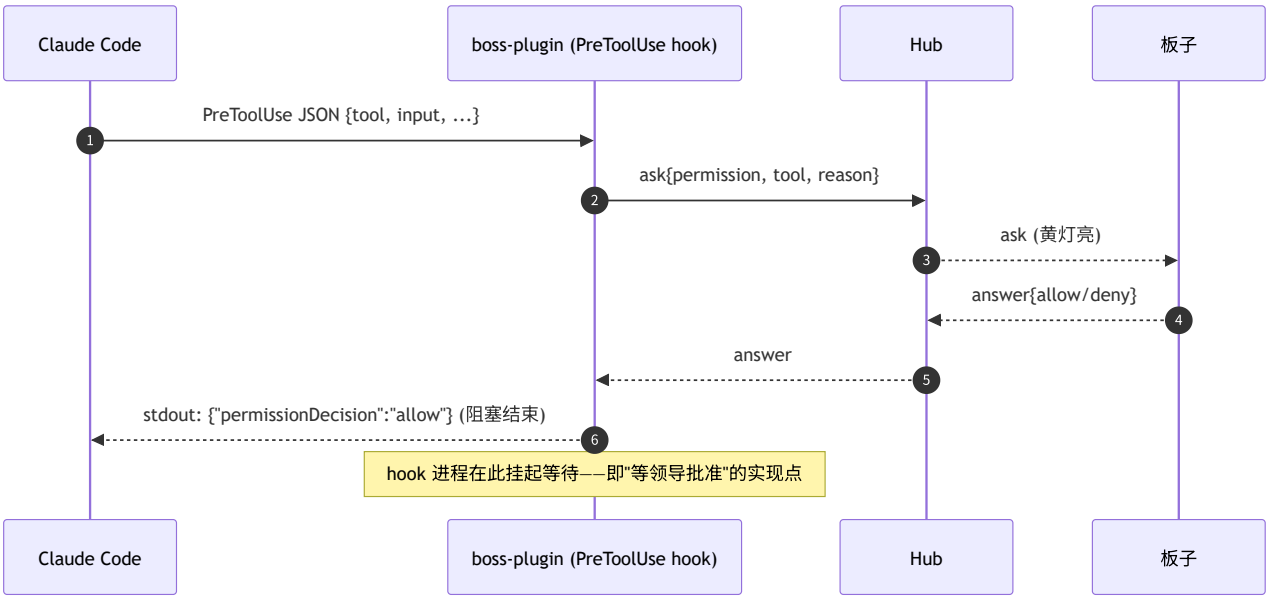
1. `permission_suspend=false`（Codex）：权限请求降级为**事后通知**——Agent 按自身 policy 执行，Hub 推"repo-B 已自动执行 X（策略 on-failure）"，黄灯退化为"通知灯"，用户可事后叫停（FR 补充）。
2. `usage=none`：Token 面板该 Agent 显示"不支持统计"。
3. `commit_hook=false`：由 Hub 定时 `git log` 兜底采集。

# 4. Claude Code 适配器（M1 打样）

## 4.1 hooks 配置（由安装器写入 `.claude/settings.json`）

```
{
  "hooks": {
    "SessionStart": [{ "hooks": [{ "type": "command", "command": "boss-plugin claude-code session-start" }]
}],
    "PreToolUse":   [{ "hooks": [{ "type": "command", "command": "boss-plugin claude-code pre-tool" }] }],
    "PostToolUse":  [{ "hooks": [{ "type": "command", "command": "boss-plugin claude-code post-tool" }] }],
    "Notification": [{ "hooks": [{ "type": "command", "command": "boss-plugin claude-code notify" }] }],
    "Stop":         [{ "hooks": [{ "type": "command", "command": "boss-plugin claude-code stop" }] }]
  }
}
```

## 4.2 权限挂起流程（"黄灯"核心实现）




- hook 进程在等 answer 期间**阻塞**（Claude Code 因此暂停工具执行），超时（默认 120s，可配）回 deny 并 notify 用户。
- 多选/输入题复用 §PROTOCOL 6.2 的 ask/answer 结构。

## 4.3 状态上报

- `SessionStart` → presence online；`Stop` → presence offline。
- `PostToolUse` 解析当前任务 → state `running` + task 摘要。
- `Notification`（agent 主动提示）→ proactive_ask → ring。
- token usage：hook 事件内 usage 字段 → Hub 聚合。

# 5. Codex 适配器

- 配置 `~/.codex/config.toml` 的 `[notify]` 指向 boss-plugin codex 入口（JSON 事件：`agent-turn-complete` / `token-count` / `update-available` 等）。
- `exec --json` 事件流解析任务进度 → state。

- **无挂起**：approval_policy 预设（`on-request` / `on-failure`）；`on-request` 模式依赖 Codex 自身的交互式确认——插件不拦截，仅转发事件到 Hub 做通知。降级规则见 §3-1。
- usage：`token-count` 事件 → exact。

## 6. OpenCode 适配器

- OpenCode 支持本地插件（Node/TS），但我们的桥是 Python——**采用其进程外事件协议**：插件侧用其官方 plugin SDK 写一个极薄桥（把事件转成 stdin/stdout JSON 交给 boss-plugin opencode 进程），避免全栈 TS。
- `permission` 事件可异步等待外部决定（我们由 Hub 回答后经桥回写）。
- 事件：`chat.message` / `tool.execute.start` / `tool.execute.end` / `permission`。

## 7. Trae 适配器（待调研，wrapper 兜底）

- 若 Trae 无开放 hook：外层 wrapper 进程启动 Trae，解析其 TUI 输出/exit code/日志推断状态。
- 能力降级：status_report=approx（解析日志）、permission_suspend=false、usage=none。

## 8. 安装与生命周期

- `boss-plugin install <cli> --repo <path> --hub <ws://127.0.0.1:port>`：写入对应 CLI 配置 + 注册到 Hub。
- `boss-plugin uninstall`：移除配置。
- 常驻：适配器进程与 Hub 本机 WS 长连；Hub 挂则指数退避重连。
- 安全：本机回环 WS + 简单 token 鉴权（DESIGN_OVERVIEW §8.3 对应）。

## 9. M1 交付清单

| # | 交付 | 验收 |
|---|---|---|
| 1 | plugins/core（AgentAdapter + 协议绑定） | `pytest` 通过，协议类型与 Hub 共享（`protocol/python`） |
| 2 | claude-code 适配器（hooks 五事件 + 挂起） | 真机：Claude Code 触发 PreToolUse → 板子批准 → 恢复执行 |
| 3 | codex 适配器（notify 降级） | 状态/usage 上报通，权限走通知灯 |
| 4 | 安装器（install/uninstall） | 一键配置 hooks 并注册 |

## 10. 风险

| # | 风险 | 应对 |
|---|---|---|
| P1 | Claude Code hooks JSON 结构随版本变动 | 版本锁定 + 结构校验 + 快速失败提示升级插件 |
| P2 | Codex "事后通知"体验弱 | UI 措辞：通知灯显示"已自动执行，可叫停"；后续推动 Codex 提供挂起机制 |
| P3 | 每个仓库一个适配器进程的资源开销 | 单进程多适配器实例共享事件循环与协议栈；空闲适配器进入轻量轮询 |
| P4 | Trae 能力不明 | 保持 wrapper 兜底设计；能力实测后回填矩阵 |

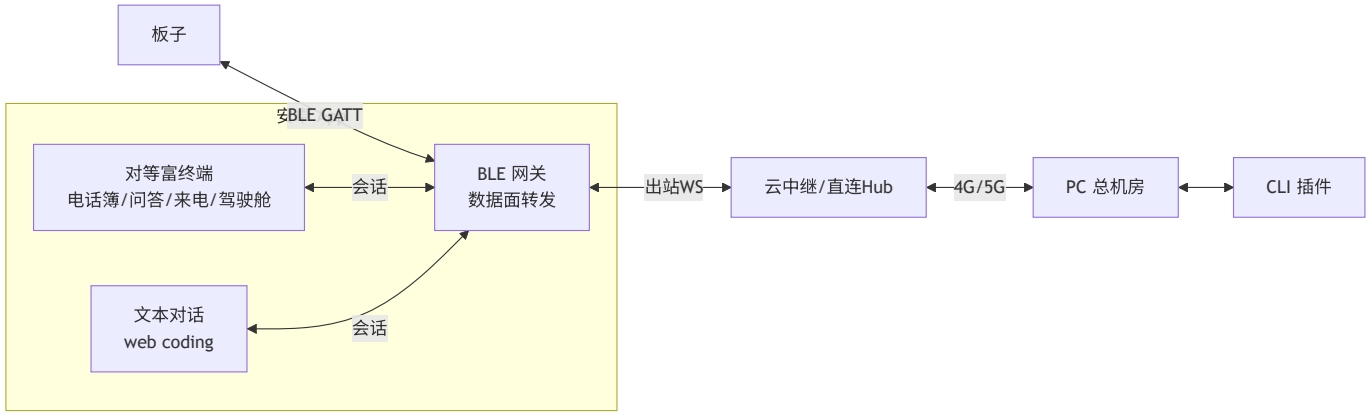
# BOSS PHONE 安卓 App 设计

| 项目 | 内容 |
| --- | --- |
| 状态 | Draft v0.1 |
| 上游 | BOSS_PHONE_PRD.md §3.2/§5.10/§4.3 · DESIGN_OVERVIEW §7.3 |
| 技术栈 | Kotlin + Jetpack Compose + 原生 BLE + kotlinx-serialization + Hilt |
| 日期 | 2026-08-24 |

App 的双重角色：**户外 BLE 口袋网关**（数据面，转发板子↔云中继）与**对等富终端**（web coding 客户端，与板子功能对等且更丰富）。

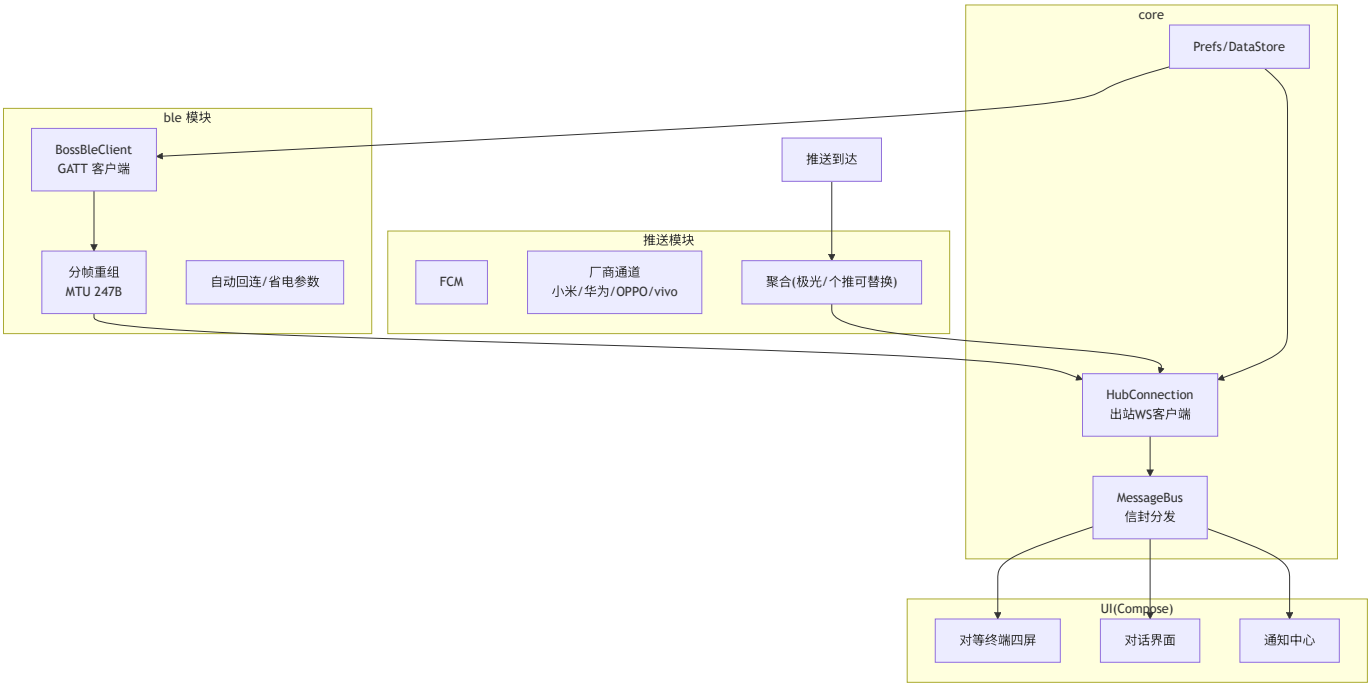
---

## 1. 双重角色




- **设备在局域网时**：App 不建 BLE、不收推送，**零负担**（PRD §4.3）。
- **设备离网时**：推送唤醒 App → 建 BLE 连接 → 成为数据面网关。

## 2. 模块划分



# 3. BLE 网关（数据面）

## 3.1 GATT 服务

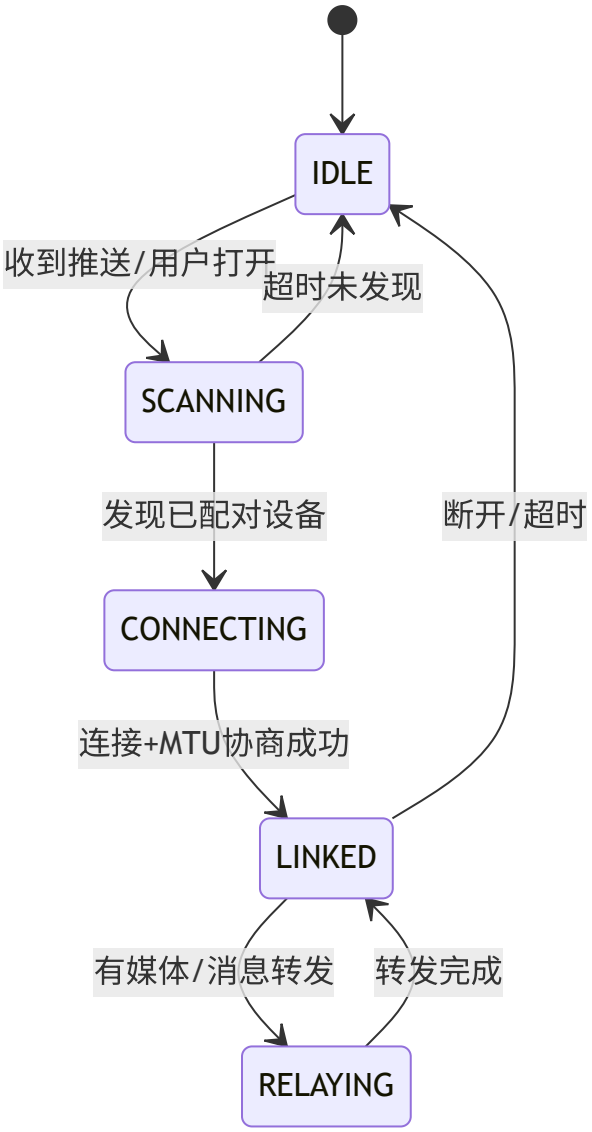
| 元素 | 值 |
| --- | --- |
| 服务 UUID | `0000FFFF-0000-1000-8000-00805F9B34FB`（自定义） |
| RX 特征值（App 收） | `0000FFF1-...`，Notify |
| TX 特征值（App 发） | `0000FFF2-...`，Write with Response |

## 3.2 分帧与可靠传输（PROTOCOL §4.1 BLE 分支细化）

```
App→板子 TX：FRAG(2B seq)|FRAG(2B 分片号)|payload ≤247B
板子→App RX：同格式 Notify
ACK：板子对每帧回 ACK(seq)，超时重传；窗口=4 帧
```

- 连接参数：交互态 interval 15-30ms，空闲态 100-500ms（supervision 10s），兼顾跟手与省电。
- 配对绑定：`BOND_BONDING`；App 记录板子 MAC，重启后自动回连。

### 3.3 BLE 网关状态机




## 4. 推送接入

- **FCM 主通道** + 各厂商通道（小米/华为/OPPO/vivo），经聚合 SDK（极光/个推）或自实现厂商 SDK 双通道。
- **推送只唤醒，不承载业务数据**：payload 仅 `{op:"wake", hub:"relay"}`；App 唤醒后建 BLE + 出站 WS 拉取真实消息（安全 + 免厂商长度限制）。
- 厂商后台限制：引导用户加自启动/后台白名单（应用内引导页）。
- 零负担原则：设备在局域网时 Hub 不触发推送（路由决策见 DESIGN_HUB §3）。

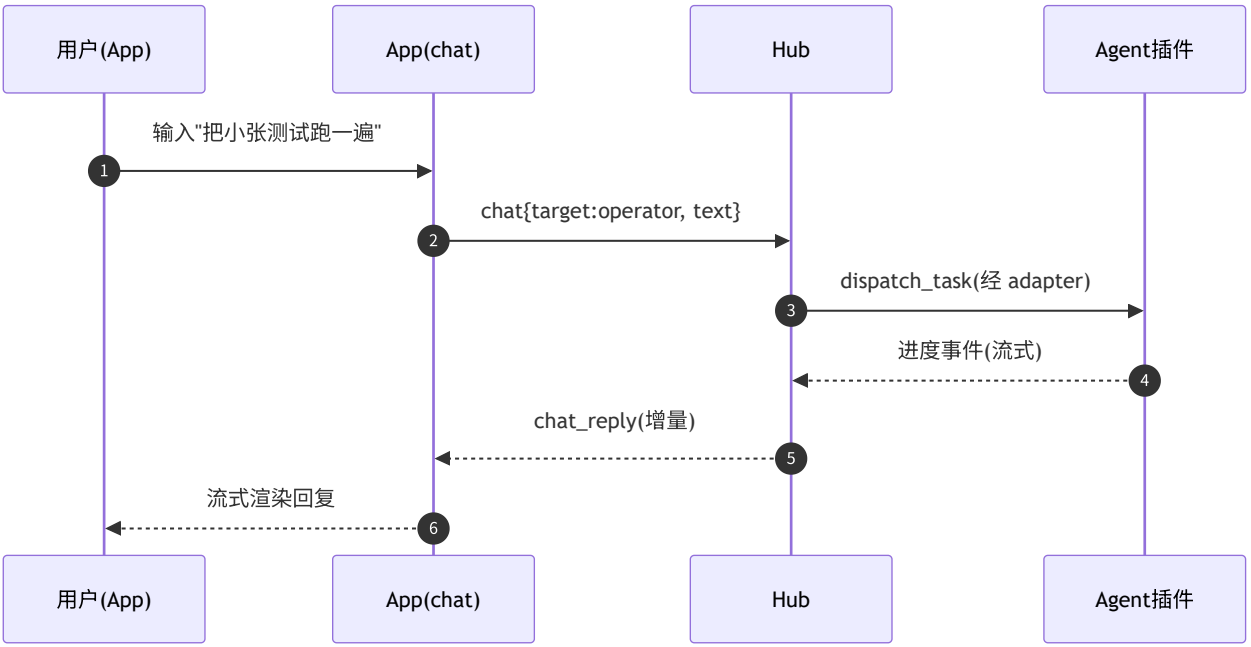
## 5. 对等富终端 UI（Compose）

### 5.1 终端四屏（与板子同构）

| 屏 | 说明 |
| --- | --- |
| 电话簿 | 触摸列表替代三键；联系人=头像+名称+状态灯；首项总机 |
| 权限问答 | 单选/多选/输入；输入框直接键盘输入（FR-52，语音替代） |
| 来电/报警 | 全屏弹窗 + 本地通知 + 可后台响应 |

| 屏 | 说明 |
| --- | --- |
| 总览驾驶舱 | 四面板（Agent/Token/PC/CI），触摸切页 |

### 5.2 文本对话（web coding，FR-51）




- 对话支持：@联系人、代码块高亮、diff 折叠、错误信息粘贴解析。
- 与板子关系：同一 ask 双端可答，先答先得（PROTOCOL §8）。
- App 登录/鉴权：配对 token 与 Hub 出站 WS 验证。

## 6. 通知中心（FR-54）

- 聚合：来电记录、报警历史、权限审批历史。
- 每类可回查详情；报警可一键转总览对应 Agent。

## 7. 前台服务与保活

- 数据面/网关活跃时启动前台服务（类型 `connectedDevice`），通知栏常驻图标。
- BLE 断开且无中继消息时自动降级后台，避免常驻耗电。

## 8. M3 App 交付清单

| # | 交付 | 验收 |
| --- | --- | --- |
| 1 | HubConnection（出站 WS）+ MessageBus | 与 Hub 握手/收状态 |
| 2 | 对等终端四屏（触摸版） | 局域网直连形态下与板子同步操作 |
| 3 | BLE 网关（GATT + 分帧 + 回连） | 板子 BLE_ONLINE 形态下消息双向可达 |
| 4 | 推送接入（FCM + 厂商 + 唤醒） | 设备离网 → 推送 → App 网关激活 → 板子收消息 |
| 5 | 对话界面（web coding） | 文本指挥 Agent 全链路 |

## 9. 风险

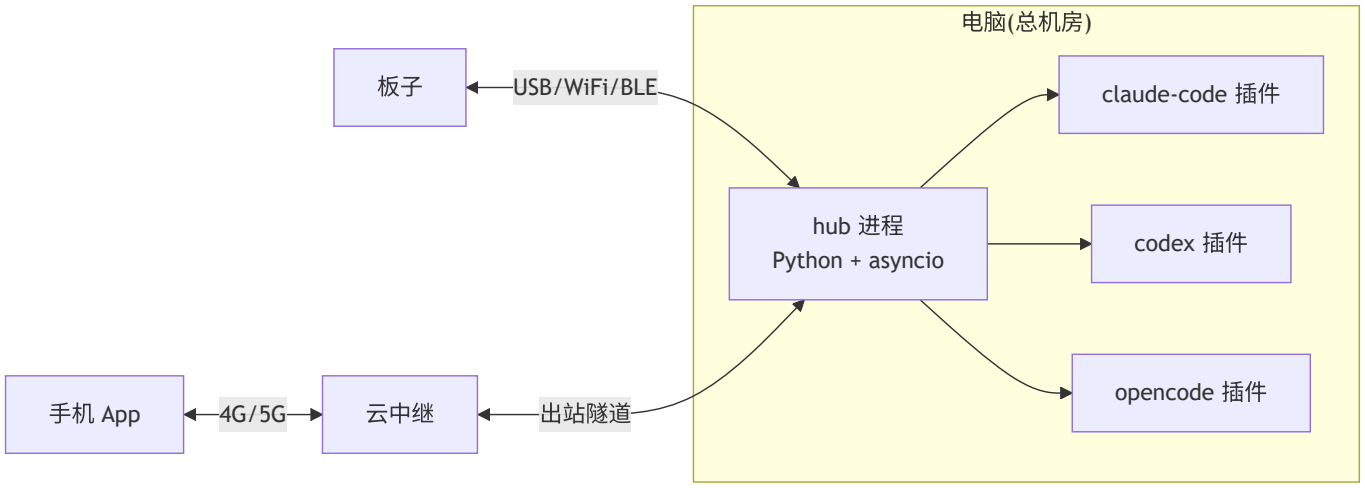
| # | 风险 | 应对 |
|---|---|---|
| A1 | 各厂商后台限制差异大 | 聚合 SDK + 引导白名单；B 端可考虑 MDM 预授权 |
| A2 | BLE 吞吐 vs 语音 | Opus 16k 对讲 + 半双工；codec2 兜底 |
| A3 | FCM 国内不可用 | 厂商通道优先，FCM 仅海外兜底 |
| A4 | 前台服务被系统冻结 | 短会话保活 + 断开即退；依赖推送唤醒而非常驻 |

# BOSS PHONE 部署与运维指南

| 项目 | 内容 |
| --- | --- |
| 状态 | Draft v0.1 |
| 上游 | BOSS_PHONE_PRD.md · DESIGN_HUB · DESIGN_PLUGIN |
| 日期 | 2026-08-24 |

本文覆盖从"拿到硬件"到"稳定运行"的全过程：PC 端部署、插件安装、设备配对配网、升级、运维与故障排查。

## 1. 部署拓扑与所需软件




| 端 | 需要部署 | 可选 |
| --- | --- | --- |
| PC | Python 3.11+ 运行时 + hub 包（`pip install`）；`boss-plugin` 安装器；Claude Code/Codex/OpenCode 本体 | 云中继客户端（阶段 3） |
| 板子 | 烧录 Boss Phone 固件（M1 起）；首次 USB 配网 | — |
| 手机 | Boss Phone App；厂商推送通道授权 | — |
| 云端 | 云中继服务（自建 frp/WS 或第三方） | 阶段 3 |

## 2. Hub 部署（PC 端）

### 2.1 安装

```
# 1) 安装 Python 3.11+（Windows 安装器勾选 "Add to PATH"）
# 2) 创建虚拟环境并安装依赖
python -m venv .venv
.venv\Scripts\activate            # Windows；Linux/macOS: source .venv/bin/activate
pip install -r hub/requirements.txt
```

```
# 3) 安装 hub 包（可编辑模式，源码更新即生效）
pip install -e hub/
```

## 2.2 配置 hub.toml

```
[serial]
port = "COM5"            # 板子 CDC1 串口；留空则纯 WS 模式
baud  = 115200           # CDC 虚拟串口波特率（可任意，常为 115200）

[ws]
listen = "0.0.0.0:8765"  # 局域网 WebSocket 入口
token  = "change-me"     # 设备/插件连接鉴权（DESIGN_SECURITY）

[relay]                  # 阶段 3
enabled = false
server  = "wss://relay.example.com"
token   = "..."

[operator]
provider = "openai"      # 或 deepseek/qwen...
model    = "gpt-4o-audio-preview"  # 多模态直听；两段式则填 STT+LLM 两模型
api_key  = "..."

[telemetry]
interval_sec = 30
github_token = ""        # CI 查询用（可选，留空走 gh CLI）
budget_month = 500       # token 预算告警(元)
```

## 2.3 运行与自启

| 平台 | 方式 |
| --- | --- |
| Windows | 计划任务开机自启；或 NSSM 服务化 |
| Linux/macOS | systemd unit（示例见下）/ launchd |

```
# /etc/systemd/system/boss-hub.service
[Unit]
Description=Boss Phone Hub
After=network.target
[Service]
ExecStart=/opt/boss/.venv/bin/python -m hub serve --config /opt/boss/hub.toml
Restart=on-failure
[Install]
WantedBy=multi-user.target
```

## 2.4 无板调试模式

`python -m hub serve --virtual-cdc`：虚拟一个串口设备，配合模拟板子脚本（`python -m hub sim`）可脱离硬件开发调试。

---

## 3. 插件安装（PC 端）

```
boss-plugin install claude-code --repo C:\repos\repo-a --hub ws://127.0.0.1:8765
boss-plugin install codex       --repo C:\repos\repo-b --hub ws://127.0.0.1:8765
boss-plugin install opencode    --repo C:\repos\repo-c --hub ws://127.0.0.1:8765
boss-plugin list                # 查看已注册 Agent
boss-plugin uninstall repo-b    # 卸载
```

- 安装器自动写入各 CLI 配置（Claude Code `.claude/settings.json` hooks、Codex `config.toml` notify、OpenCode 插件目录）。
- 验证：`boss-plugin status repo-a` 应显示 `connected`，且 Hub `hub link` 能看到该 Agent。
- 故障：hook 路径写错 → 以 `boss-plugin doctor` 诊断。

---

## 4. 设备配对与配网

### 4.1 板子首次接线

1. Type-C 连接 PC，板子进入 DESK_USB：CDC0=调试台（日志），CDC1=协议传输。
2. `hub serve` 检测到 CDC1 → 自动 hello 握手。
3. 无 USB 时：预配置的 WiFi 连接（设置→链路→配网），或经 BLE 由手机 App 配网（阶段 3）。

### 4.2 配对码（首次连接）

- USB 首连：板子屏幕显示 6 位配对码，Hub 端 `hub pair <code>` 确认，绑定后写入 nvs。
- BLE 配对：App 与板子走系统级 `BOND_BONDING`，绑定一次后续自动回连。
- 云中继首次注册：PC 生成一次性配对 token，App/板子扫码/输入后绑定。

### 4.3 WiFi 配网（阶段 2）

| 方式 | 说明 |
| --- | --- |
| USB 配网 | 设置页填 SSID/密码，经 CDC1 下发并保存 nvs |
| BLE 配网 | App 扫描到板子广播 → 下发 WiFi 凭据（阶段 3） |

---

## 5. App 部署（手机）

1. 安装 APK（侧载或应用市场）。
2. 首次启动 → 授权：通知权限、定位（BLE 扫描所需，Android 12+ 附近设备权限）、后台运行白名单。
3. 厂商推送：小米/华为/OPPO/vivo 按引导开启"自启动/后台运行/通知"，否则离网唤醒不可达（DESIGN_ANDROID §4）。
4. 云中继配对：输入 PC 生成的配对 token。
5. 验证：局域网内打开 App → 电话簿应显示与板子一致的 Agent 列表。

---

## 6. 升级流程

| 对象 | 链路 | 说明 |
|---|---|---|
| 板子固件 | USB 优先 / WiFi OTA / 中继暂存 OTA | OTA A/B 双分区（DESIGN_FIRMWARE §2）；升级后自动回滚校验失败 |
| Hub | 更新代码 + 重启（systemd 平滑） | 协议只增不改，老固件不失效 |
| 插件 | 更新 boss-plugin 包 + `install --upgrade` | 与 Hub 版本兼容校验 |
| App | 应用商店/侧载 | 与 Hub 协议协商降级 |

**升级铁律**：先升级 Hub → 再插件 → 最后固件/App；任何一端失败不阻塞其他端（hello 协商降级）。

---

## 7. 运维

- **健康检查**：`hub telemetry`（CPU/内存/会话）；`hub link`（各设备链路+seq）。
- **日志**：Hub 标准输出结构化日志；板子 CDC0 调试台；App logcat 过滤 `BossPhone`。
- **备份**：`hub backup` 导出 offline queue 与 Agent 注册表（JSON）。
- **告警阈值**：token 预算 80%/100% 两级告警（设置→总览面板 2 可看）。

---

## 8. 故障排查速查表

| 现象 | 可能原因 | 处理 |
|---|---|---|
| 插线后板子不进 USB 模式 | TinyUSB 未识别/端口占用 | CDC1 端口是否被串口监视器占用；`hub doctor serial` |
| 板子 WS 连不上 | token 不匹配 / 端口错误 | 核对 `hub.toml` token 与设备 nvs；`hub link` |
| Agent 黄灯一直不消 | 插件挂起超时/掉线 | `boss-plugin status`；超时默认 120s 自动 deny |
| CI 不刷新 | GitHub API 限流 | 等缓存(5min)；`telemetry.github_token` 配置 |
| App 收不到离网推送 | 厂商后台限制 / FCM 不可用 | 检查白名单；换厂商通道 |
| BLE 连不上板子 | 配对被清 / 距离 | 重新 BOND；查 App 日志 |
| 内存不足（板子重启） | LVGL 缓冲过大 | 查空闲堆基线；回退缓冲配置 |

---

## 9. 待定项

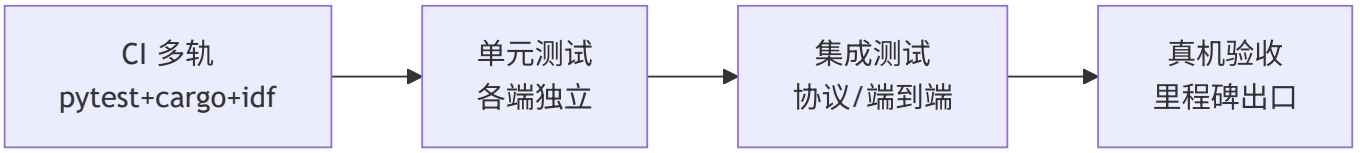
| # | 事项 |
|---|---|
| D1 | Windows 服务化与更新器（自动下载新版本） |
| D2 | 多 PC 迁移（导出/导入整机配置） |
| D3 | 云中继高可用（阶段 3 设计） |

# BOSS PHONE 测试与验收方案

| 项目 | 内容 |
| --- | --- |
| 状态 | Draft v0.1 |
| 上游 | BOSS_PHONE_PRD.md §9 里程碑 · README 验收清单 |
| 日期 | 2026-08-24 |

本文把 PRD 的里程碑出口标准展开成**可执行验收清单**与**真机测试矩阵**。遵循 AGENTS.md：`idf.py build` 是最小检查，硬件功能必须真机逐项验证并记录结果。

## 1. 测试分层




| 层 | 范围 | 环境 |
| --- | --- | --- |
| 单元 | 协议编解码、seq 去重、分帧、路由决策、适配器解析 | 各端 CI |
| 集成 | 协议一致性（C/Rust/Python/Kotlin 互操作）、Hub↔插件、端↔Hub | 无板（虚拟 CDC + 模拟 Agent） |
| 真机验收 | 每个里程碑出口标准 | 真实硬件 + 真实 CLI |

## 2. CI 多轨

| 轨 | 触发 | 内容 |
| --- | --- | --- |
| `ci-firmware` | push 到 `main` / PR | `idf.py build`（ESP-IDF 5.5.x），检查零 warning |
| `ci-rust` | 同 | `cargo test`（固件 Rust 业务层 workspace），协议一致性测试 |
| `ci-hub` | 同 | `pytest`（hub + plugins Python 包），协议一致性测试 |
| `ci-android` | 同（M3 起） | `./gradlew test assembleDebug` |

> 协议 Schema 变更必须四端生成物同步提交，否则 CI 失败（DESIGN_OVERVIEW §6）。

## 3. M1 验收清单（骨架版•USB）

| # | 项 | 操作 | 通过标准 |
| --- | --- | --- | --- |
| M1-1 | 分区与存储 | 烧录后 `hub doctor` | storage(LittleFS) 可读写，空间 3.9MB |
| M1-2 | USB 握手 | 插线开 Hub | hello/hello_ack 成功；CDC0 日志 + CDC1 协议并存 |
| M1-3 | 电话簿 | 注册 2 个模拟 Agent | 联系人头像+名称+绿灯显示，▲▼ 可切换 |
| M1-4 | 状态灯 | 推送 state 变化 | 绿→黄→红实时变化，无轮询 |

| # | 项 | 操作 | 通过标准 |
|---|---|---|---|
| M1-5 | 权限问答 | 下发含单选+多选+输入的 ask | 单选 OK 自动跳题；多选长按切题；输入长按 OK 语音 |
| M1-6 | 来电报警 | 下发 ring(red_alert) | 扬声器响铃 + 事件层全屏弹窗 + 阻塞摘要 |
| M1-7 | 备忘录 | 录音→回放→列表 | 录音落盘、回放正常、显示时长 |
| M1-8 | 亮度管理 | 设置页调亮度/息屏 | 背光变化；超时息屏；按键即亮；USB 常亮 |
| M1-9 | 端到端 | 真实 Claude Code 触发 PreToolUse | 板子批准 → 恢复执行（DESIGN_PLUGIN §4.2） |
| M1-10 | 内存 | 长跑 1h | 空闲堆 ≥15%，无泄漏增长 |

## 4. M2 验收清单（无线+语音）

| # | 项 | 通过标准 |
|---|---|---|
| M2-1 | WiFi 传输 | 离开 USB，局域网 WS 连上，状态刷新 <1s |
| M2-2 | 对讲 | 长按 OK 说话 → 接线员转写回显 → 意图路由正确 |
| M2-3 | 语音指令 | "喂，让小张跑测试" → 目标 Agent 收到任务 |
| M2-4 | 备忘录转写 | 录音同步 → STT 回填标题/正文 → 列表标"已转写" |
| M2-5 | 备忘派发 | 已转写备忘 → 派发 → 目标 Agent 收到任务，状态回写 |
| M2-6 | 遥测四面板 | Token/PC 资源/提交与 CI 实时刷新；CI 失败触发红灯+报警 |
| M2-7 | 接线员容错 | 无多模态模型时走 STT 两段式，功能不降级 |

## 5. M3 验收清单（户外+富终端）

| # | 项 | 通过标准 |
|---|---|---|
| M3-1 | BLE 网关 | 出门 WiFi 断开 → BLE 回连手机 → 消息双向可达 <3s |
| M3-2 | 推送唤醒 | 设备离网 → 云中继 → 厂商推送 → App 激活网关 → 板子收消息 |
| M3-3 | 三端联动 | 回家 WiFi 直连后手机零推送（抓包确认无中继流量） |
| M3-4 | 离线队列 | 全离线期间事件不丢；回连 sync 补收，去重无重复 |
| M3-5 | 富终端四屏 | App 触摸版电话簿/问答/来电/驾驶舱与板子同步操作 |
| M3-6 | web coding | App 文本对话指挥 Agent，流式回复渲染 |
| M3-7 | 先答先得 | 板子与 App 同 ask 双端竞答，后答端收到 answer_done |
| M3-8 | 电量 | 户外 BLE 模式日均 <10mA 实测记录 |

## 6. M4 验收清单（LTE 板）

| # | 项 | 通过标准 |
|---|---|---|
| M4-1 | transport_lte | 新板插 SIM 即用，hello 声明 `relay+lte` |
| M4-2 | 真人联系人 | type:human 走模块语音通道，UI 与 Agent 通话统一 |

| # | 项 | 通过标准 |
| --- | --- | --- |
| M4-3 | 老板子回归 | 老固件不受影响，协议只增不改验证 |

## 7. 真机硬件测试矩阵

> 每项记录结果到 PR（AGENTS.md 要求），含照片/截图。

| 硬件 | 测试项 | 覆盖里程碑 |
| --- | --- | --- |
| 显示 | 亮度档位/息屏/来电点亮；无花屏、无残影 | M1 |
| 按键 | 短按/长按语义全表（PRD §6.1）；长按 ▲ 全局后退 | M1 |
| 音频 | 对讲回声/延迟；响铃音量；录音无爆音；音量控制 | M1/M2 |
| 电池 | 四档电源策略实测续航（全亮/低亮/息屏/深睡） | M1/M3 |
| USB | 双 CDC 并存；烧录仍可用；充电时屏幕常亮 | M1 |
| WiFi | 断线重连；外网故障时局域网直连不中断 | M2 |
| BLE | 配对绑定；自动回连；信号弱丢帧重传 | M3 |
| 存储 | 备忘录写满 3.9MB 行为；OTA 前后空间检查 | M1/M2 |

## 8. 回归与兼容测试

- **协议只增不改回归**：用 v1 固件连接 v2 Hub、v2 固件连接 v1 Hub，均须可用（hello 协商降级）。
- **内存回归**：每次固件改动跑空闲堆基线对比。
- **链路切换回归**：USB→WiFi→BLE→深睡往返，消息零丢失（seq 校验）。
- **双端并发回归**：板 + App 同时在线，ask 竞答、状态一致。

## 9. 无板开发环境

| 工具 | 用途 |
| --- | --- |
| `hub sim` | 模拟板子：虚拟 CDC + 脚本化收发，跑集成用例 |
| 模拟 Agent | 按插件协议回放 presence/state/ask，测 Hub 与 UI |
| `tools/pdf-export` | 文档回归（非功能） |

# BOSS PHONE 安全设计

| 项目 | 内容 |
| --- | --- |
| 状态 | Draft v0.1 |
| 上游 | BOSS_PHONE_DESIGN_HUB · DESIGN_ANDROID · DESIGN_FIRMWARE |
| 日期 | 2026-08-24 |

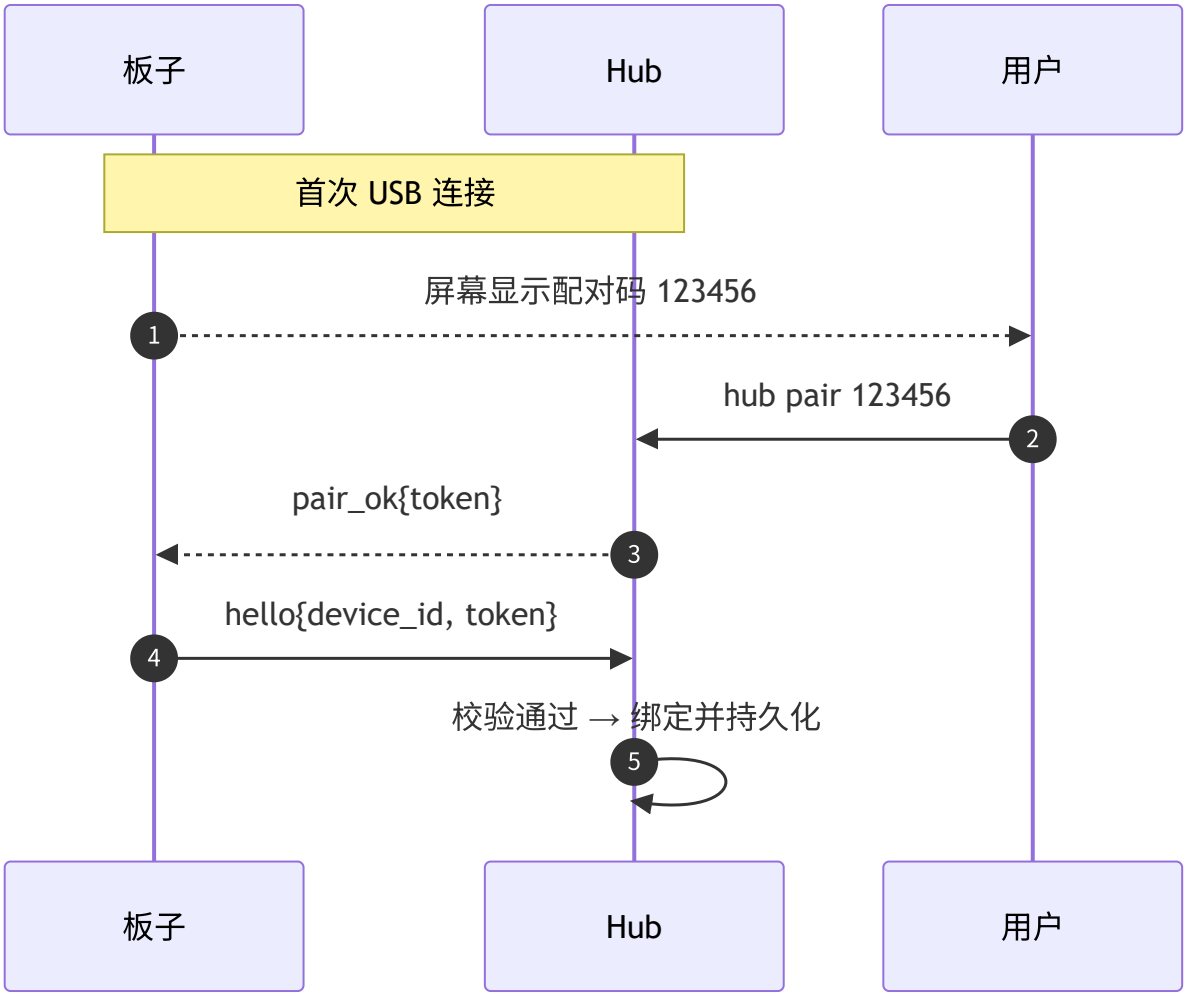
产品核心安全价值：**局域网/USB 形态零公网暴露**；公网形态只经出站隧道与加密链路。本文给出威胁模型与各层控制。

## 1. 威胁模型

| # | 威胁 | 攻击面 | 等级 |
| --- | --- | --- | --- |
| T1 | 局域网内他人冒充设备/Hub | WS 接入 | 高 |
| T2 | 工位邻居 USB 插线冒充 | CDC 接入 | 中 |
| T3 | BLE 范围内窃听/伪装 | BLE 链路 | 中 |
| T4 | 云中继被攻破/中间人 | 公网隧道 | 中 |
| T5 | 恶意 CLI 插件上报假状态/诱导批准 | 插件桥 | 高 |
| T6 | 隐私泄露（仓库名/任务/录音） | 所有链路+日志 | 中 |
| T7 | 固件/OTA 被篡改 | OTA 链路 | 中 |

## 2. 身份与鉴权

| 层 | 机制 | 说明 |
| --- | --- | --- |
| 设备标识 | `device_id`（hello 携带，USB 绑定写入 nvs） | 唯一身份，不可伪造（nvs 受保护） |
| 设备↔Hub token | `hub.toml` 的 `ws.token`；设备 nvs 保存 | 连接鉴权；首次由配对码建立（见下） |
| 配对码 | USB 首连：板子屏幕 6 位码，`hub pair <code>` | 防 T2：工位邻居插线无法通过 |
| 插件 token | 插件安装器生成，写入 CLI 配置 + Hub | 防 T5 冒名插件 |
| App token | 云中继配对 token（一次性） | 防 T4 越权 |




## 3. 链路安全

| 链路 | 安全措施 |
| --- | --- |
| USB (CDC1) | 本地物理访问 + 配对码（T2） |
| WiFi 局域网 WS | 环回/局域网内 WS + token；**不暴露公网**；可选 TLS（自签） |
| BLE | 系统级 `BOND_BONDING`（配对绑定 + 链路加密）；分帧含校验 |
| 云中继 | 出站 `wss://`（TLS）+ token；服务端不留业务 payload 明文（脱敏） |
| OTA | 镜像签名校验（esp_secure_boot/签名验证）；失败自动回滚 |

**默认安全态**：局域网内 Hub 监听仅绑定局域网接口并校验 token；云中继默认关闭，用户显式开启（阶段 3）。

## 4. 隐私与数据最小化

| 数据 | 处理 |
| --- | --- |
| 仓库名/Agent 名 | 电话簿展示必需；中继转发时信封含，payload 可配脱敏 |
| 任务摘要/状态 | PC 端生成，**只传摘要不传全文**（PRD FR-17） |
| 权限请求内容 | 必要字段最小集（工具名+参数摘要） |

| 数据 | 处理 |
| --- | --- |
| 录音（备忘/对讲） | 传输走 Opus 压缩；云端不留存；本地上传即删可配 |
| Token 用量 | 聚合统计，不含 prompt/响应内容 |
| 日志 | 默认不落 payload 明文；`hub.toml` 可开 `debug_payload`（本地排障） |

**隐私原则**：

1. 摘要化：一切语义内容 PC 端本地生成。
2. 零接触：局域网/USB 形态不经任何第三方。
3. 可配置脱敏：中继开启时默认启用 `sanitize`（仓库名可替换为代号）。

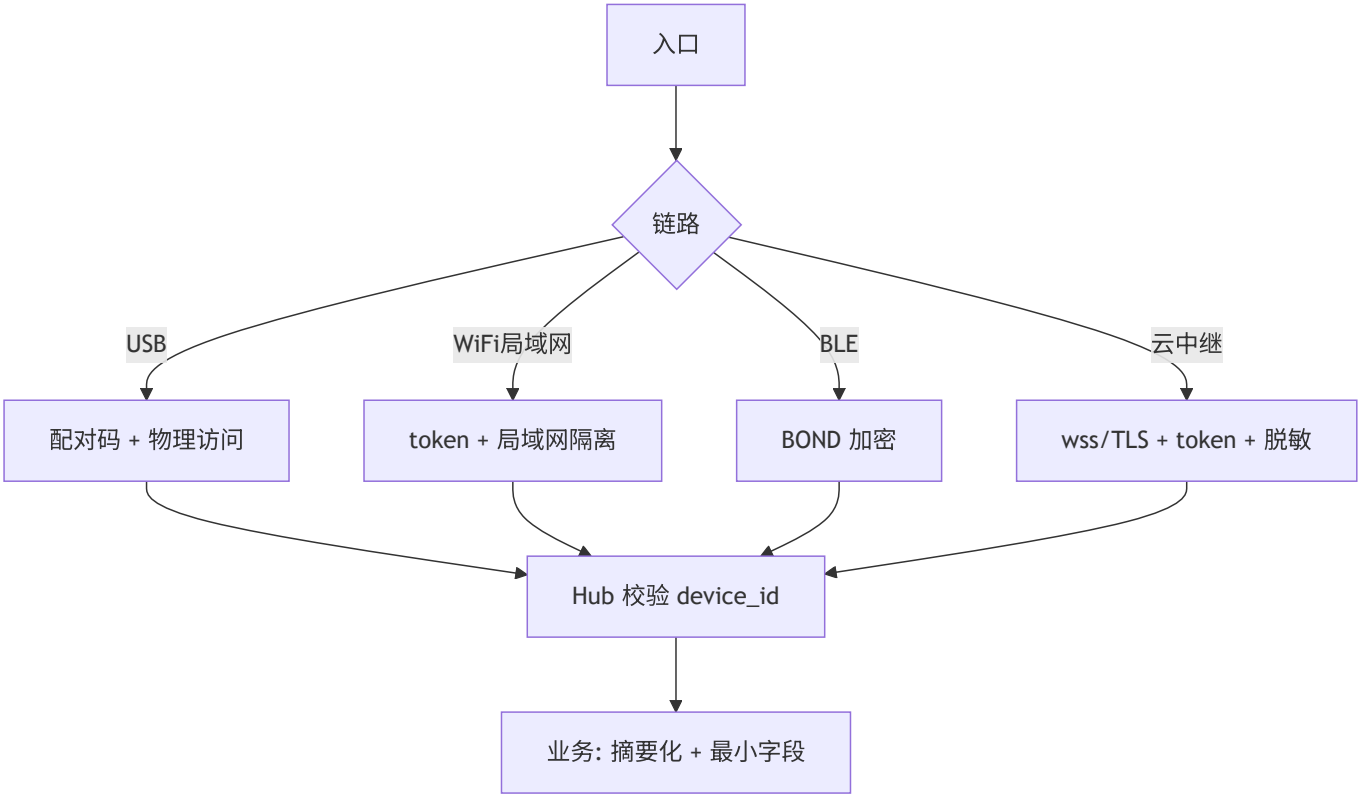
---

## 5. 权限最小化与供应链

- **插件最小权限**：插件只能读写自己仓库目录 + 与 Hub 通信；安装时校验 checksum（`boss-plugin install` 从固定发布源下载）。
- **Hub 权限**：仅本机回环/局域网监听；不申请管理员权限。
- **App 权限最小化**：仅通知、BLE 扫描、网络；不申请通讯录/短信等无关权限。
- **固件**：nvs 不存明文密钥（token 经 esp_partition 加密区可配）。

---

## 6. 安全矩阵汇总




---

## 7. 待定项

| # | 事项 |
|---|---|
| S1 | 云中继是否引入设备级端到端加密（payload E2E） |
| S2 | OTA 签名体系选型（esp_secure_boot 依赖 C3 支持度） |
| S3 | 局域网 WS 是否默认 TLS（自签证书分发成本） |